

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите
Руководитель ОП
_____ А.В. Щукин
« _____ » _____ 202_ г.

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
РАБОТА БАКАЛАВРА**

**РАЗРАБОТКА ПЛАГИНА СОВМЕСТНОЙ ЭЛЕКТРОННОЙ ТАБЛИЦЫ В
MOODLE**

по направлению подготовки 09.03.03 Прикладная информатика
Направленность (профиль) 09.03.03_03 Интеллектуальные инфокоммуникацион-
ные технологии

Выполнил
студент гр. з5130903/00301

Е.Н. Мазур

Руководитель
ст. преподаватель ВШ ПИ

В.А. Пархоменко

Консультант
по нормоконтролю

В.А. Пархоменко

Санкт-Петербург - 2025

**САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ПЕТРА ВЕЛИКОГО**

Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

Руководитель ОП

_____ А.В. Щукин

« _____ » _____ 202_ г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

студенту Мазуру Егору Николаевичу гр. з5130903/00301

1. Тема работы: Разработка плагина совместной электронной таблицы в Moodle.
2. Срок сдачи студентом законченной работы: 11.01.2025.
3. Исходные данные по работе: основным источником литературы является документация для разработчиков Moodle, руководство по РНР, документация по API jQuery.
4. Содержание работы (перечень подлежащих разработке вопросов):
 - 4.1. Исследование возможностей использования совместных электронных таблиц в Moodle.
 - 4.2. Проектирование плагина для работы с электронными совместными таблицами в Moodle.
 - 4.3. Разработка и внедрение плагина в Moodle.
 - 4.4. Апробация и тестирование разработанного плагина.
5. Перечень графического материала (с указанием обязательных чертежей):
 - 5.1. Архитектура разработанных плагинов.
6. Консультанты по работе:
 - 6.1. Ст. преподаватель ВШ ПИ, В.А. Пархоменко (нормоконтроль).
7. Дата выдачи задания: 12.10.2024.

Руководитель ВКР _____ В.А. Пархоменко

Задание принял к исполнению 12.10.2024

Студент _____ Е.Н. Мазур

РЕФЕРАТ

На 56 с., 27 рисунков, 3 таблицы, 2 приложения

КЛЮЧЕВЫЕ СЛОВА: СОВМЕСТНЫЕ ТАБЛИЧНЫЕ ТЕХНОЛОГИИ, РАБОТА В MOODLE, PHP, ВЕБ-СОКЕТ.

Тема выпускной квалификационной работы: «Разработка плагина совместной электронной таблицы в Moodle».

Целью работы является разработка плагина для совместной работы с электронными таблицами в Moodle. Объект исследования – платформа для дистанционного обучения Moodle. Предмет исследования – табличные технологии и средства совместной работы. Основными методами проведения работы являются методы системного анализа и синтез, а также методы графического проектирования и процедурного программирования. В данной работе предоставлен процесс разработки плагина на базе платформы Moodle с использованием технологий PHP, JavaScript, MySQL. Проведено исследование возможностей Moodle в сфере табличных технологий и возможности совместной работы в реальном времени. Проанализированы современные табличные технологии, и произведено сравнение этих технологий с возможностями разрабатываемого плагина. Изучена технология проектирования и разработки плагинов в Moodle. Спроектирован и разработан плагин для совместной работы с электронными таблицами с применением веб-сокета для передачи и обновления данных на странице, и возможностью привязки и сокрытия ячеек для отдельных пользователей. Проведено тестирование и апробация плагина на группе из 6 человек. Разработанный плагин может быть использован в дальнейшем для организации учебного процесса, или составления заданий для студентов на курсе.

ABSTRACT

56 pages, 27 figures, 3 tables, 2 appendices

KEYWORDS: COLLABORATIVE SPREADSHEET, WORK IN MOODLE, PHP, WEBSOCKET.

The subject of the graduate qualification work is «Developing a collaborative spreadsheet plugin in Moodle».

The aim of the work is to develop a collaborative spreadsheet plugin for Moodle. The object of the research – Moodle distance learning platform. The subject of the study – spreadsheet technology and collaboration tools. The main methods of work are methods of system analysis and synthesis as well as methods of graphic design and procedural programming. In the given work the process of plugin development, which is based on Moodle platform using PHP, JavaScript, MySQL technologies. The study of Moodle capabilities in the field of spreadsheet technologies and collaboration tools. The modern table technologies are analyzed, and comparison with the capabilities of the developed plugin. Studied the technology of designing and developing plugins in Moodle. Designed and developed a plugin for collaborative work with spreadsheets using a web-socket to transfer and update data on the page, and the ability to bind and hide cells for individual users. Testing and approbation of the plugin on a group of 6 people was carried out. The developed plugin can be used in the future to organize the learning process, or to make assignments for students in the course.

СОДЕРЖАНИЕ

Введение	7
Глава 1. Анализ средств для работы с совместными таблицами в образовательной среде	9
1.1. Основные функции и возможности Moodle	9
1.1.1. Группы.....	10
1.1.2. Форумы.....	10
1.1.3. Вики.....	10
1.1.4. Чаты.....	10
1.1.5. База данных	11
1.1.6. Глоссарий	11
1.1.7. Семинар.....	11
1.2. Сторонние технологии Moodle	11
1.2.1. Rocket.Chat	11
1.2.2. Rtcollaboration	12
1.2.3. Matching Game Plugins.....	12
1.3. Преимущества и недостатки	12
1.3.1. Преимущества выбора Moodle	13
1.3.2. Ограничения Moodle.....	13
1.4. Табличные технологии	14
1.4.1. Microsoft Excel	14
1.4.2. Google Sheets.....	15
1.4.3. Apple Numbers.....	15
1.4.4. Yandex Tables.....	16
1.4.5. Airtable	16
1.5. Выводы	17
Глава 2. Проектирование плагина совместных таблиц для Moodle	19
2.1. Моделирование случаев использования плагина	19
2.1.1. Установка и настройка плагина	20
2.2. Выбор средств.....	21
2.2.1. Организация кода Moodle.....	21
2.2.2. База данных Moodle	21
2.2.3. Способы передачи данных	22
2.3. Проектирование архитектуры приложения.....	24
2.3.1. Общая архитектура плагина	24

2.3.2. Проектирование функциональной модели приложения	25
2.3.3. Проектирование модели сущность-связь	27
2.4. Выводы	28
Глава 3. Реализация плагина совместной таблицы	29
3.1. Интерфейс.....	29
3.1.1. Блок загрузки данных из других источников.....	30
3.1.2. Блоки редактирования текста ячеек.....	31
3.1.3. Блок выставления оценок	33
3.2. Визуализация	35
3.3. Взаимодействие с базой данных.....	38
3.4. WebSocket.....	39
3.5. Выводы	40
Глава 4. Тестирование и апробация разработанного плагина.....	42
4.1. Выбор подходов к тестированию.....	42
4.2. Модульное тестирование с PHPUnit.....	42
4.3. Модульное тестирование с Jest.....	44
4.4. Тестирование веб-сокета.....	46
4.5. Загрузка на хостинг	49
4.6. Апробация.....	50
4.7. Соответствие критериям оценивания	51
4.8. Выводы	52
Заключение	53
Список сокращений и условных обозначений.....	54
Список использованных источников.....	55
Приложение 1. Код разработанного плагина	57
Приложение 2. Код разработанного веб-сокета.....	173

ВВЕДЕНИЕ

В контексте современного образовательного процесса, который с каждым днем все больше применяет цифровые технологии и стремится к высокому уровню интерактивности и взаимодействия, платформы для управления обучением, или Learning Management Systems (далее — LMS) играют ключевую роль. Modular Object-Oriented Dynamic Learning Environment (далее — Moodle), является одной из самых популярных и доступных платформ для управления обучением [18]. Эта платформа предоставляет образовательным учреждениям и преподавателям мощный инструментарий для создания и распространения учебного контента и оценки работы студентов.

Актуальность данной работы заключается в существующем пробеле в функциональности данной платформы в области совместного редактирования документов и таблиц в частности. На данный момент нам удалось найти несколько примеров плагинов [17; 20; 23] для совместной работы в реальном времени, но они не актуальны в более поздних версиях Moodle, либо существуют в виде теоретического проекта.

Широко известны случаи использования в образовательной среде Moodle электронных таблиц отдельно от платформы для составления учебных заданий, составления табелей с оценками, вывод результатов опросов и тестирования в табличном формате для удобства просмотра и анализа. Кроме обычных таблиц особый интерес представляют совместные таблицы, которые позволяют пользователям взаимодействовать с данными в режиме реального времени для создания, обработки, анализа и отображения информации.

Несмотря на то, что различные табличные технологии применяются в образовательном процессе вне Moodle, мы решили реализовать возможность использования совместных электронных таблиц внутри платформы, при этом реализовать в таблице уникальные возможности, направленные на улучшение организации процесса обучения в Moodle.

Объектом исследования данной работы является платформа для дистанционного обучения Moodle.

Предметом исследования данной работы являются табличные технологии и средства совместной работы.

Целью исследования данной работы является разработка плагина для системы дистанционного обучения Moodle, позволяющего работать с таблицей в режиме реального времени.

Для достижения поставленных целей необходимо решить следующие задачи:

- А. Изучить существующие возможности LMS по возможностям совместных таблиц.
- В. Изучить и описать случаи использования групповой работы в образовании.
- С. Провести анализ сервисов для работы с совместными таблицами.
- Д. Спроектировать и разработать плагин для работы с совместными таблицами в Moodle.

Основными методами проведения работы являются методы системного анализа и синтез, а также методы графического проектирования и процедурного программирования.

Первая глава посвящена обзору возможностей Moodle в сфере табличных технологий и совместной работы, а также анализу современных табличных технологий и сравнению их возможностей с возможностями Moodle. Во второй главе рассматривается проектирование плагина совместных электронных таблиц, с описанием его работы в виде Use Case, IDF0 и ArchiMate диаграмм, а также модели сущность-связь. В третьей главе идет описание разработки плагина для работы с совместными электронными таблицами. Четвертая глава посвящена процессу тестирования и апробации разработанного плагина.

ГЛАВА 1. АНАЛИЗ СРЕДСТВ ДЛЯ РАБОТЫ С СОВМЕСТНЫМИ ТАБЛИЦАМИ В ОБРАЗОВАТЕЛЬНОЙ СРЕДЕ

В параграфе 1.1 этой главы рассматриваются основные возможности Moodle в виде его активностей. В параграфе 1.2 рассматриваются возможности Moodle в виде сторонних плагинов. Рассматривались плагины имеющие возможность отображать действия других пользователей в реальном времени. В параграфе 1.3 рассматриваются преимущества и недостатки Moodle и его возможностей. В параграфе 1.4 рассматриваются современные табличные технологии.

1.1. Основные функции и возможности Moodle

Moodle является одной из самых популярных платформ для управления обучением LMS. Это открытая система, которая предлагает широкий спектр инструментов для создания и управления курсами, включая создание контента, управление участниками, оценку и отслеживание прогресса.

Moodle впервые был выпущен в 2002 году и разработан в качестве альтернативы Blackboard Мартином Дугиамасом на языке PHP, который легко мог изучить человек, с целью создания большого сообщества. Для хранения данных в Moodle используется база данных MySQL. Сама платформа может как устанавливаться на локальный персональный компьютер так и храниться в облачном хранилище, что не требует установки на сервере и упрощает процесс создания онлайн-курсов [5].

С точки зрения структуры контента Moodle очень жесткий, он позволяет создавать только курсы, состоящие из разделов, куда загружаются материалы для обучения, при этом он не позволяет упорядочить разделы или ресурсы, а доступ к самим ресурсам может предоставляться по неделям, то есть каждую следующую неделю студент получает доступ к следующему разделу [5].

С точки зрения организации Moodle позволяет раздать пользователям заранее определенные роли: администратор, учащийся и преподаватель. У роли есть полномочия, которые назначаются каждой из них, так же есть возможность определять роли в инструментах, то есть один пользователь может учеником на курсе и при этом администратором форума на этом курсе [5].

Взаимодействие студентов это одна из основных сильных сторон Moodle. Платформа предлагает ряд инструментов для мероприятий под руководством студентов и их совместной работы, которые помогают улучшить взаимодействие и

сотрудничество между участниками курса. Далее будут рассмотрены несколько основных функций Moodle для совместной работы.

1.1.1. Группы

Группы Moodle — это способ выражения групп пользователей в рамках курса. Они могут быть определены преподавателем на странице участников курса или созданы автоматически во время массовой загрузки пользователей. Также группы могут быть объединены и им могут быть назначены отдельные мероприятия, которые будут отображаться только для членов конкретных групп.

1.1.2. Форумы

Форум, или доска объявлений – это инструмент, позволяющий участникам курса взаимодействовать между собой и преподавателями, новости и изменения связанные с курсом и обмениваться информацией. Это особенно важно, когда доходит до обучения в онлайн-среде.

Будучи асинхронным инструментом, форумы позволяют участникам курса общаться друг с другом в любое время и из любого места, где есть подключение к Интернету.

1.1.3. Вики

Вики-страница – это набор веб-страниц, которые ученики вашего курса могут создавать индивидуально, или в группе прямо в браузере, без необходимости знать HTML. В Moodle вики может стать мощным ресурсом для совместной работы студентов. Все участники курса могут совместно редактировать документ, добавляя новые страницы или редактировать старые, создавая общий продукт, при этом история изменения каждой страницы сохраняется, позволяя увидеть процесс лежащий в основе конечного продукта.

1.1.4. Чаты

Чат в системе Moodle позволяет вести общение между участниками обучающего курса в режиме онлайн. Хотя форум или вики-активность позволяют студентам вносить свой вклад, когда им заблагорассудится, чат наиболее полезен, когда вы хотите, чтобы студенты общались друг с другом одновременно. Чаты

также работают в сочетании с групповыми режимами, что означает, что вы можете создавать комнаты чата групп для проведения онлайн-встреч.

1.1.5. База данных

База данных Moodle — это мощный инструмент, который позволяет преподавателям создавать набор структурированных данных и управлять ими в рамках системы управления обучением. Moodle позволяет преподавателям создавать индивидуальные базы данных, которые могут быть адаптированы под конкретные потребности и помогут улучшить совместное обучение студентов.

1.1.6. Глоссарий

Глоссарий в системе Moodle – это электронный аналог справочника специальных терминов, словаря. Учителя могут ограничить доступ к глоссарию, чтобы только они могли создавать записи, или они могут разрешить учащимся добавлять новые записи. Как и в случае с модулем «вики», студенты могут последовательно заполнять глоссарий в течение всего периода обучения.

1.1.7. Семинар

Семинар в Moodle это вид занятий, где каждый студент не только выполняет собственную работу, но и оценивает результаты работы других студентов. Итоговая оценка учитывает не только качество собственных работ, но и деятельность студентов в качестве рецензентов. Преподаватель тщательно планирует тему семинара, выделяет наиболее важные вопросы.

1.2. Сторонние технологии Moodle

1.2.1. Rocket.Chat

Плагин Rocket.Chat – это плагин позволяющий преподавателям держать синхронизированных пользователей, зарегистрированных в курсе Moodle, в специальном приватном канале чата [11]. Плагин интегрирует сторонний сервис Rocket.Chat в Moodle, где пользователи должны создать свой аккаунт, или объединить новый аккаунт с существующим. Есть возможность установить Rocket.Chat на свой сервер или хостинг.

Плагин представляет из себя уже готовый мессенджер, который позволяет общаться в реальном времени и имеет открытый исходный код.

1.2.2. Rtcollaboration

Rtcollaboration – совместный редактор в реальном времени, работающий как Google Docs, где два пользователя могут одновременно работать в одном документе [20]. Плагин периодическими запросами, при помощи XMLHttpRequest, проверяет изменения в тексте, и в случае изменения текста обновляет его. Автор пишет, что у учителя есть возможность «переиграть» работу студента в документе при помощи редактора, но проверить на практике, что это означает не получилось, так как плагин не обновлялся 13 лет и на новых версиях Moodle не может быть установлен из-за того, что активность, на основе которой плагин создан, удалена из Moodle. Также стоит отметить, что плагин использует устаревшую технологию передачи данных Long Polling, которая подходит для взаимодействия учителем и одним студентом, но не подходит для таблицы с которой будут взаимодействовать множество пользователей одновременно.

1.2.3. Matching Game Plugins

В статье [17] приведено теоретическое описание плагина новой активности для Moodle. Работа написана I Nyoman Darma Kotama, Komang Oka Saputra и Linawati Linawati из университета Udayana University в Индонезии, для журнала International Journal of Emerging Technologies in Learning в 2019 году. В своей работе они описывают активность в виде многопользовательской, обучающей игры, в которой используется WebSocket для передачи данных между большим количеством пользователей в реальном времени. На данный момент практической реализации нам найти не удалось, но идея использования WebSocket показалась наиболее интересной и актуальной.

1.3. Преимущества и недостатки

В данном параграфе мы рассмотрим преимущества и недостатки выбора Moodle в качестве платформы для расширения её функциональности и разработки на её основе плагина для работы с совместными таблицами.

1.3.1. Преимущества выбора Moodle

Одной из ключевых особенностей Moodle является то, что эта платформа свободно распространяется как программное обеспечение с открытым кодом на условиях GNU, что означает, что пользователи могут свободно загружать, использовать, изменять и даже распространять его при условии, что они согласны [24]:

- предоставлять исходный текст другим;
- не изменять и не удалять оригинальную лицензию и авторские права;
- применять ту же лицензию к любой производной работе.

Открытый код, простота изучения инструментов на которых создана платформа, а также поддержка множества сторонних ресурсов, систем и сервисов сформировали вокруг Moodle довольно большое сообщество, платформа получила высокую степень признания и используется в ряде учебных заведений. Как разработчики так и простые пользователи участвуют в развитии проекта, активно дискутируют на форумах, делятся советами, а список плагинов постоянно пополняется.

Moodle может использоваться практически на всех серверах или выгружаться в облако, работает на всех системах, поддерживающих PHP, таких как Unix, Linux и Windows. Пользователи могут установить его на любом компьютере или использовать в мобильном приложении, а также легко обновлять с одной версии на другую.

Он обладает многими возможностями, такими как простота установки и использования, хорошая поддержка, справка и множество образовательных инструментов, полезных для студентов и такими как отличная документация, мощная поддержка безопасности и администрирования, полезные для преподавателей и разработчиков.

1.3.2. Ограничения Moodle

Несмотря на большое количество возможностей и преимуществ выбранной для исследования платформы Moodle, он так же имеет ряд ограничений и недостатков, некоторые из которых проистекают из её преимуществ.

К примеру, хотя в Moodle и присутствует большое сообщество, которое поддерживает и совершенствует платформу, в ней, из-за этого, отсутствует профессиональная техническая поддержка, а саму систему придется собирать с нуля

своими руками, что может быть сложно для обычных пользователей, а от преподавателей могут требоваться некоторые технические знания в области веб-разработки.

Так же использование простого для изучения PHP наложило свои ограничения. Дело в том, что сам по себе PHP не является фреймворком MVC и веб-сайты на базе обычно плохо разделяют задачи. Moodle, в котором код бизнес-логики переплетается с кодом пользовательского интерфейса, сталкивается с той же проблемой. Это создает запутывание из-за которого становится сложно добавить новые задачи [5].

1.4. Табличные технологии

Одной из ключевых проблем, из-за которой была выбрана данная тема выпускной квалификационной работы, — это то, что в Moodle нет прямого внедрения совместных таблиц. Есть возможность загрузить на курс файл Excel или сделать ссылку на таблицу Google Docs, но использовать электронные таблицы прямо на курсе может вызвать множество трудностей, так как регламентировать работу с такой таблицей на курсе довольно проблематично.

Поэтому далее в этой работе будут рассмотрены несколько вариантов табличных технологий, которые мы сравним между собой и с ближайшими к ним возможностями Moodle.

1.4.1. Microsoft Excel

Excel – программа для работы с электронными таблицами, созданная корпорацией Microsoft. Он одинаково хорош как для составления примитивных отчётов или графиков, так и для глубокого анализа больших объёмов информации.

Среди особенностей электронных таблиц Excel стоит выделить следующие моменты [10; 13; 15]:

- анализ и моделирование, выполненное с помощью вычислительных операций и обработки полученных данных;
- оформление отчетов и прочих документов;
- составление диаграмм нужного вида;
- создание баз данных и осуществление управления этими данными;
- вставка таблиц из других приложений;
- автоматизация стандартных задач.

1.4.2. Google Sheets

Google Sheets – онлайн-инструмент для создания и ведения электронных таблиц разработанный Google LLC как аналог таблиц Microsoft Excel и принципы работы у них схожи, но есть и отличия. Вот ключевые различия между ними [8; 14; 15]:

- Google Sheets это полностью онлайн-электронная таблица, в то время как Microsoft Excel – настольная электронная таблица, хотя у нее есть и интернет-версия;
- Google Sheets это бесплатная электронная таблица, и вы можете бесплатно пользоваться всеми ее функциями. В то время как Microsoft Excel не является бесплатным, поскольку это лицензионный продукт, который поставляется в пакете Microsoft Office;
- формулы Excel и формулы Google Sheets также отличаются. Основные функции у них общие, но Excel имеет некоторые расширенные возможности, такие как XLOOKUP. В свою очередь Google Sheets имеет свои специальные функции;
- языком сценариев для Google Sheets является Google Apps Script, в то время как Microsoft Excel использует VBE;
- Excel может работать с большим объемом данных, исчисляемым миллионами, в то время как Google Sheets может работать только с небольшими объемами данных.

1.4.3. Apple Numbers

Numbers — это в значительной степени эквивалент Excel от Apple, он входит в состав iWork Productivity Suite для устройств macOS и iOS. Аналогично с Google Sheets, Numbers схожа по своей функциональности с Excel, но имеет свои особенности [7; 10]:

- Numbers значительно уступает по количеству встроенных функций. Более 420 в Excel против 250 в Numbers;
- Numbers бесплатна для всех пользователей macOS и iOS. Вы получите полный доступ к ее функциям без подписки на какую-либо лицензию;
- Numbers создана больше для личного и повседневного использования, он уступает по размеру и сложности обрабатываемых данных Excel;

- Numbers имеет лучшие возможности для диаграмм и графических построений. Вы можете контролировать их оформление и сделать их более привлекательными, чем стандартные опции Excel;
- Numbers работает только на Mac и iOS, поэтому предназначена исключительно для пользователей Apple;
- Numbers позволяет гораздо проще ориентироваться в ее функциях и инструментах. Но она использует только одно большое полотно, поэтому вы не можете рассортировать свои данные по разным листам.

1.4.4. Yandex Tables

Яндекс таблицы предлагают аналогичный набор функций и возможностей, который есть в Google Sheets. Этот инструмент также позволяет создавать, редактировать и анализировать данные в реальном времени, а также импортировать и экспортировать таблицы в различные форматы.

Гугл таблицы имеют больше шаблонов и большую интеграцию с другими сервисами Google, но Яндекс таблицы могут быть предпочтительнее для русскоговорящих пользователей [25].

1.4.5. Airtable

Airtable был основан как налог Google Sheets в 2012 году, но публично запущен только в 2015 году. На первый взгляд этот инструмент может показаться гораздо более красочной версией приложения Google для работы с электронными таблицами. Однако на сравнении электронных таблиц Airtable сходства в значительной степени заканчиваются.

Airtable, гораздо более универсален и гибок. Являясь гибридом базы данных и электронной таблицы, этот инструмент позволяет пользователям работать с огромными объемами данных. Можно управлять своими данными и отображать их различными способами, а также подключать свои наборы данных и таблицы.

Благодаря многочисленным интеграциям и сторонним приложениям функциональность Airtable расширяется еще больше, что делает ее полезной для управления взаимоотношениями с клиентами, управления проектами, складскими запасами, электронной коммерции, управления продуктами, маркетинга, продаж и т.д.

Отличие Airtable от более традиционных электронных таблиц [6; 8]:

- Airtable позволяет явно задавать тип каждого столбца, позволяя вводить обычный текст, даты, отдельные выборки, вложения и многое другое;
- Airtable позволяет связывать записи в разных электронных таблицах, или в том, что Airtable называет таблицами;
- связанные записи позволяют вам устанавливать связи между различными наборами данных;
- нет возможности задавать формулы для каждой отдельной ячейки. Формулы применяются ко всему столбцу сразу.

1.5. Выводы

Сравнивая приведенные выше таблицы можно сделать вывод, что в общем и целом они не слишком друг от друга отличаются. Существует Microsoft Excel и его аналоги, которые практически полностью скопировали уже работающую формулу, иногда урезая ненужные для общего потребителя функции, иногда добавляя новые, но большинство пользователей этого не заметит.

Единственным, кто выделяется из общей массы, является Airtables, так как структура таблицы в нем больше напоминает таблицу базы данных. Изменение форматов и добавление формул идет на весь столбец, а не на конкретную ячейку.

Сам Moodle не имеет встроенных табличных технологий совместного использования которые можно сравнить с предоставленными выше аналогами, и среди плагинов таких найти также не удалось, поэтому в дальнейшем сравнении вместо Moodle будет использоваться разрабатываемый плагин, и его планируемые возможности будут сравниваться с возможностями предоставленных выше таблиц.

Из табл.1.1 видно, что по основным табличным технологиям практически все сервисы схожи, и какие-то различия между ними минимальны.

После анализа приведенных табличных технологий было решено сделать функционал разрабатываемой таблицы с учетом нужд учебного процесса в Moodle. Взять основные возможности таблицы по редактированию текста и добавить возможности связанные с организацией учебного процесса.

Рассмотрим планируемые, дополнительные, функции для таблицы в разрабатываемом плагине и сравним их с возможностями существующих таблиц.

В табл.1.2 можно увидеть, что совместная работа присутствует во всех табличных сервисах, и одной из основных задач разрабатываемого плагина, это закрыть данный пробел в Moodle.

Таблица 1.1

Таблица основных функций электронных таблиц

	Microsoft Excel [13]	Google Sheets [14]	Apple Numbers [7]	Yandex Tables [25]	Airtable [6]	Разрабатываемое решение
Диаграммы	+	+	+	+	+	-
Поддержка формул	+	+	-	+	+	-
Поддержка функций	+	+	+	+	+	-
Поддержка Excel-файлов	+	+	-	+	-	+
Возможность выгрузки результатов из других активностей	+	+	-	+	-	+

Также в разрабатываемом решении будет возможность выдавать права доступа отдельным пользователям, или группам к редактированию отдельных ячеек. Будет возможность ограничить доступ к просмотру содержимого ячеек для тех пользователей у которых нет прав доступа к этим ячейкам.

Хотя в Moodle существует возможность выставить оценку активности, в разрабатываемом плагине будет реализована возможность оценить каждую отдельную ячейку предоставленную пользователю и на основе этих оценок вывести окончательный результат эффективности работы студента в таблице.

Кроме того в Moodle присутствует возможность выгружать данные некоторых активностей в excel таблицы и в разрабатываемом решении будет возможность выгрузить данные в таблицу плагина, как из другой активности, так и из excel файла.

Таблица 1.2

Таблица основных функций электронных таблиц

	Microsoft Excel [13]	Google Sheets [14]	Apple Numbers [7]	Yandex Tables [25]	Airtable [6]	Разрабатываемое решение
Совместная работа	+	+	+	+	+	+
Настройка прав доступа к отдельным ячейкам	-	-	-	-	-	+
Ограничение просмотра содержимого ячеек для отдельных пользователей	-	-	-	-	-	+
Возможность оценить таблицу или ячейки	-	-	-	-	-	+
Возможность выгрузки результатов из других активностей	+	+	-	+	-	+

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ПЛАГИНА СОВМЕСТНЫХ ТАБЛИЦ ДЛЯ MOODLE

В параграфе 2.1 этой главы описаны случаи использования плагина. В параграфе 2.2 описан выбор средств разработки плагина для Moodle. В параграфе 2.2 описано проектирование плагина в виде диаграмм.

2.1. Моделирование случаев использования плагина

Для уточнения требований к функциям плагина выведем несколько основных случаев его использования. Кроме установки самого плагина, предположим использование плагина в качестве активности с заданием и в качестве таблицы

для выставления оценок преподавателями. Изобразим эти три случая в качестве диаграмм вариантов использования.

2.1.1. Установка и настройка плагина

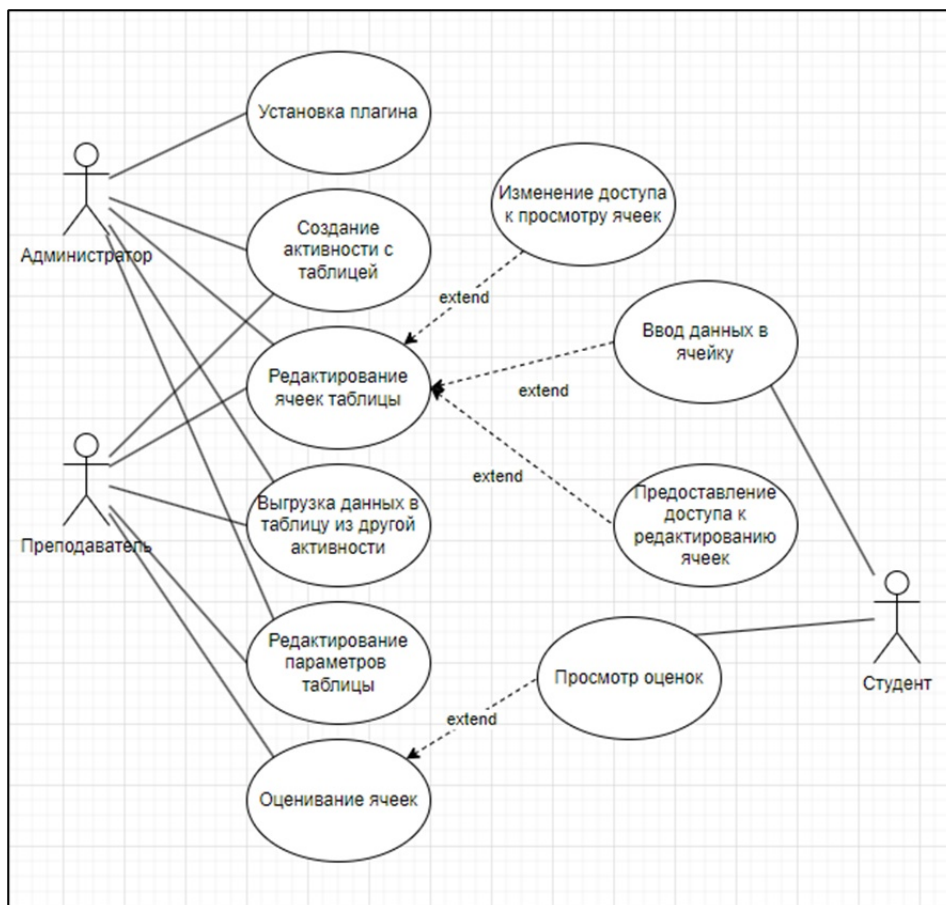


Рис.2.1. Диаграмма установки и настройки плагина

Главными выделяются три роли, это администратор преподаватель и студент. Администратор устанавливает и настраивает плагин в системе, у него есть все права доступа. У преподавателя есть часть прав доступа администратора. После установки плагина администратором, преподаватель может добавлять активность таблицы на курс. После он может редактировать саму таблицу и ячейки вручную, или загружать данные из xlsx файлов и других активностей. Также преподаватель может выделить студентам определенные ячейки для редактирования. Также преподаватель может редактировать возможность просмотра содержимого отдельных ячеек. Студент, после предоставления ему доступа, может взаимодействовать с ячейкой, изменяя её параметры и изменяя содержимое. Также у преподавателя есть возможность оценивать прикрепленные к студентам ячейки рис.2.1.

2.2. Выбор средств

Как уже говорилось ранее Moodle написана на языке PHP. Это скриптовый язык программирования, который, применяется для разработки веб-приложений. Он отвечает за генерацию HTML кода для клиента путем выполнения PHP скриптов, выполняющихся на сервере.

2.2.1. Организация кода Moodle

В Moodle, в основном, используется подход, основанный на скрипте транзакций. Это сценарий, где каждое взаимодействие между клиентской системой и серверной системой содержит определенную логику, которая организуется как единая процедура, в которой вызовы выполняются непосредственно к базе данных или через тонкую оболочку базы данных.

За этим базовым подходом к сценариям транзакций большая часть основной функциональности была рефакторингована в библиотеки (в основном в папке lib). Это обеспечивает элементы доменной модели. Но так как Moodle появился до того, как PHP научился обрабатывать объектно-ориентированный код, объектно-ориентированная модель появляется, в большинстве случаев, только в более поздних частях кода Moodle [19; 24].

2.2.2. База данных Moodle

Для хранения данных использует базу данных, управляемую MySQL, PostgreSQL, Microsoft SQL Server, MariaDB или Oracle. После нескольких лет разработки модель базы данных стала очень сложной и содержит более 250 таблиц, но при этом эта структура довольно понятна, так как таблицы для конкретного модуля ссылаются друг на друга и имеют определенный способ именования. Все системные таблицы имеют префикс "mdl_" затем идет имя модуля, например "mdl_course" предоставляет модули принадлежащие курсам, а "mdl_mod" модули принадлежащие активностям. Таким образом Moodle содержит 50 базовых таблиц, которые также разбиты на группы, что позволяет легко понимать их функционал [9; 19].

2.2.3. Способы передачи данных

Для отображения изменения содержимого ячейки в реальном времени нужно обеспечить постоянное соединение клиента с сервером, чтобы он в любой момент мог принять данные и отобразить их на странице. Есть несколько методов решения этой задачи.

2.2.3.1. Short polling

Первый способ передачи данных — это Short polling. Не самый эффективный метод для решения подобной задачи, но наиболее простой. Клиент раз в несколько секунд будет опрашивает сервер на наличие событий, даже в том случае если ничего не происходит.

Преимущества:

- простота в реализации;
- можно реализовать практически на любой платформе.

Недостатки:

- высокая нагрузка на сервер из-за постоянных, чаще всего ненужных, запросов;
- низкая скорость и задержка при передаче данных;
- сервер хранит данные пока клиент не заберет их или пока они не устареют.

2.2.3.2. Long polling

Следующим идет Long polling или длинные опросы. Это улучшенный вариант Short polling. Клиент будет отправлять запрос на сервер, который держит соединение открытым до тех пор, пока не придут какие-нибудь данные или клиент не отключится самостоятельно. Как только данные пришли сервер отправляет ответ и закрывает соединение для открытия нового для приема следующих запросов и так далее. Это был наиболее популярный метод для обновления контента на странице в реальном времени до появления веб-сокетов.

Преимущества:

- простота в реализации;
- асинхронная передача данных со стороны сервера;
- минимальное количество запросов;
- высокая временная точность событий;

- сервер хранит события только на время реконнекта.

Недостатки:

- задержки в передаче данных из-за ожиданий и таймаутов;
- высокая нагрузка на сервер из-за большого количества открытых соединений.

2.2.3.3. *Server-Sent Events*

Более лучший вариант подходящий под нашу задачу, это Server-Sent Events (далее — SSE). SSE – технология позволяющая серверу отправлять клиенту поток данных по одностороннему соединению. Сервер будет отправлять пустые события с определенной периодичностью, чтобы предотвратить закрытие соединения из-за таймаута, таким образом поддерживая соединение открытым.

Преимущества:

- сложнее чем предыдущие варианты, но проще чем веб-сокеты;
- автоматическое восстановление соединения при обрыве обеспечивает надежную передачу данных;
- широкая поддержка браузерами и различными фреймворками;
- использует HTTP, поэтому на сервере не нужна поддержка дополнительных протоколов и нет проблем с сетевыми экранами.

Недостатки:

- только односторонняя передача данных от сервера к клиенту;
- ограничение в кол-ве одновременно открытых соединений.

2.2.3.4. *WebSocket*

Последний, но наиболее важный, это WebSocket. Независимый протокол поверх TCP. Позволяет клиенту и серверу общаться на равных. На данный момент это наиболее современное и популярное решение задачи организации передачи данных между клиентом и сервером в реальном времени.

Именно он поможет нам обновлять информацию на странице в реальном времени. Клиент сначала отправит TCP-запрос для подключения к серверу, затем сервер периодически будет опрашивать просьбу отправить ответный запрос, чтобы проверить подключение клиента. В случае отправки одним из пользователей сообщения на сервер, оно будет переслано всем пользователям, за исключением

WS

Демонстрация работы WebSocket-соединения

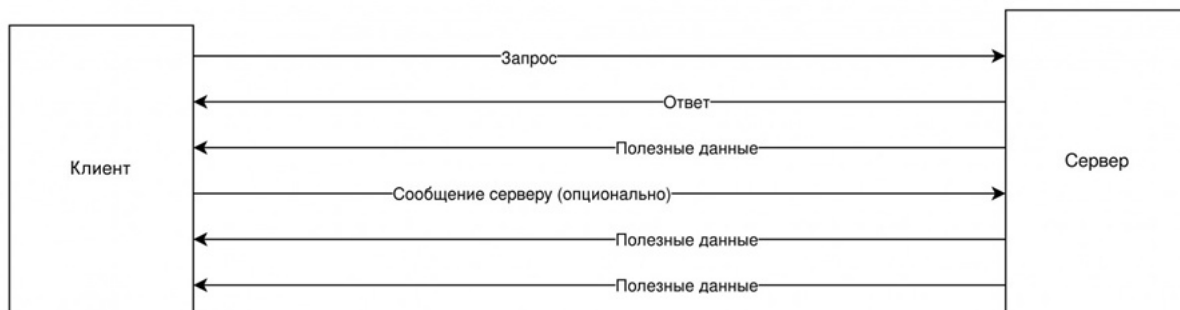


Рис.2.2. Демонстрация работы WebSocket-соединения [2]

отправителя. Таким образом можно будет отправить изменившиеся данные всем пользователям для обновления их страницы рис.2.2.

Преимущества:

- предельно высокая временная точность событий;
- количество соединений в браузере не ограничено;
- двунаправленная передача данных.

Недостатки:

- требует реализации отдельного серверного приложения;
- возможны проблемы с сетевыми экранами;
- необходима самостоятельная реализация протокола на стороне клиента.

Так как WebSocket является более современной и популярной технологией, которая позволяет более гибко решать задачи, мы проработаем решение с ее использованием.

2.3. Проектирование архитектуры приложения

2.3.1. Общая архитектура плагина

На рис.2.3 изображена общая архитектура плагина. Данные таблицы при изменении пользователем, сначала записываются в базу данных, затем отправляются на веб-сокеты, который рассылает эти данные остальным пользователям для визуализации изменений. Также можно заметить, что студент имеет отличные от

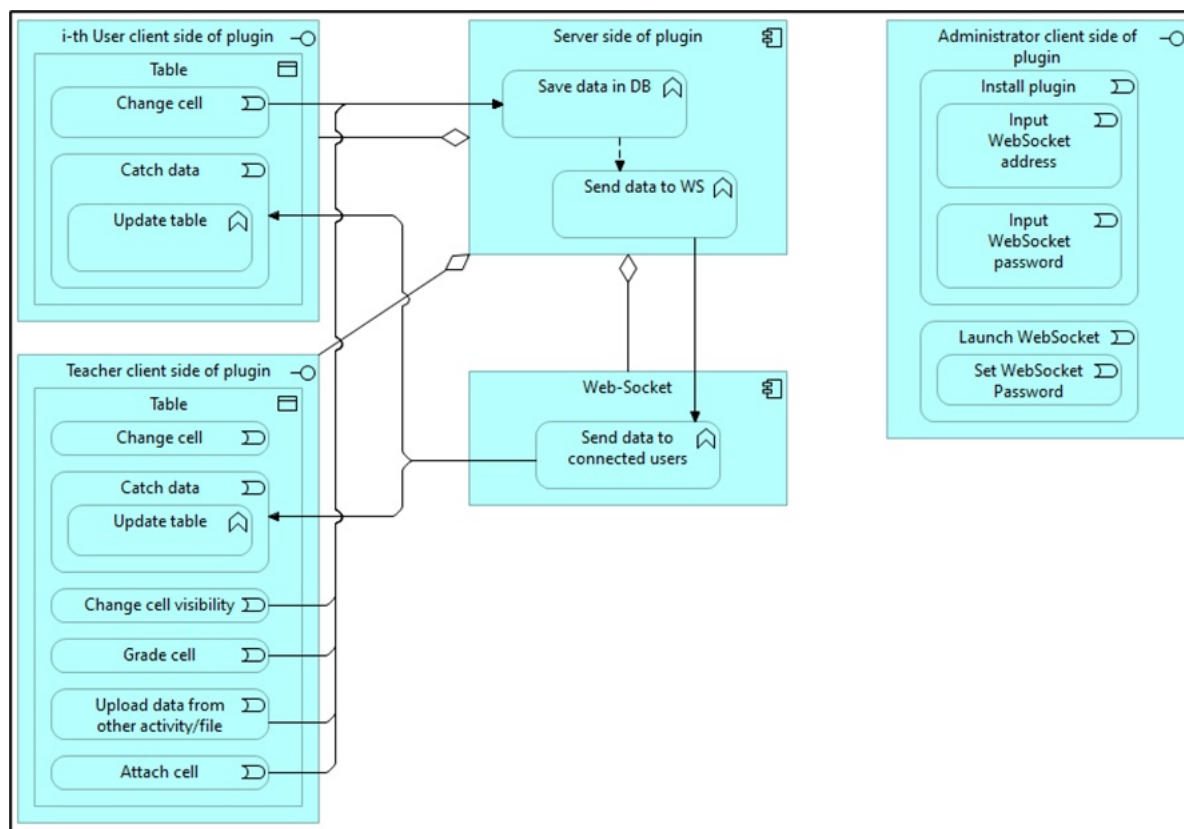


Рис.2.3. Диаграмма общей архитектуры плагина

преподавателя возможности. Интерфейс студента частично ограничен, не позволяя ему производить действия к которым у него нет доступа.

2.3.2. Проектирование функциональной модели приложения

Составим диаграмму декомпозиции плагина, начиная с контекстной диаграммы рис.2.4.

На вход подаются данные введенные пользователем в онлайн режиме. В качестве управления выступают правила работы с плагином и образовательное задание. Механизмами являются система Moodle с разработанным и установленным плагином, и преподаватель со студентом. На выходе должны получиться сохраненные в таблице данные, которые подавались на вход. Эти же данные должны записаться в историю изменения ячеек. Информация о представлении данных, это записи о том как отформатированы данные в ячейке, какой у пользователей к ячейке доступ, и кто может просматривать данные в ней.

На диаграмме первого уровня рис.2.5 можно увидеть разделение функциональности на четыре блока.

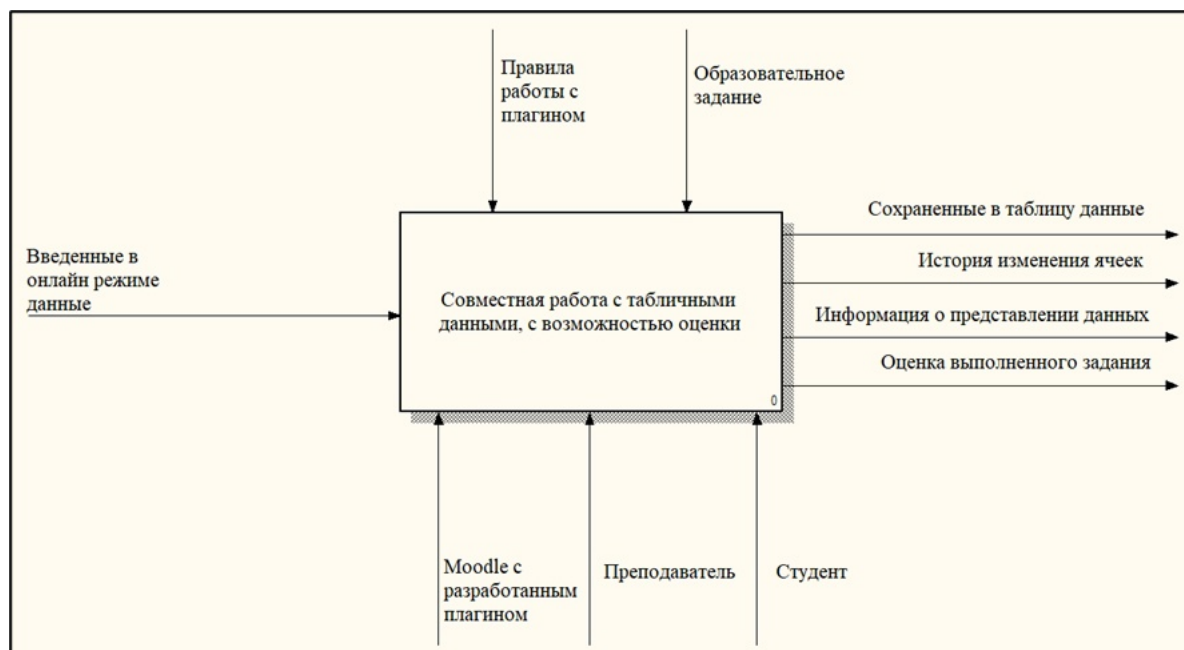


Рис.2.4. Контекстная диаграмма

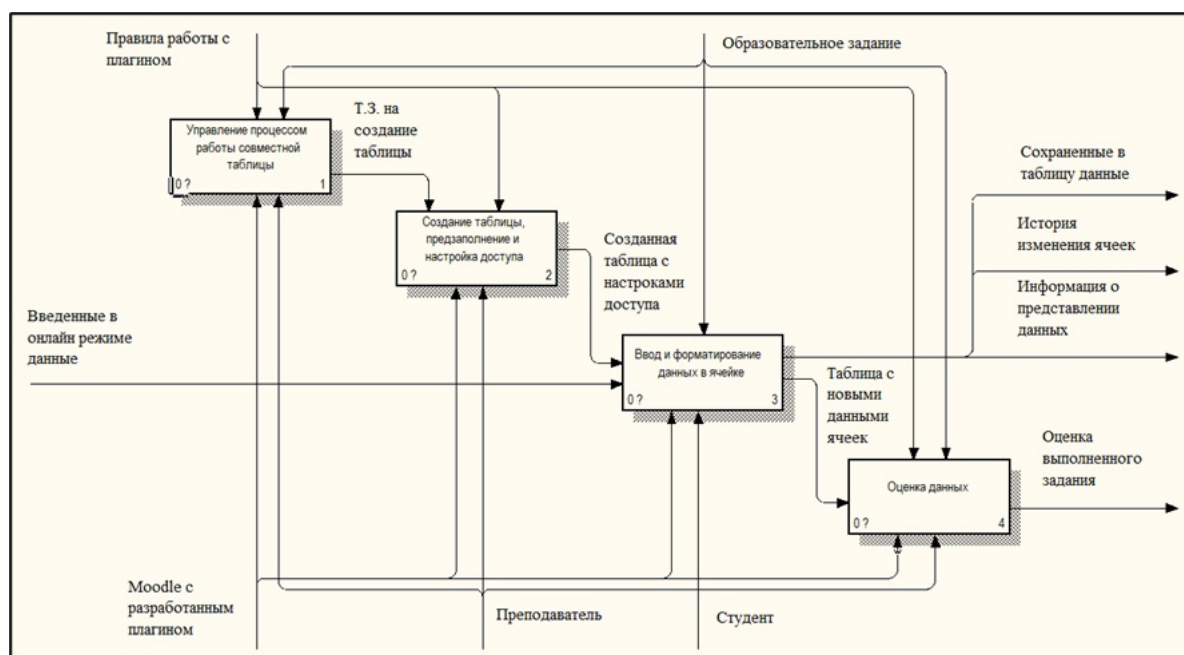


Рис.2.5. Диаграмма декомпозиции 1-го уровня

Первый блок это, управление процессом работы таблицы. В нем осуществляются различные функции планирования. В нем формируется техническое задание на создание таблицы, которое затем поступает на управление в следующий блок.

Второй блок, это создание и предзаполнение таблицы в соответствии техническим заданием, а также настройка доступа к ячейкам таблицы. В итоге во втором блоке формируется таблица с заданием, с которой студенты могут работать. Эта таблица подается на вход в следующий блок.

Третий блок, это ввод и форматирование данных в ячейке. Кроме таблицы на вход, подаются введенные в онлайн режиме данные. Простым языком, студент заполняет ячейки таблицы в соответствии с заданием. Кроме того он может изменить форматирование данных в ячейке. В результате мы получим сохраненные в таблицу данные, которые также будут записаны в историю изменения ячеек и данные о форматировании ячеек. В следующий, четвертый, блок из третьего поступит заполненная студентом таблица.

Четвертый блок, это оценка данных. Преподаватель оценивает ячейки, которые ранее заполнял студент, и в результате получается общая оценка работы, или общая групповая оценка, в зависимости от задания.

2.3.3. Проектирование модели сущность-связь

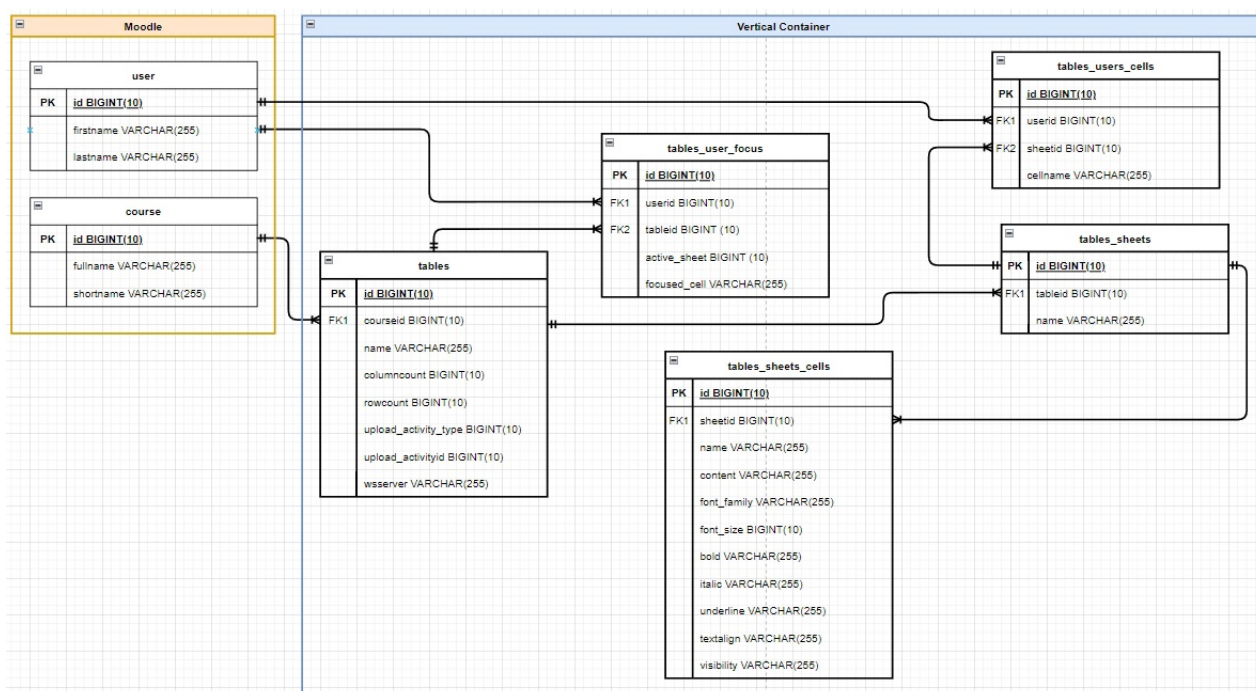


Рис.2.6. Диаграмма таблиц базы данных основных функций плагина

На рис.2.6 изображена диаграмма базы данных плагина. Moodle уже содержит таблицы course и user, на них будут ссылаться таблицы плагина. В первую очередь идет таблица tables, содержащая информацию обо всех таблицах. Таблица имеет следующие атрибуты:

- courseid – идентификатор курса которому принадлежит таблица;
- name – название таблицы отображаемое на странице;
- rowcount – количество строк таблицы;
- columncount – количество столбцов таблицы;

– wsserver – адрес веб-сокета сервера к которому будет подключаться таблица.

В таблицу `tables_user_focus` записывается `id` пользователя и ячейки с которой он взаимодействует. Она имеет следующие поля:

- `userid` – идентификаторы пользователей, которые имеют доступ к редактированию ячейки;
- `tableid` – идентификатор таблицы которой принадлежит ячейка;
- `active_sheet` – активный лист таблицы;
- `focused_cell` – активная ячейка.

В таблицу `tables_sheets` записывается информация о листах таблицы:

- `tableid` – `id` таблицы, которой принадлежит лист;
- `name` – имя листа.

В таблицу `tables_sheets_cells` записывается информация о ячейках:

- `name` – имя ячейки;
- `content` – содержимое ячейки;
- `font_size` – размер шрифта в ячейке;
- `font_family` – стиль шрифта в ячейке;
- `bold` – полужирный ли шрифт в ячейке;
- `italic` – курсив в ячейке;
- `underline` – подчеркнут ли текст в ячейке;
- `text_align` – информация о выравнивании текста в ячейке;
- `sheeted` – `id` листа, которому принадлежит ячейка.

В таблицу `tables_users_cells` записывается информация о привязанных к пользователям ячейках:

- `sheeted` – `id` листа ячейки;
- `userid` – `id` пользователя к которому привязана ячейка;
- `cellname` – имя привязанной ячейки;

Таблицы `tables_users_cells` и `tables_group_cells` содержат информацию о привязанных ячейках. В таблицу `tables_cells_grade` записываются оценки студента для каждой ячейки. Также была добавлена таблица `tables_cells_history` для записи истории изменения таблицы рис.2.7.

2.4. Выводы

В главе выбраны технологии и средства для разрабатываемой системы, а также приведены следующие виды проектирования:

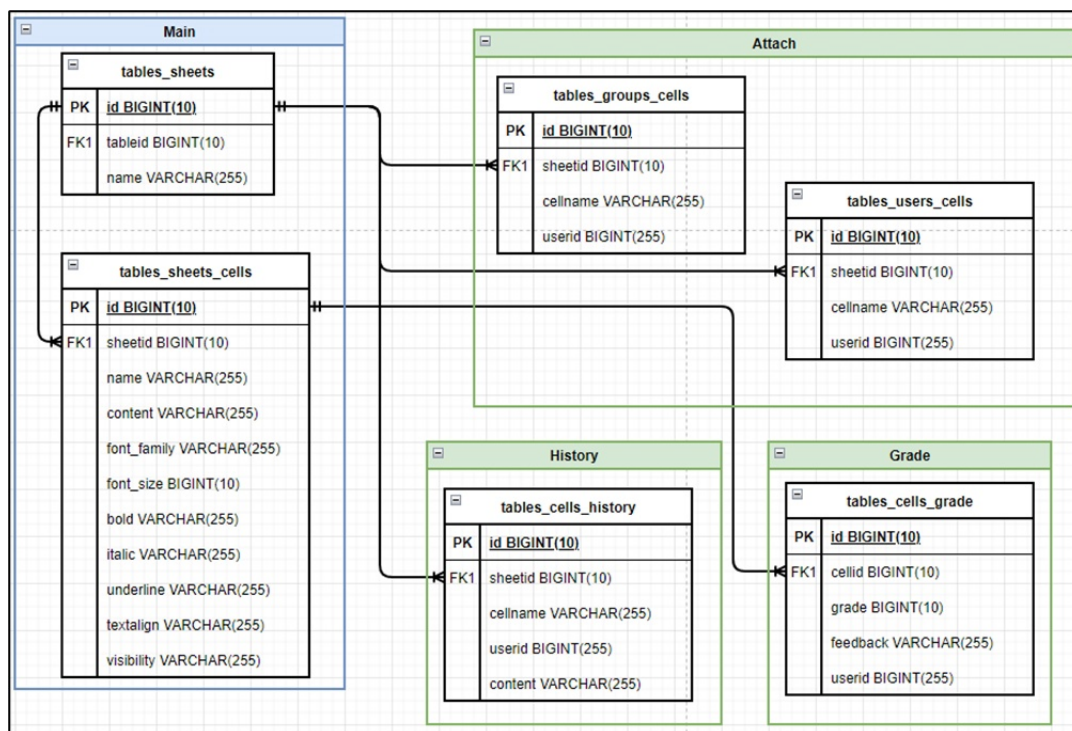


Рис.2.7. Диаграмма таблиц базы данных дополнительных функций плагина

- use-case диаграмма вариантов использования разрабатываемого приложения;
- диаграмма общей архитектуры приложения, выполненная в Archimate;
- IDF0 диаграмма функциональной модели приложения;
- диаграмма модели сущность-связь разрабатываемого приложения.

В следующей главе рассмотрим реализацию разрабатываемого приложения.

ГЛАВА 3. РЕАЛИЗАЦИЯ ПЛАГИНА СОВМЕСТНОЙ ТАБИЦЫ

В параграфе 3.1 третьей главы описан интерфейс разработанного плагина. В параграфе 3.2 описаны функции визуализации действий в таблице. В параграфе 3.3 описаны функции отвечающие за сохранение изменений в таблице в базу данных. В параграфе 3.4 описан разработанный веб-сокет.

3.1. Интерфейс

Для начала разработки, в Moodle создаются основные, для плагина, файлы, с которыми может помочь Moodle plugin skeleton generator [21]. Он позволяет создать

шаблон для будущего проекта установив основные файлы, самыми важными для нас являются:

- `install.xml` используется для определения любых таблиц базы данных, полей, индексов и ключей, которые должны быть созданы для плагина во время его первоначальной установки;
- `mod_form.php` используется при добавлении, или редактировании модуля в курс. Он содержит элементы, которые будут отображаться в форме, отвечающей за создание, или установку экземпляра вашего модуля;
- `view.php` файл в котором Moodle автоматически сгенерирует ссылки для просмотра основной страницы модуля по адресу `/view.php` с добавлением `id` курса и активности;
- `styles.css` содержит стили применяемые к плагину П1.13.

В файле `view.php` П1.1 отображается главная страница модуля разделенная на два основных блока панель инструментов и таблицу. Панель инструментов также разделена по блокам рис.3.1.

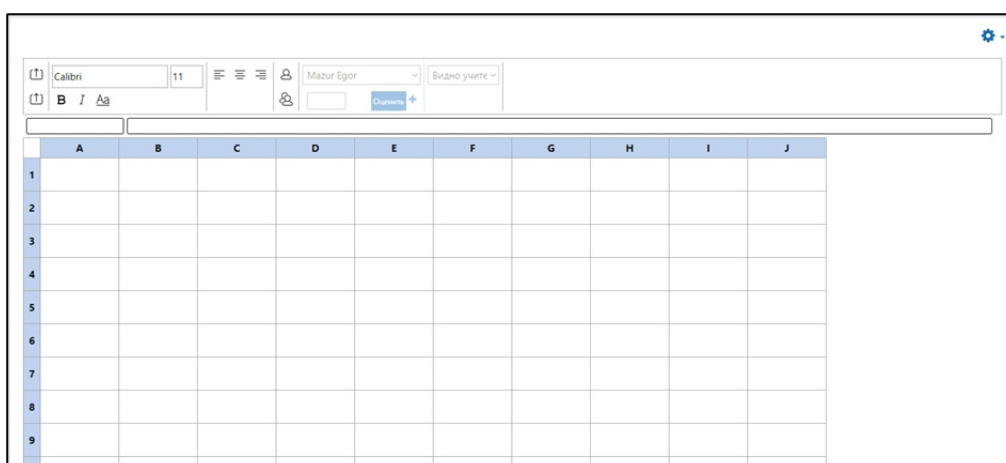


Рис.3.1. Таблица с панелью инструментов

3.1.1. Блок загрузки данных из других источников

Первый блок содержит две кнопки загрузки данных в таблицу из других источников. Верхняя кнопка позволяет загрузить данные их excel файла. При нажатии на кнопку пользователь переходит на страницу для загрузки файла рис.3.2, код страницы записан в файле `upload_from_xlsx.php` П1.15.

При выборе файла страница запускает скрипт из файла `load_excel.php`, который выгружает данные из выбранного excel [4] документа и записывает их в таблицы базы данных Moodle П1.16.

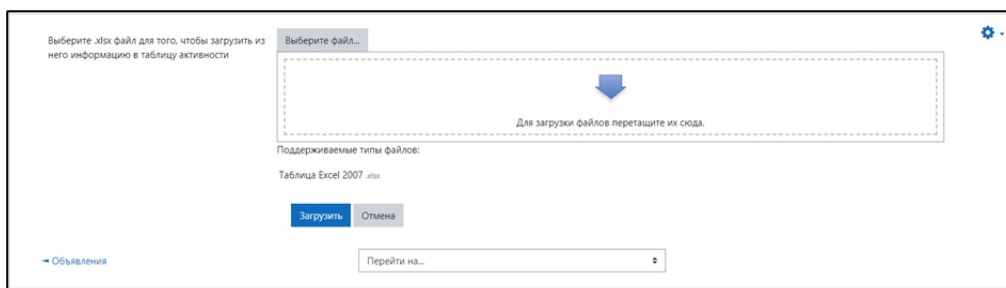


Рис.3.2. Страница загрузки excel файла

Нижняя кнопка позволяет загрузить данные из других активностей Moodle. При нажатии на кнопку пользователя переносит на страницу выбора активности рис.3.3 код которой хранится в файле `upload_from_activity.php` П1.17. На странице выбора активности отображаются доступные активности, находящиеся на том же курсе, что и таблица. На данный момент доступна выгрузка данных только из активности `quiz`.

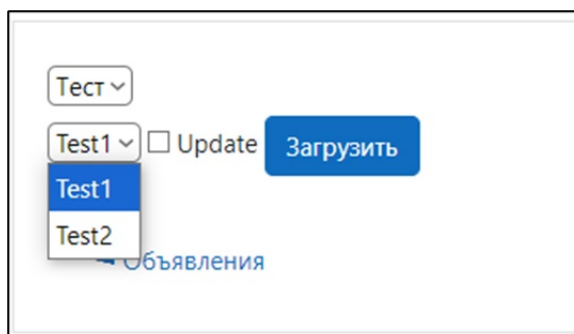


Рис.3.3. Страница выбора активности для загрузки

После подтверждения выбора запускается скрипт из файла `load_activity.php` П1.18.

3.1.2. Блоки редактирования текста ячеек

Второй содержит в себе основные функции редактирования текста рис.3.4:

- выпадающий список стилей шрифта;
- окно ввода размера шрифта;
- кнопка переключения полужирного текста;
- кнопка переключения курсива;
- кнопка переключения подчёркивания текста.

Третий блок содержит элементы для выравнивания текста по левому краю, по правому краю, по центру рис.3.4.

Также при редактировании ячеек происходит запись в отдельную таблицу базы данных, где хранится история изменения ячеек. Историю изменения можно



Рис.3.4. Блок редактирования текста ячеек

увидеть на отдельной странице код которой хранится в файле `history.php` П1.19. Четвертый блок содержит элементы для прикрепления ячеек к студентам или группам, которые изначально скрыты, для освобождения места на панели рис.3.5.

	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								

Рис.3.5. Таблица из аккаунта преподавателя, в процессе прикрепления ячеек

При нажатии на кнопку блокируются предыдущие блоки и появляется окно для поиска студента по имени, при наведение на которое появляется список студентов с чекбоксами. При выборе нескольких чекбоксов ячейки будут привязаны ко всем выбранным студентам. Вместе с окном для поиска появляются окна ввода первой и последней ячейки для прикрепления диапазона ячеек к выбранным студентам.

Также пока нажата кнопка прикрепления студентов или групп, при нажатии на ячейку она будет записана в окно выбора первой ячейки, при последующем нажатии на остальные ячейки, они будут записываться в окно выбора последней ячейки. После выбора студентов и ячеек нужно нажать на кнопку подтверждения прикрепления, или на кнопку отмены прикрепления, для отмены действий.

Как видно на рис.3.6 у той же таблицы, но просматриваемой из аккаунта студента заблокированы все ячейки, кроме тех к которым ему предоставили доступ. Также видно, что часть инструментов у него не отображается, они доступны только преподавателю. Прикрепленные ячейки можно отдельно увидеть на странице прикрепленных пользователей П1.20, где есть возможность открепить ячейки рис.3.7.

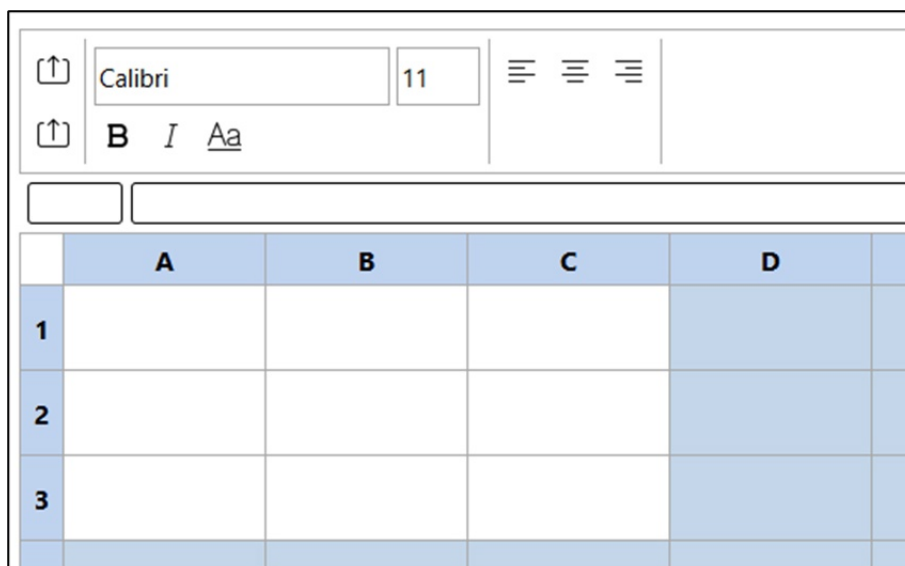


Рис.3.6. Панель инструментов и таблица из аккаунта студента

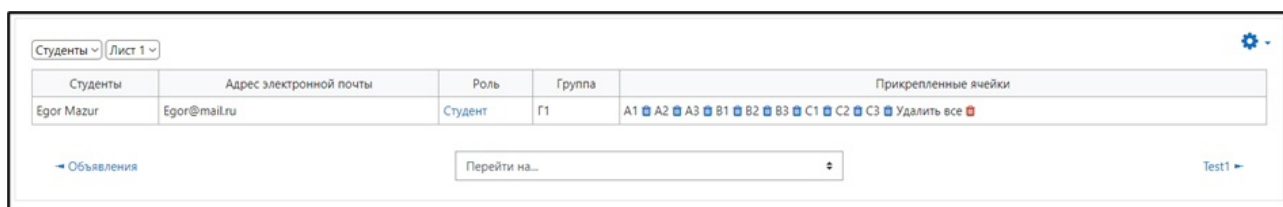


Рис.3.7. Страница прикрепленных пользователей

3.1.3. Блок выставления оценок

Пятый блок содержит элементы для оценки содержимого ячейки такие как выпадающий список со всеми студентами у которых есть прикрепленные ячейки в это модуле, окно для ввода оценки, кнопка для отображения окна для ввода комментария к оценке и кнопки подтверждения и отмены оценки.

Сама таблица содержит ячейки, количество которых можно выбрать в настройках модуля. Ячейки изначально заблокированы для всех студентов, до тех пор, пока преподаватель не предоставит к ним доступ.

При выборе ячейки пользователем, она резервируется за ним и блокируется у остальных пользователей, до тех пор пока пользователь не выберет другую ячейку или снимет резерв путём перезагрузки страницы, или удаления имени ячейки из окна и выбора активной ячейки, которое находится над таблицей рис.3.8.

Пока ячейка зарезервирована за пользователем, он может изменять её содержимое, или параметры вводимого теста используя элементы панели инструментов. Всё изменения будут отображаться в реальном времени при помощи веб-сокета.

	A	B	C	D	E	F
1						
2		Hello world				
3						
4						

Рис.3.8. Таблица от лица преподавателя, ячейку которой заполняет студент

За выставление оценки отвечает последний, на данный момент, блок на панели инструментов, его можно увидеть справа от блока прикрепления ячеек рис.3.9.

Рис.3.9. Выставление оценки

Этот блок активизируется при выборе ячейки. В верхнем поле выбирается студент которому выставляется оценка, в поле под ним вводится сама оценка. Справа от кнопки «Отправить» находится кнопка в виде плюса, которая отображает на панели окно для ввода комментария. После отправки оценки её можно увидеть на отдельной странице П1.26, где отображаются оценки всех студентов, или групп работающих на курсе рис.3.10.

Полное имя	Оценки	Суммарная оценка								
Egor Mazur	<table border="1"> <thead> <tr> <th>Имя ячейки</th> <th>Содержимое</th> <th>Отзыв</th> <th>Оценка</th> </tr> </thead> <tbody> <tr> <td>B2</td> <td>Hello world</td> <td>Good job</td> <td>75</td> </tr> </tbody> </table>	Имя ячейки	Содержимое	Отзыв	Оценка	B2	Hello world	Good job	75	75
Имя ячейки	Содержимое	Отзыв	Оценка							
B2	Hello world	Good job	75							
Администратор Пользователь		0								

Рис.3.10. Страница с оценками

3.2. Визуализация

Как было сказано ранее, отображение таблицы прописано в файле `view.php` П1.1 в корневой папке плагина. За визуальные изменения таблицы отвечают функции прописанные в файле `amd/src/update_data.js` П1.2. Основная функция `onFocusInCell` срабатывает, когда пользователь нажимает на ячейку.

При обычном взаимодействии с таблицей функция синхронизирует параметры панели инструментов с параметрами ячейки. То есть стиль и размер шрифта ячейки отображаются на панели инструментов, где его можно изменить. Также при этом имя ячейки и её содержимое выводится в соответствующие поля под панелью инструментов, они используются при взаимодействии с ячейками и при работе некоторых функций. Кроме того если с таблицей работает преподаватель, выбор ячейки открывает доступ к панели оценивания ячеек на панели инструментов. В конце функция отправляет данные о выбранной ячейке в РНР скрипт `update_cell_focus.php` П1.3 для записи ячейки в базу данных и в веб-сокеты для отображения выбора ячейки у других пользователей чтобы блокировать её до того как фокус с ячейки не будет снят.

Функция срабатывает по другому алгоритму, если преподаватель нажал на панели инструментов кнопку прикрепления ячеек к пользователю. В этом случае остальные элементы на панели блокируются и при выборе ячейки она будет выделяться как первая ячейка на привязку. При следующем выборе ячейки она выделяется как последняя на привязку. Таким образом реализована система прикрепления ячеек к пользователям или группе. Преподаватель, нажатием кнопки переходит в «режим прикрепления ячеек», выбирает из списка одного, или нескольких студентов, а затем просто выбирает диапазон ячеек путем выделения первой и последней. После выбора нужно подтвердить прикрепление нажатием кнопки подтверждения. В конце функция отправляет данные о выбранных ячейках в РНР скрипт `attach_cells.php` П1.4 для записи ячеек в базу данных и в веб-сокеты для отображения выбора ячейки у других пользователей.

За изменение параметров ячейки отвечают определенные функции из того же файла `update_data.js` П1.2:

- `updateHeight` – функция отправляет данные в скрипт `update_cell_size.php` П1.5 для записи в базу данных высоту строки таблицы;
- `updateWidth` – функция отправляет данные в скрипт `update_cell_size.php` П1.5 для записи в базу данных ширину столбца таблицы;

- `updateTableCell` – функция срабатывает при вводе текста в ячейку таблицы, она отправляет данные в скрипт `update_cell.php` П1.6 для записи в базу данных нового содержимого ячейки;
- `onInputSearch` – функция срабатывает при вводе текста в поле поиска при привязке ячеек к студентам. Функция выводит в выпадающий список только тех студентов, что содержат подстроку введенную в поле;
- `onclickAttach` – функция открывает поля для выбора студентов для прикрепления ячеек, и блокирует остальные элементы на панели инструментов. Также переводит таблицу в «режим прикрепления ячеек». При повторном нажатии производит действия в обратном порядке отключая «режим прикрепления ячеек»;
- `onclickCheckboxAttach` – функция визуально отображает выбор студента в выпадающем списке выбора студентов для прикрепления к ним ячеек;
- `onclickSubmitAttach` – функция срабатывает, когда пользователь нажимает на кнопку для подтверждения прикрепления ячеек. Она передает `id` студентов и диапазон ячеек в скрипт `attach_cells.php` для записи ячеек студентам в базе данных;
- `onclickCanselAttach` – функция срабатывает, когда пользователь нажимает на кнопку отмены прикрепления, она сбрасывает выбранные ячейки и позволяет выбрать из заново;
- `onFocusOutCell` – функция срабатывает, когда снимается фокус с ячейки, она разблокирует доступ у других пользователей путем отправки данных на веб-сокет;
- `onChangeInputCell` – функция срабатывает при изменении `input` поля выбранной ячейки под панелью инструментов. Она вызывает функцию `onFocusCell`, если новая ячейка была введена корректно. Если ячейка выбрана неправильно, или поле становится пустым, то фокус с ячеек сбрасывается;
- `onChangeInputContent` – функция срабатывает при изменении `input` поля содержимого выбранной ячейки. Она изменяет текст выбранной ячейки также, как если бы он вводился в ячейку напрямую. Поле создавалось для возможного дальнейшего развития возможностей таблицы;
- `updateFont` – функция срабатывает при взаимодействии с элементами на панели инструментов, которые отвечают за изменения шрифта текста ячейки. Функция проверяет какой элемент был нажат и отображает изменения

на экране у всех пользователей, которые взаимодействуют с таблицей. Она также отправляет данные на веб-сокеты и в скрипт `update_font.php` П1.7 для записи в базу данных новых параметров текста ячейки;

- `saveCellHistory` – функция срабатывает при изменении содержимого ячейки. Она записывает изменения таблицы хранящую историю изменения ячеек в базу данных, путем отправки новых данных в скрипт `save_history.php` П1.8;
- `createSheet` – функция срабатывает, когда пользователь нажимает на кнопку добавления нового листа таблице. Она отправляет данные в скрипт `create_sheet.php` П1.9 для записи в базу данных параметров нового листа. Затем функция обновляет страницу для отображения нового листа;
- `deleteSheet` – функция срабатывает, когда пользователь нажимает кнопку удаления листа таблицы. Она отправляет данные в скрипт `create_sheet.php` П1.9 для удаления листа из базы данных;
- `gradeCell` – функция срабатывает, когда пользователь подтверждает оценку ячейки, нажав кнопку подтверждения оценки. Она передает данные о ячейке и содержимое полей `grade` – оценка и `feedback` – отзыв. Данные передаются в скрипт `grade_cell.php` для записи оценки в базу данных. Также функция запускает анимацию, которая плавно отображает зеленую галочку, которая нужна для визуального подтверждения того, что оценка принята;
- `onChangeInputGrade` – функция срабатывает при изменении содержимого поля `grade`. Она нужна только для того, чтобы скрыть галочку, которая появляется при подтверждении оценки;
- `showFeedback` – функция срабатывает при нажатии на кнопку отображения поля отзыва. Она отображает поле для отзыва инструментов, если пользователь хочет ввести отзыв к оценке;
- `onChangeSelectVisibility` – функция срабатывает при изменении поля `select`, которое отвечает за выбор режима видимости ячейки. Функция передает данные в скрипт `update_visibility.php` для записи в базу данных нового параметра видимости ячейки.

Все функции отвечающие за изменение параметров ячейки, которые нужно отобразить у других пользователей, передают эти данные на веб-сокеты. Веб-сокеты пересылают эти данные другим пользователям взаимодействующим с таблицей. Эти данные обрабатываются в файле `connect_to_websocket.js` П2.2. В

файле определяется тип обновления, путем передачи в метод `select` параметра `update_type`, который определяется каждой функцией, отправляющей данные на веб-сокеты по-своему. Определив какие данные на странице были изменены функция отображает эти изменения на странице.

Также стоит отдельно заметить, что для изменения высоты строк и ширины столбцов курсором мыши используется JS библиотека `Interact.js`, а именно её метод `Resize`. Он вызывается в файле `interact_resize.js` П1.10. После изменения высоты или ширины вызывается соответствующая функция `updateWidth`, или `updateHeight`.

3.3. Взаимодействие с базой данных

За запись параметров таблицы и ячеек в базу данных отвечают отдельные PHP файлы со скриптами. Данные для записи, в эти файлы отправляются функциями приведенными в предыдущем параграфе. Приведем список файлов для работы с базой данных:

- `attach_cells.php` – файл создает записи в таблицах `tables_users_cells` и `tables_groups_cells`, которые отвечают за хранение данных о привязке ячеек;
- `create_sheet.php` – файл создает записи в таблице `tables_sheets`, которая отвечает хранению данных о листах таблицы;
- `grade_cell.php` – файл создает, или изменяет записи в таблице `tables_cells_grades`, которая отвечает хранению данных о оценках ячеек;
- `load_activity.php` – файл записывает данные из другой активности в ячейки таблицы;
- `load_excel.php` – файл – записывает данные в таблицу из excel файла;
- `remove_attached_cells.php` – файл удаляет строки из таблиц `tables_users_cells` и `tables_groups_cells`, тем самым открепляя ячейки от пользователей, или групп;
- `save_history.php` – файл создает записи в таблице `tables_cells_history`, отвечающей за хранение истории изменения ячеек;
- `send_grade.php` – файл отправляет итоговую оценку в таблицы Moodle, чтобы записать оценку в таблицу студента;
- `update_cell.php` – файл создает, или изменяет запись в таблице `tables_sheets_grade`, отвечающий за хранение данных ячеек;

- `update_cell_focus.php` – файл создает, или изменяет записи в таблице `tables_users_focus`, которая отвечает за хранение данных о взаимодействии пользователей с таблицей в данный момент;
- `update_cell_size.php` – файл создает, или изменяет строки в таблицах `tables_column_width` и `tables_row_height`, которые отвечают за ширину столбцов и высоту строк таблицы;
- `update_font.php` – файл изменяет строки в таблице `tables_sheets_cells`, обновляя параметры содержимого ячейки в соответствии с изменениями;
- `update_visibility.php` – файл изменяет строки в таблице `tables_sheets_cells`, обновляя параметр видимости ячейки.

На страницу данные из таблиц подгружаются встроенными в Moodle функциями для работы с базой данных. Сначала объявляется глобальная переменная `DB`, которая хранит в себе все данные для подключения к БД. Затем используются функции `get_record` для получения данных из таблицы, `insert_record` для записи данных в таблицу, `update_record` для обновления данных в существующей записи, и `record_exist` чтобы проверить существует ли запись в таблице.

3.4. WebSocket

Сам веб-сокет запускается параллельно с сервером на другом порту, который будет указываться в настройках модуля для подключения таблицы к веб-сокету. При переходе на основную страницу модуля клиент будет отправлять запрос на подключение к веб-сокету, передавая ему `id` пользователя для в качестве долина и `id` активности в качестве комнаты, таким образом и передача данных буде происходить только между теми пользователями, которые подключились из одной активности.

Изначально был разработан веб-сокет на основе PHP библиотеки Ratchet рис.3.11 [23]. Ratchet выбирался исходя из того, что сам Moodle написан на PHP и довольно прост в реализации. Но при попытке запустить его на хостинге возник ряд проблем, основной из которых было то, что не на каждом выбранном хостинге удавалось запустить веб-сокет, а на тех, что получилось, веб-сокет блокировался механизмом Cross-Origin Resource Sharing (далее — CORS).

В срочном порядке не получилось решить все проблемы, но было найдено решение в библиотеке Socket.IO для JavaScript. Эта библиотека дает больше возможностей для реализации своего веб-сокета, к примеру удалось разделить


```

6 class Chat implements MessageComponentInterface {
7     protected $clients;
8     public function __construct() {
9         $this->clients = new \SplObjectStorage;
10        echo "Server started";
11    }
12
13    public function onOpen(ConnectionInterface $conn) {
14        $this->clients->attach($conn);
15        echo "New connection! ({\$conn->resourceId})\n";
16    }
17
18    public function onMessage(ConnectionInterface $from, $msg) {
19        $numRecv = count($this->clients) - 1;
20        echo sprintf('Connection %d sending message "%s" to %d other connection%s' . "\n"
21            , $from->resourceId, $msg, $numRecv, $numRecv == 1 ? '' : 's');
22        foreach ($this->clients as $client) {
23            if ($from !== $client) {
24                $client->send($msg);
25            }
26        }
27    }
28
29    public function onClose(ConnectionInterface $conn) {
30        $this->clients->detach($conn);
31        echo "Connection {$conn->resourceId} has disconnected\n";
32    }
33
34    public function onError(ConnectionInterface $conn, \Exception $e) {
35        echo "An error has occurred: {$e->getMessage()}\n";
36        $conn->close();
37    }
38 }

```

Рис.3.11. Пример кода веб-сокета на основе Ratchet

подключенных пользователей по группам, и передача данных будет осуществляться только между пользователями внутри групп, что позволит меньше нагружать сервер.

Код веб-сокета хранится отдельно от плагина в файле `socket.js` рис.3.12

Второй разработанный сокет разработан на основе примере другого веб-сокета [3]. Изначальная версия была разработана для примера чата, но не имела возможности разделять подключаемых пользователей по группам. Также была добавлена возможность подключаться к веб-сокету по уникальному ключу П2.1.

Данные принятые клиентом обрабатываются в функции, прописанной в файле `connect_to_websocket.js` П2.2.

При отправке всегда указывается атрибут `update_type`, в который передаётся в конструкцию `switch case` для определения того, какие элементы на странице нужно обновить.

3.5. Выводы

В данной главе мы рассмотрели подход к разработке плагина для LMS Moodle. Для реализации основного функционала серверная часть плагина была разработана на PHP и клиентская часть на JavaScript с использованием библиотеки `Interact.js`.

```

1  var app = require('express')();
2
3  > const opts = {...}
7
8  var http = require('https').createServer(opts, app);
9  var socket = require('socket.io')(http);
10
11  var port = process.env.APP_PORT;
12
13  > var customer = {...};
19
20  socket.on('connection', function(client){
21      var username = false;
22      var isStaff = false;
23      client.on('login', function(data) {
24          username = data.username;
25          isStaff = data.isStaff;
26      });
27  > client.on('subscribe', function(data) {...});
87
88  > client.on('unsubscribe', function(data) {...});
111
112      client.on('send', function(data) {
113          console.log('sending message');
114  >      socket.in(data.room).emit('message', {sender: username...});
120      });
121  });
122
123  app.get('/customer/queue', function(req, res) {
124      res.json(customer.queue);
125  });
126
127  http.listen(process.env.APP_PORT, process.env.APP_IP, function(){
128      console.log('listening on *:' + process.env.APP_PORT);
129  });

```

Рис.3.12. Пример кода веб-сокета на основе Socket.IO в файле socket.js П2.1

Для реализации передачи данных в реальном времени был разработан веб-сокет на основе веб-сокета Example-chatroom [12] с использованием библиотеки Socket.IO.

ГЛАВА 4. ТЕСТИРОВАНИЕ И АПРОБАЦИЯ РАЗРАБОТАННОГО ПЛАГИНА

В параграфе 4.1 четвертой главы описан выбор средств тестирования разработанного плагина. В параграфе 4.2 описаны PHPUnit тесты, проверяющие работоспособность функций, которые отвечают за запись данных в БД. В параграфе 4.3 описаны Jest тесты, проверяющие работоспособность JavaScript функций, которые отвечают за визуализацию данных. В параграфе 4.4 описано тестирование веб-сокета. В параграфе 4.5 описан процесс выгрузки сервера Moodle с плагином и веб-сокетом на хостинг, а в параграфе 4.6 описано тестирование плагина на этом хостинге группой людей. В параграфе 4.7 описано соответствие результатов разработки с поставленными задачами.

4.1. Выбор подходов к тестированию

Для тестирования разработанного плагина выбраны несколько способов тестирования отдельных компонентов плагина:

1. Модульное тестирование для проверки серверной части плагина. В основном PHP функций и правильной записи строк в базу данных.
2. Модульное тестирование основных функций плагина, реализующих взаимодействие пользователя с таблицей.
3. API Тестирование веб-сокета проводилось с помощью Postman.

Нагрузочное тестирование плагина не проводилось, так как тестироваться будет сам Moodle и Web-Socket, которые хорошо справляются с большим количеством, одновременно работающих, пользователей.

Тестирование скорости отклика также зависит от скорости ответа сервера на котором находится веб-сокет, а функции которые применяются при обработке ответа практически не нагружают работу плагине.

4.2. Модульное тестирование с PHPUnit

Для проведения модульных тестов был выбран фреймворк для тестирования PHPUnit. Он уже встроен в Moodle и большая часть тестов в нем написана с использованием данного инструмента. Дополнительно для запуска тестов необходимо установить Composer. Все тесты находятся в файле `tables_unit_tests.php` в

папке tests, пример можно увидеть в П1.11. Файл содержит следующие тестовые функции:

- `get_column_width_test`, `get_row_height_test`, `get_cell_font_family_test`, `get_cell_font_size_test`, `get_cell_bold_test`, `get_cell_italic_test`, `get_cell_underline_test`, `get_cell_align_test` – функции проверяющие корректность получаемых из базы данных параметров ячейки таблицы. Тесты создают записи в БД и проверяют корректность возвращаемых функциями значений;
- `get_cell_range_test` – функция проверяет возврат диапазона ячеек для прикрепления. На вход подаются два имени ячейки, представляющие две угловые ячейки в многомерной матрице. Функция должна возвращать первую и последнюю ячейки этой матрицы;
- `generate_column_name_test` – функция проверяет корректность имен колон таблицы. На вход подается номер колонны, на выходе идет её имя в виде букв алфавита;
- `create_sheet_test` – функция проверяет корректность создания нового листа таблицы. На вход подаются имя нового листа, id таблицы, которой он принадлежит, а также тип взаимодействия с базой данных. Функция, в зависимости от типа взаимодействия создает, или удаляет строку в БД с указанными данными, тест проверяет существование, или отсутствие записи;
- `update_cell_test` – функция проверяет корректность записи в ячейку новых данных. На вход подаются параметры ячейки. Функция обновляет эти данные в БД. Тест проверяет существование записи и корректность данных;
- `attach_cells_test` – функция проверяет корректность прикрепления ячеек к пользователям и группам. На вход подается id листа, id пользователя, или группы, имя таблицы, имена двух ячеек. Функция создает запись в указанной базе данных. Тест проверяет существование записи;
- `rate_cell_test` – функция проверяет корректность оценки ячеек. На вход подаются id пользователя, id ячейки, оценка в виде числа, и отзыв в виде строки. Функция создает запись в базе данных с оценками ячеек. Тест проверяет существование записи и корректность введенных данных.

Модульные тесты покрывают основную часть функционала плагина отвечающего за создание и получение записей в базе данных.

```

PS D:\Program Files\MoodleServer401\server\moodle> vendor/bin/phpunit mod/tables/tests/tables_unit_tests.php
Moodle 4.1.6+ (Build: 20231102)
Php: 8.0.28, mariadb: 10.4.28-MariaDB, OS: Windows NT 10.0 AMD64
PHPUnit 9.5.28 by Sebastian Bergmann and contributors.

.....                                     14 / 14 (100%)

Time: 00:04.170, Memory: 60.00 MB

OK (14 tests, 33 assertions)
PS D:\Program Files\MoodleServer401\server\moodle>

```

Рис.4.1. Результат прохождения PHPUnit тестов

4.3. Модульное тестирование с Jest

Для тестирования клиентской части плагина и его JavaScript функций был выбран фреймворк для тестирования Jest [16]. Для того, чтобы имитировать взаимодействие пользователя со страницей, перед началом тестов создается тестовая разметка страницы П1.12.

```

PASS ./tables.test.ts
tables_tests
  ✓ Attach users test (48 ms)
  ✓ Attach groups test (10 ms)
  ✓ Submit attach users test (7 ms)
  ✓ Submit attach groups test (7 ms)
  ✓ Cansel attach test (14 ms)
  ✓ Update Cell test (1 ms)
  ✓ Click ceckbox attach test (1 ms)
  ✓ Attach focus cell test (3 ms)
  ✓ Focus cell test (4 ms)
  ✓ Change input cell test (5 ms)
  ✓ Change input content test
  ✓ Update font test (4 ms)
  ✓ Create sheet test (2 ms)
  ✓ Grade cell test
  ✓ onChangeInputGrade test
  ✓ oninputGrade test
  ✓ showFeedback test (1 ms)
  ✓ onChangeSelectVisibility test (1 ms)
-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines |
-----|-----|-----|-----|-----|
All files |    77.47 |    87.65 |    73.91 |    77.47 |
update_data.js |    77.47 |    87.65 |    73.91 |    77.47 |
-----|-----|-----|-----|-----|
Test Suites: 1 passed, 1 total
Tests:       18 passed, 18 total
Snapshots:   0 total
Time:        1.606 s

```

Рис.4.2. Результат прохождения Jest тестов

Для тестирования основных функций плагина были написаны следующие тесты:

- Attach users test и Attach groups test – тесты имитируют нажатие пользователем на кнопку прикрепления пользователей, или групп. Функ-

ция должна заблокировать остальные элементы панели инструментов и отобразить элементы для прикрепления ячеек;

- Submit attach test и Submit attach groups test – тесты имитируют подтверждение прикрепления ячеек. Функция должна скрыть элементы прикрепления и разблокировать остальные элементы на панели инструментов;
- Cansel attach test – тест имитирует отмену прикрепления ячеек. Функция должна очистить поля для ввода и снять выбор с отмеченных ячеек;
- Update Cell test – тест имитирует ввод текста в ячейку. Функция должна отобразить введенный текст в поле содержимого ячейки под панелью инструментов;
- Click checkbox attach test – тест имитирует выбор студентов для прикрепления. Функция должна выводить имя выбранного студента в окно выбранных студентов;
- Attach focus cell test – тест имитирует выбор ячеек для прикрепления. Функция должна выводить выбранные ячейки в окно выбранных ячеек и изменять цвет границы выбранных ячеек;
- Focus cell test – тест имитирует фокус на ячейку на странице браузера. Функция должна выводить параметры ячейки на панель инструментов;
- Change input cell test – тест имитирует изменение текста в окне выбора ячейки. Функция должна перемещать фокус на введенную ячейку;
- Change input content test – тест имитирует изменение текста в окне содержимого ячейки под панелью инструментов. Функция должна изменять содержимое ячейки в соответствии с введенным текстом;
- Update font test – тест имитирует изменение параметров шрифта ячейки. Функция должна, в зависимости от нажатой кнопки, изменить параметры шрифта ячейки;
- Create sheet test – тест имитирует создание нового листа таблицы. Функция должна отобразить на странице кнопку для перехода к новому листу;
- Grade cell test – тест имитирует подтверждение оценки ячейки. Функция должна отобразить элемент подтверждающий, что оценка прошла успешно и очистить окно для отзыва;

- Change grade test – тест имитирует изменение текста поля для ввода оценки. Функция должна скрыть элемент подтверждающий, что оценка прошла успешно;
- Wrong input grade test – тест имитирует ввод неправильной оценки в поля для ввода оценки. Функция должна изменить введенное число на максимально, или минимально допустимое;
- Show feedback test – тест имитирует нажатие на кнопку для показа окна для ввода отзыва. Функция должна отобразить окно для отзыва на панели инструментов;
- Change visibility test – тест имитирует изменение видимости ячейки. Функция должна изменить атрибут видимости ячейки.

Как видно из результатов тестирования рис.4.2, тесты покрывают 70 процентов функций отвечающих за отображение изменений на странице. Вместе с PHPUnit тестами они закрывают практически весь основной функционал плагина.

4.4. Тестирование веб-сокета

Тестирование веб-сокета проводилось в сервисе для тестирования API – Postman [22]. В приложении мы подключаемся к веб сокету. Подключаемся к сокету рис.4.3 и заходим в, созданную заранее, тестовую комнату рис.4.4.

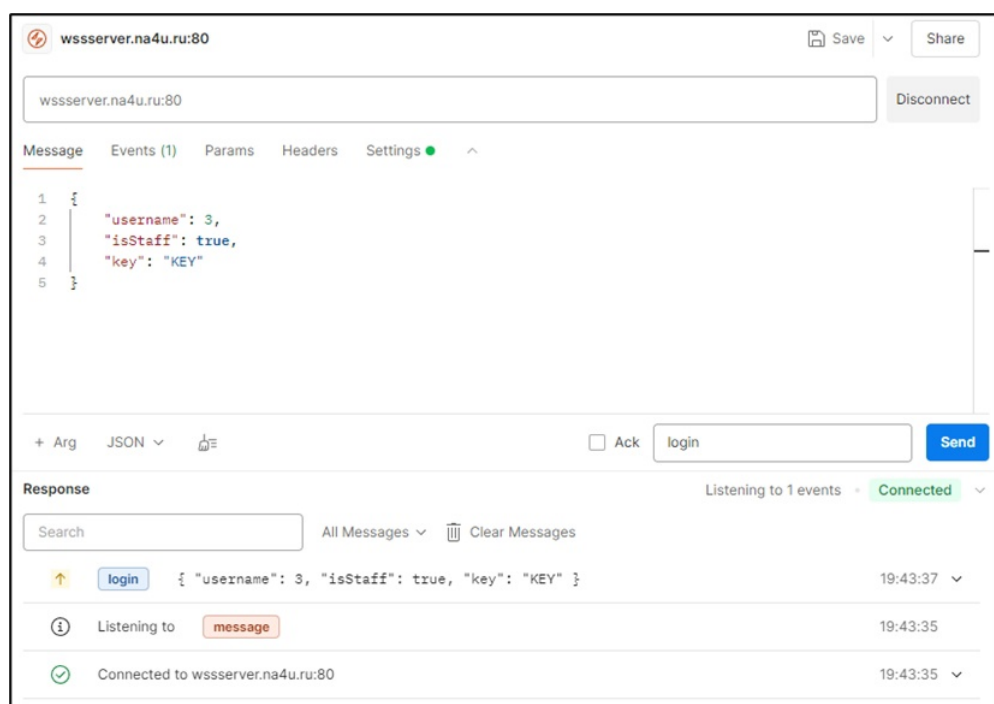


Рис.4.3. Команда для логина к веб-сокету

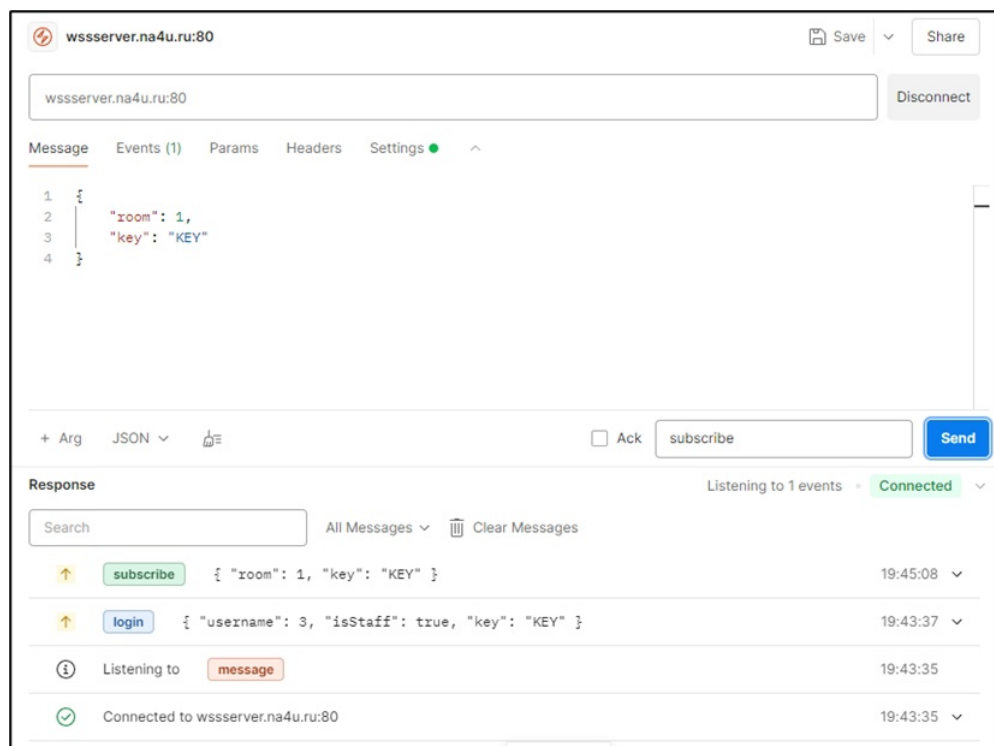


Рис.4.4. Команда для подключения к комнате веб-сокета

При обновлении страницы тестовой таблицы приходит сообщение об обновлении в Postman рис.4.5.

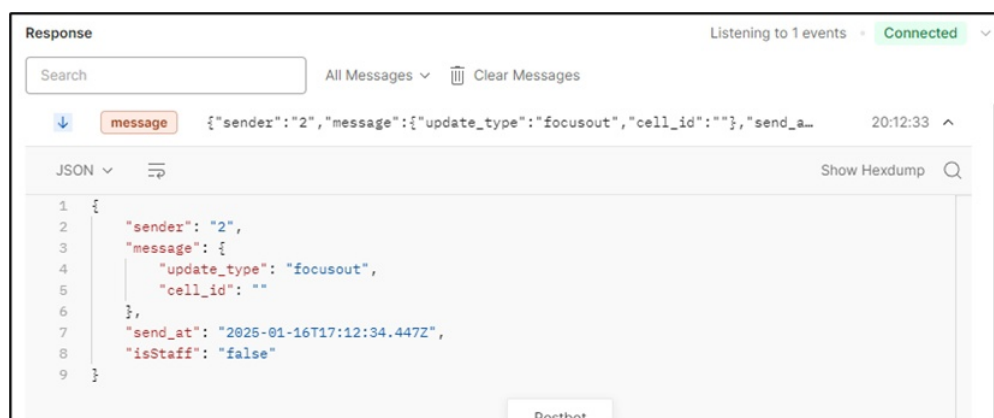


Рис.4.5. Принято сообщение из таблицы

Отправляем таблице тестовое сообщение рис.4.6. При успешной обработке сообщения веб-сокетом и плагином, текст в таблице должен измениться на «Test text».

На рисунке рис.4.7 видно, что Postman принял ответное сообщение от веб-сокета с теми данными, которые мы отправили.

В тестовой таблице также наблюдаем изменения текста в ячейке A1 рис.4.8. Для тестирования всех функций были отправлено несколько команд:

- команда для тестирования ввода текста;

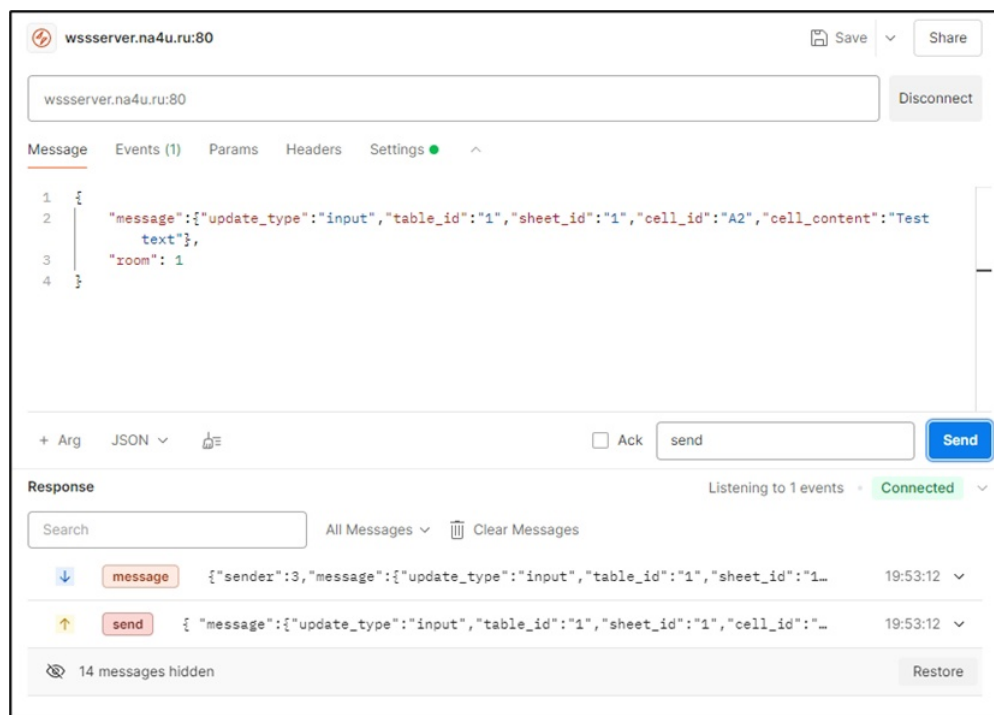


Рис.4.6. Отправка тестового сообщения



Рис.4.7. Сравнение отправленных и принятых данных

- команда для тестирования изменения видимости ячейки;
- команда для тестирования изменения фокуса ячейки;
- команда для тестирования отмены фокуса ячейки;
- команда для тестирования прикрепления ячеек;
- команда для тестирования удаления прикрепления ячейки;
- команда для тестирования удаления прикрепления всех ячеек;
- команда для тестирования изменения размера высоты строк;
- команда для тестирования изменения ширины столбцов;
- команда для тестирования изменения шрифта текста ячейки.

На все команды веб-сокета должен был отправить ответ с теми же данными, что отправлялись. Также проверялись изменения в тестовой таблице.

	A	B	C	D	E
1	Test text				
2					
3					
4					
5					

Рис.4.8. Изменения в тестовой таблице

Этим командами полностью тестируется, отправка запросов на сервер веб-сокета и обработка ответа в плагине.

4.5. Загрузка на хостинг

Для проверки работоспособности плагина был выбран хостинг netangels.ru [1], так как он является дешевым, поддерживает PHP для запуска Moodle, NodeJS для запуска веб-сокета и имеет подробный пример установки веб-сокета на этот хостинг [3]. Веб-сокет из этого примера был доработан и используется в этой работе.

Сервер Moodle и веб-сокет запускались на хостинге с версией PHP 7.4 для запуска Moodle и Node.js 23.3.0 для запуска веб сокета с библиотекой socket.io. Для сайта, хостингом, было выделено 512 Мб RAM и 100CP CPU. Сокет запускается на домене <http://wsssocket.na4u.ru>. Moodle запускается на домене <http://moodle.na4u.ru>.

Для тестирования были созданы две учетные записи учителей с логинами teacher1 и teacher2, и 6 учетных записей студентов с логинами student1, student2, student3, student4, student5, student6. Все учетные записи имеют пароль P@sw0rd3530903.

4.6. Апробация

В первую очередь целью апробации являлось проверить работоспособность веб-сокета и его взаимодействие с Moodle. Также проверялось работоспособность тех функций, которые не удалось проверить предыдущими тестами. Это те функции плагина, что отвечают за прием данных от веб сокета и отображении их на экране пользователя.

Для тестирования проводилось на группе из 6 человек. В ходе тестирования тестировщики разделились на две группы и выполняли часть задания по проведению SWOT-анализа гипотетического ИТ-проекта. Для каждой группы был создан лист таблицы с заданием. Таблица содержит следующие ячейки:

- A2 - strengths (S);
- A3 – weaknesses (W);
- B1 – opportunities (O);
- C1 – threats (T).

Эти ячейки заранее заполняются преподавателем, а пересечение столбцов и строк этих ячеек (ячейки B2, B3, C2, C3) выделяются студентам, по ячейке на человека, четвертая ячейка является общей. Студенты заполняют пересечения SO, ST, WO и WT в соответствии со своим проектом. Группы не видят результаты работы друг друга. Ячейки оцениваются по 100 бальной шкале, общий результат отображается на странице с оценками, где может корректироваться преподавателем. Полученные результаты сравнивались и оценивались по завершении работы итоговая оценка опраивлялась в таблиць оценок активности курса.

По результатам апробации был проведен опрос о качестве работы таблицы и удобстве интерфейса. Опрос состоял из следующих вопросов:

- Оцените по пятибальной шкале удобство пользовательского интерфейса таблицы.
- Оцените по пятибальной шкале скорость отображения информации от других пользователей.
- Оцените по пятибальной шкале общую работоспособность таблицы.
- С какими ошибками, или проблемами вы столкнулись при работе с таблицей?
- Каких функций вам нехватало при работе с таблицей?

По результатам оценочного опроса можно сделать вывод, что большинство респондентов оценили разработанный плагин положительно, средний балл равен

4,67. Текстовый опрос выявил несколько ошибок и недочетов, которые были исправлены, также респондентами предлагалось добавить возможность цветового выделения ячеек таблицы для удобства считывания информации.

Тот же опрос был проведен с преподавателями составлявшими задание, и проводившими оценивание результатов работ тестирующих. Отзывы преподавателей также положительны: "все отлично работает", "функциональность понравилась", "удобно".

Задание внедрено в курс «Методы принятия решений» программы магистратуры 09.04.03 Прикладная информатика.

4.7. Соответствие критериям оценивания

Обращаясь к ранее показанной табл.1.2 можно увидеть, что заявленные возможности таблицы выполнены, и хотя у аналогов более богатый функционал, разработанное решение имеет ряд уникальных возможностей:

- настройка прав доступа к отдельным ячейкам как для отдельных пользователей, так и для групп;
- ограничение просмотра содержимого для каждой ячейки. Возможность выставить режим просмотра для всех, для отдельных групп, для студентов, или только для преподавателей;
- возможность оценивания ячеек студентов индивидуально, или в ходе групповой работы. Результаты отображаются в отдельном окне, где также выводится суммарная оценка. Если студент состоит в группе, то выводится суммарная оценка для группы;
- возможность выгружать результаты прохождения тестов в таблицу.

Кроме того, в сочетании с Moodle таблица имеет:

- аутентификацию по SSO через учетные записи университета;
- возможность мониторинга активности учащихся через стандартные средства Moodle;
- открытый исходный код и удобство поддержки за счет унифицированной работы плагинов Moodle.

Точность работы большей части функционала подтверждается модульными и API тестами.

4.8. Выводы

В данной главе были выбраны методики тестирования разработанного плагина. Проведены соответствующие тестирования разработанного плагина. Проведена апробация плагина на группе из 6 человек. Тестирование показало, что разработанный плагин работоспособен и может выполнять задачи образовательного процесса в Moodle.

ЗАКЛЮЧЕНИЕ

В ходе исследования и разработки, в соответствии с поставленной целью были получены следующие результаты:

- проведен анализ возможностей Moodle совместных табличных технологий;
- проведено сравнение возможностей Moodle с современными электронными таблицами;
- спроектирован и разработан плагин для работы с совместными электронными таблицами в системе Moodle;
- проведено тестирование и апробация разработанного плагина.

Практическая значимость данной работы заключается в расширении возможностей Moodle. Созданный плагин позволяет работать с таблицей в реальном времени, настраивать таблицу для выполнения различных задач и проводит оценку работы в рамках системы Moodle, что может быть востребовано различными организациями или университетами. Кроме того, на основе разработанного плагина возможно создание новых плагинов, или расширение возможностей уже существующего решения.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

- LMS** Learning Management Systems.
- Moodle** Modular Object-Oriented Dynamic Learning Environment.
- SSE** Server-Sent Events.
- CORS** Cross-Origin Resource Sharing.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. NetAngels. — URL: <https://www.netangels.ru/> (дата обращения: 28.12.2023).
2. WebSocket: разбираем как работает. — URL: <https://habr.com/ru/sandbox/171066> (дата обращения: 28.12.2023).
3. Установка веб-сокета. — URL: <https://www.netangels.ru/support/cloud-hosting/websockets> (дата обращения: 28.12.2023).
4. Чтение файла XLSX на PHP без сторонних библиотек. — URL: https://www.manhunter.ru/webmaster/1969_chtenie_fayla_xlsx_na_php_bez_storonnih_bibliotek.html (дата обращения: 28.12.2023).
5. Adding Process-Driven Collaboration Support in Moodle. — URL: <https://ieeexplore.ieee.org/document/5350457> (visited on 08.12.2009).
6. Airtable documentation. — URL: <https://support.airtable.com/docs/> (visited on 20.10.2023).
7. Apple Numbers user manual. — URL: <https://manualsbrain.com/en/manuals/1776940/> (visited on 20.10.2023).
8. Chriselle Sy. Airtable vs Google Sheets: The Ultimate 2023 Comparison. — URL: <https://spreadsheetpoint.com/airtable-vs-google-sheets/> (visited on 20.10.2023).
9. Database schema introduction. — URL: https://docs.moodle.org/dev/Database_schema_introduction (visited on 01.11.2023).
10. Emma Collins. Excel vs Numbers: Which Is the Superior Spreadsheet Program. — URL: <https://spreadsheetpoint.com/excel-vs-numbers/> (visited on 20.10.2023).
11. esup-mod-rocketchat. — URL: <https://github.com/EsupPortail/esup-mod-rocketchat/tree/main/> (visited on 18.01.2024).
12. Example-chatroom. — URL: <https://github.com/ethaizone/Example-chatroom-with-NodeJS-PHP> (visited on 28.12.2023).
13. Excel help learning. — URL: <https://support.microsoft.com/en-us/excel/> (visited on 20.10.2023).
14. Google Sheets documentation. — URL: <https://developers.google.com/sheets/api/reference/rest/v4/spreadsheets/sheets?hl=en> (visited on 20.10.2023).
15. Google Sheets vs Excel – Which Is Better In 2023. — URL: <https://spreadsheetpoint.com/excel-vs-google-sheets/> (visited on 20.10.2023).
16. Jest. — URL: <https://jestjs.io> (visited on 28.12.2023).
17. Kotama I. N. D., Saputra K. O., Linawati L. Proposed Model of Multiplayer Matching Game Plugins Using Websocket in Moodle // International Journal of Emerging

Technologies in Learning / ed. by N. L. Jon-Chao Hong. — 2019. — Vol. 14(11). — P. 194–201. — URL: <https://online-journals.org/index.php/i-jet/article/view/10190/5727> (visited on 18.01.2024).

18. Moodle. — URL: <https://moodle.org> (visited on 28.12.2023).

19. Moodle architecture. — URL: https://docs.moodle.org/dev/Moodle_architecture (visited on 01.11.2023).

20. moodle-assignment-rtcollaboration. — URL: https://github.com/jleyva/moodle-assignment_rtcollaboration/ (visited on 18.01.2024).

21. Plugin skeleton generator. — URL: https://moodle.org/plugins/tool_pluginskel (visited on 28.12.2023).

22. Postman. — URL: <https://www.postman.com/product/what-is-postman/> (visited on 28.12.2023).

23. Ratchet. — URL: <http://socketo.me/docs> (visited on 28.12.2023).

24. Why Moodle. — URL: <https://ieeexplore.ieee.org/document/4683115> (visited on 21.11.2008).

25. Yandex Cloud documentation. — URL: <https://cloud.yandex.com/en/docs> (visited on 20.10.2023).

Код разработанного плагина

П1.1. view.php

```

<?php

require(__DIR__.'../../config.php');
require_once(__DIR__.'lib.php');

global $DB, $USER, $CFG, $OUTPUT, $PAGE;

// Course module id.
5 $id = optional_param('id', 0, PARAM_INT);
// Activity instance id.
$t = optional_param('t', 0, PARAM_INT);

if ($id) {
15     $cm = get_coursemodule_from_id('tables', $id, 0, false,
        MUST_EXIST);
    $course = $DB->get_record('course', array('id' => $cm->
        course), '*', MUST_EXIST);
    $moduleinstance = $DB->get_record('tables', array('id' =>
        $cm->instance), '*', MUST_EXIST);
} else {
    $moduleinstance = $DB->get_record('tables', array('id' =>
20     $t), '*', MUST_EXIST);
    $course = $DB->get_record('course', array('id' =>
        $moduleinstance->course), '*', MUST_EXIST);
    $cm = get_coursemodule_from_instance('tables',
        $moduleinstance->id, $course->id, false, MUST_EXIST);
}

require_login($course, true, $cm);

25 $modulecontext = context_module::instance($cm->id);
$coursecontext = context_course::instance($course->id);

$event = \mod_tables\event\course_module_viewed::create(
    array(
30     'objectid' => $moduleinstance->id,
    'context' => $modulecontext

```

```

));
$event->add_record_snapshot('course', $course);
$event->add_record_snapshot('tables', $moduleinstance);
35 $event->trigger();

$PAGE->set_url('/mod/tables/view.php', array('id' => $cm->id
));
$PAGE->set_title(format_string($moduleinstance->name));
$PAGE->set_heading(format_string($course->fullname));
40 $PAGE->set_context($modulecontext);

$PAGE->requires->jquery();
$PAGE->requires->js(new moodle_url($CFG->wwwroot . '/mod/
    tables/amd/src/connect_to_websocket.js?v=4.4'));
$PAGE->requires->js(new moodle_url($CFG->wwwroot . '/mod/
    tables/amd/src/update_data.js?v=7.6'));
45 $PAGE->requires->js(new moodle_url($CFG->wwwroot . '/mod/
    tables/amd/src/interact_resize.js?v=2.1'));
$PAGE->requires->js(new moodle_url($CFG->wwwroot . '/mod/
    tables/amd/src/jquery_tables_functions.js?v=1.2'));

$roles = get_default_enrol_roles($modulecontext);
$user_roles = get_user_roles_in_course($USER->id, $course->
    id);
50

echo'<script>
localStorage.clear();
</script>';

55 if((strpos($user_roles, $roles[1]) !== false) || (strpos(
    $user_roles, $roles[3]) !== false)){
    $user_activity_role = "teacher";
    echo'<script>
        localStorage.activity_role = "'.$user_activity_role.'"
        </script>';
60 }
elseif(strpos($user_roles, $roles[4]) !== false){
    $user_activity_role = "assistant";
    echo'<script>
        localStorage.activity_role = "'.$user_activity_role.'"
65 </script>';
}
else{
    $user_activity_role = "student";
    echo'<script>

```

```

70     localStorage.activity_role = "'. $user_activity_role.'"
    </script>';
}

if($DB->record_exists('tables_users_focus', array('tableid'
=> $moduleinstance->id, 'userid' => $USER->id))){
75     $user_focus_data = $DB->get_record('tables_users_focus',
        array('tableid' => $moduleinstance->id, 'userid' =>
            $USER->id));
    $prev_cell = $user_focus_data->focused_cell;
    echo '<input hidden id="prev_element" type="text" value="'.
        $prev_cell.'" />';
    $user_focus_data->focused_cell = null;
    if($_POST["sheet"]){
80         $active_sheet = $DB->get_record('tables_sheets', array('
            id' => $_POST["sheet"]));
        $user_focus_data->active_sheet = $active_sheet;
    }
    else{
        $active_sheet = $DB->get_record('tables_sheets', array(
            'id' => $user_focus_data->active_sheet));
85    }
    $DB->update_record('tables_users_focus', $user_focus_data)
        ;
}
else{
    $active_sheet = $DB->get_record('tables_sheets', array('
        tableid' => $moduleinstance->id));
90    $DB->insert_record('tables_users_focus', array('tableid'
        => $moduleinstance->id, 'userid' => $USER->id, "
        active_sheet" => $active_sheet->id,
        'timecreated' => time()));
}

if($active_sheet->activityloadid != "" && $active_sheet->
    activityloadtype != "" && $active_sheet->
    updateonreloadpage != "false"){
95    load_from_activity($modulecontext, $course, $active_sheet
        ->id, $active_sheet->activityloadtype, $active_sheet->
        activityloadid);
}

//Toolbar
$fonts = array('Arial', ... );
100

```

```

echo $OUTPUT->header();

echo '
105 <div class="m-tables-toolbar">
    <div class="m-tables-toolbar-block">
        <form class="m-tables-toolbar-load-up" method="post"
            action="upload_from_xlsx.php?id='.$id.'">
            <button class="m-tables-toolbar-button" type="submit">
                
            </button>
110 </form>
        <form class="m-tables-toolbar-load-down" method="post"
            action="upload_from_activity.php?id='.$id.'">
            <button class="m-tables-toolbar-button" type="submit">
                
            </button>
115 </form>
    </div>
    <div id="toolbar_font" class="m-tables-toolbar-block">
        <div class="m-tables-toolbar-font-up">
            <input class="m-tables-font-family-selector" id="font-
                family-selector" title="'.get_string('
                    font_family_title', 'mod_tables')." name="font-
                    family-selector" type="text" value="Calibri"
                    autocomplete="off" onchange="updateFont(this)" list
                        ="fonts"/>
120 <datalist id="fonts">';
            foreach ($fonts as &$font){
                echo '<option value="'. $font.'">'. $font.'</option>
                    '
            }
            echo '</datalist>
125 <input class="m-tables-font-size-selector" id="font-
                size-selector" title="'. get_string('
                    font_size_title', 'mod_tables')." onchange="
                    updateFont(this)" type="number" min="1" max="409"
                    value="11" xmlns="http://www.w3.org/1999/html"/>
        </div>
        <div class="m-tables-toolbar-font-down">
            <button class="m-tables-toolbar-button" id="font-bold-
                button" onclick="updateFont(this)" title="'.
                    get_string('font_bold_title', 'mod_tables').">

```



```

130         
    </button>
    <button class="m-tables-toolbar-button" id="font-italic-
        button" onclick="updateFont(this)" title="''.
        get_string('font_italic_title', 'mod_tables').'">
        
    </button>
    <button class="m-tables-toolbar-button" id="font-
        underline-button" onclick="updateFont(this)" title="''.
        get_string('font_underline_title', 'mod_tables').'">
135     
    </button>
</div>
</div>

140 <div id="toolbar_align" class="m-tables-toolbar-block">
    <div class="m-tables-toolbar-align">
        <button class="m-tables-toolbar-button" id="text-left-
            button" onclick="updateFont(this)" title="''.
            get_string('text_align_left_title', 'mod_tables').'"
            >
            
        </button>
145 <button class="m-tables-toolbar-button" id="text-center-
            button" onclick="updateFont(this)" title="''.
            get_string('text_align_center_title', 'mod_tables').'"
            >
            
        </button>
        <button class="m-tables-toolbar-button" id="text-right-
            button" onclick="updateFont(this)" title="''.
            get_string('text_align_right_title', 'mod_tables').'"
            >
            
150 </button>
    </div>
    <div class="m-tables-toolbar-align-down">

</div>

```

```

155     </div>';

    if($user_activity_role == "teacher") {
        echo '
160     <div class="m-tables-toolbar-block-attach">
        <div class="m-tables-toolbar-attach">
            <button class="m-tables-toolbar-button" id="
                attach_cell_to_users" onclick="onclickAttach(this)"
                title="' . get_string('attachcellstostudents', '
                mod_tables') . '" value="off" data-attach-to="user">
                
            </button>
            <div class="m-dropdown" id="dropdown_attach_students"
                style="display:none;" >
165         <div class="m-dropdown-display">
            <input class="m-dropdown-checked" type="text" id="
                display_selected_students">

            <input class="m-dropdown-search" autocomplete="off"
                type="text" oninput="onInputSearch(this)" id="
                search_students" data-attach-to="user">
            </div>
170         <div class="m-dropdown-content" id="dropdown-content-
            users">';

        $role_users = get_role_users(5, $coursecontext);

        foreach ($role_users as $user) {
175         echo '
            <p>
                <input class="m-user-check" data-attach-to="user"
                data-attach-name="' . $user->firstname . " " .
                $user->lastname . '" value="' . $user->id . '"
                type="checkbox" onclick="onclickCheckboxAttach(
                this)">
                <label class="m-tables-user-label">' . $user->
                firstname . " " . $user->lastname . '</label>
            </p>';
180     }
    echo '
        </div>
    </div>
    <input class="m-dropdown-students-cell" id="first_cell-
        students" type="text" readonly>

```

```

185 <input class="m-dropdown-students-cell" id="last_cell-
      students" type="text" readonly>
<div id="submit_user_btns" style="display: none">
  <span class="m-tables-green-btn">
    <i class="fa fa-check" data-attach-to="user" onclick
      ="onclickSubmitAttach(this, messages)" ></i>
  </span>
190 <span class="m-tables-red-btn">
    <i class="fa fa-times" data-attach-to="user" onclick
      ="onclickCanselAttach(this)" ></i>
  </span>
</div>
</div>
195 <div class="m-tables-toolbar-attach">
  <button class="m-tables-toolbar-button" id="
    attach_cell_to_groups" onclick="onclickAttach(this)"
    title="" value="off" data-attach-to="group">
    
  </button>
<div class="m-dropdown" id="dropdown_attach_groups" style
  ="display:none;" >
200 <div class="m-dropdown-display">
  <input class="m-dropdown-checked" type="text" id="
    display_selected_groups">
  <input class="m-dropdown-search" autocomplete="off"
    type="text" oninput="onInputSearch(this)" id="
    search_groups" data-attach-to="group">
</div>
<div class="m-dropdown-content" id="dropdown-content-
  groups">';
205
$groups = groups_get_all_groups($course->id);

foreach ($groups as $group) {
  echo '
210 <p>
    <input class="m-group-check" data-attach-to="group"
      data-attach-name="' . $group->name . '" value=" '
      . $group->id . '" type="checkbox" onclick="
        onclickCheckboxAttach(this)">
    <label class="m-tables-user-label">' . $group->name
      . '</label>
    </p>';
}

```

```

215     echo '
        </div>
    </div>
    <input class="m-dropdown-groups-cell" id="first_cell -
        groups" type="text" readonly>
    <input class="m-dropdown-groups-cell" id="last_cell-groups
        " type="text" readonly>

220
        <div id="submit_group_btns" style="display: none">
            <span class="m-tables-green-btn">
                <i class="fa fa-check" data-attach-to="group"
                    onclick="onclickSubmitAttach(this, messages)" ></i>
                <i>
            </span>
            <span class="m-tables-red-btn">
                <i class="fa fa-times" data-attach-to="group"
                    onclick="onclickCanselAttach(this)" ></i>
            </span>
        </div>
    </div>
230 </div>
<div id="grade_block" class="m-tables-toolbar-block disabled
    ">
    <div class="m-tables-toolbar-grade-up">
        <select id="select_user_grade">';
    $sheet_users = get_role_users(5, $modulecontext, true, 'u.
        id, u.firstname, u.lastname');
235 foreach ($sheet_users as $user) {
    echo '
        <option value="' . $user->id . '">
            ' . $user->lastname . ' ' . $user->firstname . '
        </option>';
240 }
    echo '
        </select>
    </div>
    <div class="m-tables-toolbar-grade-down">
245     <input id="input_grade" type="number" min="0" max="100"
        oninput="oninputGrade(this)" onchange="
            onChangeInputGrade()">
        <i class="fa fa-check m-tables-check-grade" id="
            check_grade"></i>
        <button class="btn btn-primary m-tables-toolbar-grade-
            button" onclick="gradeCell()">Оценить</button>
        <span class="m-tables-blue-btn">

```

```

250         <i class="fa fa-plus" id="show_feedback_btn" onclick="
            showFeedback(this)"></i>
        </span>
    </div>
</div>
    <div id="feedback_block" class="m-tables-toolbar-block m-
        tables-toolbar-grade-textarea" style="display: none">
        <textarea id="feedback_textarea"></textarea>
255    </div>
    <div class="m-tables-toolbar-block">
        <div id="visibility_block" class="m-tables-toolbar-
            visible-up disabled">
            <select id="select_cell_visibility" onchange="
                onChangeSelectVisibility()">
                <option value="teacher">'.get_string("visibleteacher
                    ", 'mod_tables').'</option>
260            <option value="all">'.get_string("visibleall", '
                mod_tables').'</option>
            <option value="user">'.get_string("visibleuser", '
                mod_tables').'</option>
            <option value="group">'.get_string("visiblegroup", '
                mod_tables').'</option>
            </select>
        </div>
265    </div>';
}
echo '
</div>';

270 //Input bar
echo '
    <div class="m-tables-input-bar" id="input_bar">
        <input style="display: none" type="text" id="prev_cell">
        <input class="m-tables-focused-cell" type="text" id="
            focused_cell" onchange="onChangeInputCell(this)" />
275    <input class="m-tables-focused-cell-content" type="text"
        onchange = "onChangeInputContent(this)" id="
            focused_cell_content" />
    </div>';

    //Table
    $rows = $moduleinstance->rowcount;
280    $columns = $moduleinstance->columncount;

    echo '

```

```

285 <div class="m-tables-settings">
    <table id="main_table" data-id="' . $id.'" data-
        moduleinstance="' . $moduleinstance->id.'" data-sheet="' .
        $active_sheet->id.'" data-user-role="' .
        $user_activity_role.'" data-user="' . $USER->id.'">
    <thead>
        <tr>
            <td></td>';
for ($column = 0; $column < $columns; $column++) {
    $columnname = generate_column_name($column);
290 $columnwidth = get_column_width("col_". $columnname,
        $active_sheet->id);
    echo '
        <td>
            <input class="resizable-column" type="text" id="
                col_'. $columnname.'" style="width: '.
                $columnwidth.'px;" value="' . $columnname.'"
                readonly />
            </td>';
295 }
    echo '
        </tr>
        </thead>
        <tbody>';
300 for ($row = 1; $row <= $rows; $row++) {
    $rowheight = get_row_height("row_". $row, $active_sheet->id
    );
    echo '
        <tr>
            <td>
305         <input class="resizable-row" type="text" id="row_'
                . $row.'" style="height:'. $rowheight.'px;" value
                ="' . $row.'" readonly />
            </td>';
        for ($column = 0; $column < $columns; $column++) {
            $cell = array('name' => generate_column_name($column).
                $row, 'sheetid' => $active_sheet->id);
            $useronfocus = null;
310
            if($DB->record_exists('tables_users_focus', array('
                focused_cell' => $cell['name'], 'active_sheet' =>
                $cell['sheetid']))) {
                $useronfocus = $DB->get_record('tables_users_focus',
                array('focused_cell' => $cell['name'], 'active_sheet'
                => $cell['sheetid']), '*', MUST_EXIST);

```

```

315     }

    if($user_activity_role == 'teacher'){
        $disablecell = '';
    }
    else{
320         $disablecell = 'disabled';
        $group_visibility = 'false';
        $user_visibility = 'false';
    }

325     $user_groups = groups_get_user_groups($course->id, $USER
        ->id);

    foreach($user_groups as $user_group){
        foreach($user_group as $group_id){
            if($DB->record_exists('tables_groups_cells', array('
                sheetid' => $active_sheet->id, 'groupid' =>
                $group_id, 'cellname' => $cell['name']))) {
330                 $disablecell = '';
                $group_visibility = 'group';
            }
        }
    }

335     if($DB->record_exists('tables_users_cells', array('
        sheetid' => $active_sheet->id, 'userid' => $USER->id,
        'cellname' => $cell['name']))) {
        $disablecell = '';
        $user_visibility = 'user';
    }

340     if($useronfocus->userid != null && $useronfocus->userid
        != $USER->id){
        $disablecell = 'disabled';
    }

345     if($DB->record_exists('tables_sheets_cells', $cell)){
        if($user_activity_role != 'teacher'){
            $cell_visibility = $DB->get_record('
                tables_sheets_cells', $cell, '*', MUST_EXIST)->
                visibility;

            echo '
350         <script>

```

```

        localStorage.'.$cell["name"].' = "all";
</script>
';

355 switch ($cell_visibility){
    case 'all':{
        $cell['content'] = $DB->get_record('
            tables_sheets_cells', $cell, '*', MUST_EXIST)
            ->content;
        echo '
360 <script>
        localStorage.'.$cell["name"].' = "all";
</script>
        ';
        break;
    }
365 case 'group':{
        if($group_visibility == $cell_visibility){
            $cell['content'] = $DB->get_record('
                tables_sheets_cells', $cell, '*',
                MUST_EXIST)->content;
            echo '
370 <script>
            localStorage.'.$cell["name"].' = "group";
</script>
            ';
        }
        break;
375 }
    case 'user':{
        if($user_visibility == $cell_visibility){
            $cell['content'] = $DB->get_record('
                tables_sheets_cells', $cell, '*',
                MUST_EXIST)->content;
            echo '
380 <script>
            localStorage.'.$cell["name"].' = "user";
</script>
            ';
        }
385 break;
    }
}
}
else{

```



```

390 $cell_visibility = $DB->get_record('
    tables_sheets_cells', $cell, '*', MUST_EXIST)->
    visibility;
$cell['content'] = $DB->get_record('
    tables_sheets_cells', $cell, '*', MUST_EXIST)->
    content;
echo '
<script>
    localStorage.'.$cell["name"].' = "teacher";
395 </script>
';
}

echo '
400 <td>
    <textarea name="cell_textarea" '.$disablecell.'
        data-visibility = "'.$cell_visibility.'" data-
        attached="'.$DB->record_exists('
            tables_users_cells', array('sheetid' =>
                $active_sheet->id, 'userid' => $USER->id, '
                cellname' => $cell['name']))).' "
    style="
        font-family: '.get_cell_font_family($cell['name']
            ), $moduleinstance->id).'; font-size: '.
            get_cell_font_size($cell['name'],
                $moduleinstance->id).'pt;
        font-weight: '.get_cell_bold($cell['name'],
            $moduleinstance->id).';
405 font-style: '.get_cell_italic($cell['name'],
            $moduleinstance->id).';
        text-decoration: '.get_cell_underline($cell['
            name'], $moduleinstance->id).';
        text-align: '.get_cell_align($cell['name'],
            $moduleinstance->id).'; "
        onfocus="onFocusInCell(this)"
        onchange="saveCellHistory(this)"
410 oninput="updateTablesCell(this)"
        id='.$cell['name'].'>'.$cell['content'].'</
        textarea>
    </td>';
}
else{
415 $cell_visibility = 'all';
    $cell['content'] = null;
    echo '

```

```

420         <td>
            <textarea name="cell_textarea" '. $disablecell.'
            data-visibility = "'. $cell_visibility.'"
            data-attached="'. $DB->record_exists('
                tables_users_cells', array('sheetid' =>
                $active_sheet->id, 'userid' => $USER->id, '
                cellname' => $cell['name']))).' "
            style="
                font-family: '.get_cell_font_family($cell['name']
                    ], $moduleinstance->id).';
                font-size: '.get_cell_font_size($cell['name'],
                    $moduleinstance->id). 'pt;
425                font-weight: '.get_cell_bold($cell['name'],
                    $moduleinstance->id).';
                font-style: '.get_cell_italic($cell['name'],
                    $moduleinstance->id).';
                text-decoration: '.get_cell_underline($cell['
                    name'], $moduleinstance->id).';
                text-align: '.get_cell_align($cell['name'],
                    $moduleinstance->id).'; "
            onfocus="onFocusInCell(this)"
430            onchange="saveCellHistory(this)"
            oninput="updateTablesCell(this)"
            id='. $cell['name']. '>'. $cell['content']. '</
                textarea>
            </td>';
        }
435    }
    echo '
        </tr>';
    }
    echo '
440    </tbody>
</table>
<ul class="m-sheet-custom-menu" id="custom_menu_' .
    $moduleinstance->id. '">
    <li id="delete_sheet" data-action = "delete_sheet" '; if(
        $user_activity_role!="teacher"){echo'hidden="hidden"';}
        echo'>Delete</li>
    </ul>
445    <form method="post">
        <div class="m-tables-sheet-bar" id="sheet_bar">';
        $sheets = $DB->get_records('tables_sheets', array('tableid'
            =>$moduleinstance->id));
        foreach($sheets as $sheet){

```

```

450     echo '
        <button class="m-tables-sheet-select" type="submit" name
            ="sheet" value="' . $sheet->id . '" id="sheet_' . $sheet->
            id . '"'; if($active_sheet->id == $sheet->id){echo '
            disabled';} echo '>
            '.get_string("sheet", "mod_tables")." ". $sheet->name .'
        </button>';
    }
    echo '
455     </div>';
    if($user_activity_role == "teacher"){
        echo '
        <span class="m-tables-sheet-add">
            <i class="fa fa-plus" id="add_sheet_for_module_' .
                $moduleinstance->id . '" onclick="createSheet(this)"></
                i>
460     </span>';
    }
    echo '
    </form>';
    if($user_activity_role == "teacher") {
465     echo '
        <button class="btn btn-primary" style="border-radius: 5px"
            onclick="deleteSheet()">Delete sheet</button>';
    }
    echo '
</div>';
470
    echo '
    <script src="https://cdn.jsdelivr.net/npm/interactjs/dist/
        interact.min.js"></script>
    <script src="//cdn.socket.io/socket.io-1.2.0.js"></script>
    <script>
475     localStorage.KEY = "' . $moduleinstance->wskey . '"';
    if("' . $moduleinstance->wsserver . '" == ""){
        localStorage.socket = "false";
    }
    else{
480     localStorage.socket = "true";
    }
    let socket = io("' . $moduleinstance->wsserver . '"')
    let messages = ["'.get_string("alertselectstudents", "
        mod_tables")."', "'.get_string("alertselectcellss", "
        mod_tables").'"]
    </script>';

```

485

```
echo $OUTPUT->footer();
```

II1.2. amd/src/update_data.js

```

/**
 * Update cell information for page and database.
 *
5  * @param {object} object html object.
 */
function updateTablesCell(object) {
    let data = {
        update_type: "input",
10    table_id: document.getElementById('main_table').
        getAttribute('data-moduleinstance'),
        sheet_id: document.getElementById('main_table').
        getAttribute('data-sheet'),
        cell_id: object.id,
        cell_content: object.value
    };

15
    if(document.getElementById('select_cell_visibility')){
        data['cell_visibility'] = document.getElementById('
            select_cell_visibility').value;
    }
    else{
20    data['cell_visibility'] = localStorage[object.id];
    }

    document.getElementById("focused_cell_content").value =
        data['cell_content'];

25
    // Send information to other users
    if(localStorage.socket !== "false") {
        console.log("_____upd")
        console.log(data)
        console.log("_____upd")
30    console.log(localStorage[object.id])
        // Send information to other users
        socket.emit('send', {
            room: document.getElementById('main_table').
                getAttribute('data-moduleinstance'),
            message: data
        })
    }
}

```

```

35     });
    }

    // Send information to update_cell.php for updating
    // database
    $.ajax({
40         method: "POST",
        url: "update_cell.php",
        data: data
    });
}

45 /**
 * Show dropdown list of users.
 *
 * @param {object} object html object.
50 */
function onInputSearch(object) {
    let children;

    switch (object.getAttribute('data-attach-to')){
55     case 'user':{
        children = document.querySelectorAll("#dropdown-
            content-users p");

        break;
    }
60     case 'group':{
        children = document.querySelectorAll("#dropdown-
            content-groups p");

        break;
    }
65 }

    let searchstr = object.value.toLowerCase();

    for (let i = 0; i < children.length; i++) {
70         let value = children[i].querySelector("label").innerHTML
            .toLowerCase();

        if (value.includes(searchstr)) {
            children[i].style.display = 'block';
        }
75         else {

```

```

        children[i].style.display = 'none';
    }
}

80 }

/**
 * Show input for search students to attach.
 *
85 * @param {object} object html object.
 */
function onclickAttach(object){
    let main = document.getElementById('main_table');

90    let dropdown_block;
    let dropdown_button;
    let cellboxes;
    let submit_btns;

95    switch (object.getAttribute('data-attach-to')){
        case 'user':{
            dropdown_block = document.getElementById('
                dropdown_attach_students');
            dropdown_button = document.getElementById('
                attach_cell_to_users');
            cellboxes = document.getElementsByClassName('m-
                dropdown-students-cell');
100            submit_btns = 'submit_user_btns';

            if(document.getElementById('first_cell-students').
                value){
                let first_sell = document.getElementById('first_cell
                    -students');
                document.getElementById(first_sell.value).style.
                    border = "";
105                //first_sell.value = null;
            }
            if(document.getElementById('last_cell-students').value
                ){
                let last_sell = document.getElementById('last_cell-
                    students');
                document.getElementById(last_sell.value).style.
                    border = "";
110                //last_sell.value = null;
            }
        }
    }
}

```

```

        break;
    }
115 case 'group':{
        dropdown_block = document.getElementById('
            dropdown_attach_groups');
        dropdown_button = document.getElementById('
            attach_cell_to_groups');
        cellboxes = document.getElementsByClassName('m-
            dropdown-groups-cell');
        submit_btns = 'submit_group_btns';

120
        if(document.getElementById('first_cell-groups').value)
        {
            let first_sell = document.getElementById('first_cell
                -groups');
            document.getElementById(first_sell.value).style.
                border = "";
            //first_sell.value = null;
125
        }
        if(document.getElementById('last_cell-groups').value){
            let last_sell = document.getElementById('last_cell-
                groups');
            document.getElementById(last_sell.value).style.
                border = "";
            //last_sell.value = null;
130
        }

        break;
    }
}

135
if(dropdown_block.style.display === 'none'){
    object.value = 'on';
    cellboxes[0].style.display = 'inline-block';
    cellboxes[1].style.display = 'inline-block';
140
    dropdown_block.style.display = 'inline-block';
    dropdown_button.style.border = "1px solid black";
    Array.prototype.forEach.call(document.
        getElementsByClassName('m-tables-toolbar-block'),
        function (element, idx){
            element.classList.add('disabled');
        })
145
    document.getElementById('input_bar').classList.add('
        disabled');

```

```

try{
    onFocusOutCell(document.getElementById('prev_cell').
        value)

150     let data = {
        update_type: "focusout",
        table_id: main.getAttribute('data-moduleinstance'),
        sheet_id: main.getAttribute('data-sheet'),
        cell_id: null
155     };

    $.ajax({
        method: "POST",
        url: "update_cell_focus.php",
160        data: data
    });
}
catch(e){
    console.log('No focused cells')
165    console.log(e)
}
}
else{
    object.value = 'off';
170    dropdown_block.style.display = 'none';
    dropdown_button.style.border = "";
    cellboxes[0].style.display = 'none';
    cellboxes[1].style.display = 'none';
    document.getElementById(submit_btns).style.display = "
        none";
175    Array.prototype.forEach.call(document.
        getElementsByClassName('m-tables-toolbar-block'),
        function (element, idx){
            element.classList.remove('disabled');
        })
    document.getElementById('input_bar').classList.remove('
        disabled');

180 }
}

/**
* Outputs student names to the selection window
185 *

```



```

* @param {object} object html object.
*/
function onclickCheckboxAttach(object){
  let label_display;
190   let checked_students;

  switch (object.getAttribute('data-attach-to')){
    case 'user':{
      label_display = document.getElementById('
        display_selected_students');
195      checked_students = document.querySelectorAll('.m-user -
        check:checked');

      break;
    }
    case 'group':{
200      label_display = document.getElementById('
        display_selected_groups');
      checked_students = document.querySelectorAll('.m-group
        -check:checked');

      break;
    }
205   }

  if(checked_students.length > 1){
    label_display.value = checked_students[0].getAttribute('
      data-attach-name').concat(" +", checked_students.
        length-1);
  }
210 else if(checked_students.length === 0){
    label_display.value = null;
  }
  else{
    label_display.value = checked_students[0].getAttribute('
      data-attach-name');
215   }
  }

  /**
  * Submit the selected cells for the attachment.
  *
220  * @param {object} object html object.
  * @param {string[]} messages array of messages.
  */

```

```

function onclickSubmitAttach(object, messages){
225   let selected;
      let first_cell_input_id;
      let last_cell_input_id;
      let checked;
      let submit_btns;
230   let attach_cell_to;
      let update_type;

      switch (object.getAttribute('data-attach-to')){
        case 'user':{
235           selected = document.getElementById('
              display_selected_students');
           checked = document.querySelectorAll('.m-user-check:
              checked');
           first_cell_input_id = 'first_cell-students';
           last_cell_input_id = 'last_cell-students';
           submit_btns = 'submit_user_btns';
240           attach_cell_to = 'attach_cell_to_users';
           update_type = 'students';

           break;
        }
245       case 'group':{
           selected = document.getElementById('
              display_selected_groups');
           checked = document.querySelectorAll('.m-group-check:
              checked');
           first_cell_input_id = 'first_cell-groups';
           last_cell_input_id = 'last_cell-groups';
250           submit_btns = 'submit_group_btns';
           attach_cell_to = 'attach_cell_to_groups';
           update_type = 'groups';

           break;
255       }
      }

      let first_cell_input = document.getElementById(
        first_cell_input_id);
      let last_cell_input = document.getElementById(
        last_cell_input_id);
260

      if(selected.value !== ""){

```

```

let module_id = document.getElementById('main_table').
    getAttribute('data-moduleinstance');
let sheet_id = document.getElementById('main_table').
    getAttribute('data-sheet');

265 object.setAttribute('data-table', module_id);
    object.setAttribute('data-sheet', sheet_id);
    object.setAttribute('data-first-cell',
        first_cell_input_id);
    object.setAttribute('data-last-cell', last_cell_input_id
        );
    object.setAttribute('data-update-type', update_type);

270 for(let i = 0; i < checked.length; i ++){
    object.setAttribute('data-user', checked[i].value);

    attachCells(object, messages)
275 }

let first_cell = document.getElementById(
    first_cell_input.value);
first_cell.style.border = "";
let last_cell = document.getElementById(last_cell_input.
    value);
280 last_cell.style.border = "";

first_cell_input.value = null;
last_cell_input.value = null;

285 document.getElementById(submit_btns).style.display = "
    none";

    onclickAttach(document.getElementById(attach_cell_to))
}
else{
290 alert(messages[0])
}
}

/**
295 * Cancels the selected cells for the attachment.
*
* @param {object} object html object.
*
*/

```

```

300 function onclickCanselAttach(object){
    let first_cell_input;
    let last_cell_input;
    let submit_btns;

305 switch (object.getAttribute('data-attach-to')){
    case 'user':{
        first_cell_input = document.getElementById('first_cell
            -students');
        last_cell_input = document.getElementById('last_cell-
            students');
        submit_btns = 'submit_user_btns';

310 break;
    }
    case 'group':{
        first_cell_input = document.getElementById('first_cell
            -groups');
        last_cell_input = document.getElementById('last_cell-
315 groups');
        submit_btns = 'submit_group_btns';

        break;
    }
320 }

    let first_cell = document.getElementById(first_cell_input.
        value);
    first_cell.style.border = "";
    first_cell_input.value = "";

325 if(last_cell_input.value !== ""){
        let last_cell = document.getElementById(last_cell_input.
            value);
        last_cell.style.border = "";
        last_cell_input.value = ""

330 }
    document.getElementById(submit_btns).style.display = "none
        ";
}

/**
335 * On focus cell when attach button active.
*
* @param {object} object html object (textarea cell).

```

```

*/
function onFocusInCellAttach(object){
340   if(document.getElementById('attach_cell_to_users').value
      === 'on'){
      let first_cell = document.getElementById('first_cell -
        students');
      let last_cell = document.getElementById('last_cell -
        students');

      if(first_cell.value === "" && object.id === last_cell.
        value){
345         first_cell.value = object.id;
         last_cell.value = "";
         object.style.border = "1px solid #27a7d8";
      }
      else if(first_cell.value === ""){
350         first_cell.value = object.id;
         object.style.border = "1px solid #27a7d8";
      }
      else if(last_cell.value === "" && first_cell.value ===
        object.id){
355         last_cell.value = object.id;
         object.style.border = "1px solid #ff9a00";
         object.style.borderLeftColor = "#27a7d8";
         object.style.borderTopColor = "#27a7d8";
      }
      else if(last_cell.value === ""){
360         last_cell.value = object.id;
         object.style.border = "1px solid #ff9a00";
      }
      else if(first_cell.value === last_cell.value &&
        first_cell.value !== object.id){
365         document.getElementById(last_cell.value).style.border
          = "1px solid #27a7d8";
         last_cell.value = object.id;
         object.style.border = "1px solid #ff9a00";
      }
      else if(first_cell.value === object.id){
370         object.style.border = null;
         first_cell.value = "";
      }
      else{
         document.getElementById(last_cell.value).style.border
          = null;
         last_cell.value = object.id;
      }
    }
  }
}

```

```

375     object.style.border = "1px solid #ff9a00";
    }

    if(first_cell.value !== "" && last_cell.value !== ""){
        document.getElementById('submit_user_btns').style.
            display = "inline-block";
380    }
    else{
        document.getElementById('submit_user_btns').style.
            display = "none";
    }

385    document.activeElement.blur();
}
else if(document.getElementById('attach_cell_to_groups').
    value === 'on'){
    let first_cell = document.getElementById('first_cell-
        groups');
    let last_cell = document.getElementById('last_cell-
        groups');
390

    if(first_cell.value === "" && object.id === last_cell.
        value){
        first_cell.value = object.id;
        last_cell.value = "";
        object.style.border = "1px solid #27a7d8";
395    }
    else if(first_cell.value === ""){
        first_cell.value = object.id;
        object.style.border = "1px solid #27a7d8";
    }
400    else if(last_cell.value === "" && first_cell.value ===
        object.id){
        last_cell.value = object.id;
        object.style.border = "1px solid #ff9a00";
        object.style.borderLeftColor = "#27a7d8";
        object.style.borderTopColor = "#27a7d8";
405    }
    else if(last_cell.value === ""){
        last_cell.value = object.id;
        object.style.border = "1px solid #ff9a00";
    }
410    else if(first_cell.value === last_cell.value &&
        first_cell.value !== object.id){

```

```

        document.getElementById(last_cell.value).style.border
            = "1px solid #27a7d8";
        last_cell.value = object.id;
        object.style.border = "1px solid #ff9a00";
    }
415 else if(first_cell.value === object.id){
        object.style.border = null;
        first_cell.value = "";
    }
    else{
420         document.getElementById(last_cell.value).style.border
            = null;
        last_cell.value = object.id;
        object.style.border = "1px solid #ff9a00";
    }

425 if(first_cell.value !== "" && last_cell.value !== ""){
        document.getElementById('submit_group_btns').style.
            display = "inline-block";
    }
    else{
        document.getElementById('submit_group_btns').style.
            display = "none";
430    }

    document.activeElement.blur();
}
else{
435    onFocusInCellFocus(object)

    document.getElementById('grade_block').classList.remove(
        'disabled');
    document.getElementById('visibility_block').classList.
        remove('disabled');
    document.getElementById('check_grade').classList.remove(
        'm-tables-show');
440 }
}

/**
 * On focus cell when attach button deactive.
445 *
 * @param {object} object html object (textarea cell).
 */
function onFocusInCellFocus(object){

```

```

let data = {
450   update_type: "focusin",
      table_id: document.getElementById('main_table').
        getAttribute('data-moduleinstance'),
      sheet_id: document.getElementById('main_table').
        getAttribute('data-sheet'),
      cell_id: object.id,
      cell_content: object.value
455 };

let prev_cell = document.getElementById("prev_cell").value
    ;

if (prev_cell !== object.id && prev_cell !== "") {
460   onFocusOutCell(prev_cell);
}

document.getElementById(object.id).style.border = "1px
    solid black";
document.getElementById("focused_cell").value = object.id;
465 document.getElementById("prev_cell").value = object.id;
document.getElementById("focused_cell_content").value =
    object.value;
document.getElementById("font-family-selector").value =
    object.style.fontFamily;
document.getElementById("font-size-selector").value =
    object.style.fontSize.replace("pt", "");

470 if(document.getElementById('select_cell_visibility')){
    document.getElementById("select_cell_visibility").value
        = object.getAttribute('data-visibility');
}
else{
    object.setAttribute('data-visibility', localStorage[
        object.id]);
475 }

if (object.style.fontWeight === "bold") {
    document.getElementById("font-bold-button").style.border
        = "1px solid black";
} else {
480   document.getElementById("font-bold-button").style.border
        = "";
}

if (object.style.fontStyle === "italic") {

```



```

        document.getElementById("font-italic-button").style.
            border = "1px solid black";
    } else {
485     document.getElementById("font-italic-button").style.
        border = "";
    }
    if (object.style.textDecoration === "underline") {
        document.getElementById("font-underline-button").style.
            border = "1px solid black";
    } else {
490     document.getElementById("font-underline-button").style.
        border = "";
    }
    switch (object.style.textAlign) {
        case "left":
            document.getElementById("text-left-button").style.border
                = "1px solid black";
495     document.getElementById("text-center-button").style.
        border = "";
            document.getElementById("text-right-button").style.
                border = "";
            break;
        case "center":
            document.getElementById("text-left-button").style.border
                = "";
500     document.getElementById("text-center-button").style.
        border = "1px solid black";
            document.getElementById("text-right-button").style.
                border = "";
            break;
        case "right":
            document.getElementById("text-left-button").style.border
                = "";
505     document.getElementById("text-center-button").style.
        border = "";
            document.getElementById("text-right-button").style.
                border = "1px solid black";
            break;
    }

510 // Send information to other users
    if(localStorage.socket !== "false") {
        socket.emit('send', {
            room: document.getElementById('main_table').
                getAttribute('data-moduleinstance'),

```

```

515         message: data
    });
}

    // Send information to update_cell_focus.php for updating
    database

520    $.ajax({
        method: "POST",
        url: "update_cell_focus.php",
        data: data
    });
}

525
/**
 * Update cell focus information for page and database.
 *
 * @param {object} object html object (textarea cell).
530 */
function onFocusInCell(object) {
    if(document.getElementById('attach_cell_to_users')){
        onFocusInCellAttach(object)
    }
535    else{
        onFocusInCellFocus(object)
    }
}

540
/**
 * Send on WS unfocused cell id
 *
 * @param {string} cell_id cell name.
545 */
function onFocusOutCell(cell_id) {
    let data = {
        update_type: "focusout",
        cell_id: cell_id
    };
550

    document.getElementById(cell_id).style.border = "";
    document.getElementById("text-left-button").style.border =
        "";
    document.getElementById("text-center-button").style.border
        = "";
    document.getElementById("text-right-button").style.border
        = "";

```

```

555 document.getElementById("font-bold-button").style.border =
      "";
document.getElementById("font-italic-button").style.border
      = "";
document.getElementById("font-underline-button").style.
      border = "";
document.getElementById("focused_cell").value = "";
document.getElementById("focused_cell_content").value = ""
      ;
560 if(document.getElementById('main_table').getAttribute('
      data-user-role') === 'teacher'){
      document.getElementById('grade_block').classList.add('
          disabled');
      document.getElementById('visibility_block').classList.
          add('disabled');
      }

565 // Send information to other users
if(localStorage.socket !== "false") {
      socket.emit('send', {
          room: document.getElementById('main_table').
              getAttribute('data-moduleinstance'),
          message: data
570      });
      }
}

/**
575 * Update cell focus information for page and database.
*
* @param {Object} object html object.
*/
function onChangeInputCell(object) {
580     try {
        if (object.value === "") {
            let prev_cell = document.getElementById("prev_cell").
                value;

            document.getElementById(prev_cell).style.border = "";
585 document.getElementById("focused_cell").value = "";
            document.getElementById("prev_cell").value = "";
            document.getElementById("focused_cell_content").value
                = "";

            onFocusOutCell(prev_cell);

```

```

590     } else {
        let cell = document.getElementById(object.value);

        onFocusInCellFocus(cell);
    }
595 } catch (e) {
    alert("Incorrect cell name");
}

}

600
/**
 * Update cell information for page and database.
 *
 * @param {Object} object html object.
605 */
function onChangeInputContent(object) {
    let cell_id = document.getElementById("focused_cell").
        value;
    let cell = document.getElementById(cell_id);
    cell.value = object.value;

610    updateTablesCell(cell);
}

/**
615 * Update cells height for page and database.
 *
 * @param {object} object html object.
 */
function updateHeight(object) {
620    let row_id = "row_".concat(object.value);

    let data = {
        update_type: "resize_h",
        table_id: document.getElementById('main_table').
            getAttribute('data-moduleinstance'),
625        sheet_id: document.getElementById('main_table').
            getAttribute('data-sheet'),
        name: row_id,
        height: document.getElementById(row_id).offsetHeight
    };

630    // Send information to other users
    if(localStorage.socket !== "false") {

```

```

        socket.emit('send', {
            room: document.getElementById('main_table').
                getAttribute('data-moduleinstance'),
            message: data
635     });
    }

    // Send information to update_cell_size.php for updating
        database
    $.ajax({
640         method: "POST",
        url: "update_cell_size.php",
        data: data
    });
}

645 /**
 * Update cells width for page and database.
 *
 * @param {object} object html object.
 */
function updateWidth(object) {
    let col_id = "col_".concat(object.value);

    let data = {
655         update_type: "resize_w",
        table_id: document.getElementById('main_table').
            getAttribute('data-moduleinstance'),
        sheet_id: document.getElementById('main_table').
            getAttribute('data-sheet'),
        name: col_id,
        width: document.getElementById(col_id).offsetWidth
660     };

    // Send information to other users
    if(localStorage.socket !== "false") {
        socket.emit('send', {
665             room: document.getElementById('main_table').
                getAttribute('data-moduleinstance'),
            message: data
        });
    }

    //Send information to update_cell_size.php for updating
        database

```

```

$.ajax({
    method: "POST",
    url: "update_cell_size.php",
    data: data
675 });
}

/**
 * Update cell text parameters information for page and
    database.
680 *
 * @param {object} object html object.
 */
function updateFont(object) {
    let data = {
685     update_type: "font",
        button_type: object.id,
        table_id: document.getElementById('main_table').
            getAttribute('data-moduleinstance'),
        sheet_id: document.getElementById('main_table').
            getAttribute('data-sheet'),
        cell_id: document.getElementById("focused_cell").value,
690     input_content: object.value
    };

    if (data['cell_id']) {
        switch (object.id) {
695         case "font-family-selector":
            document.getElementById(data['cell_id']).style.
                fontFamily = object.value;
            break;
            case "font-size-selector":
                document.getElementById(data['cell_id']).style.
                    fontSize = object.value.concat("pt");
700             break;
            case "font-bold-button":
                if (document.getElementById(data['cell_id']).style.
                    fontWeight !== "bold") {
                    document.getElementById(object.id).value = "bold";
                    document.getElementById(object.id).style.border = "1
                        px solid black";
705                     document.getElementById(data['cell_id']).style.
                        fontWeight = "bold";
                        data['input_content'] = "bold";
                    } else {

```

```

document.getElementById(object.id).value = "normal";
document.getElementById(object.id).style.border = ""
;
710 document.getElementById(data['cell_id']).style.
      fontWeight = "normal";
data['input_content'] = "normal";
}
break;
case "font-italic-button":
715 if (document.getElementById(data['cell_id']).style.
      fontStyle !== "italic") {
document.getElementById(object.id).value = "italic";
document.getElementById(object.id).style.border = "1
      px solid black";
document.getElementById(data['cell_id']).style.
      fontStyle = "italic";
data['input_content'] = "italic";
720 } else {
document.getElementById(object.id).value = "normal";
document.getElementById(object.id).style.border = ""
;
document.getElementById(data['cell_id']).style.
      fontStyle = "normal";
data['input_content'] = "normal";
725 }
break;
case "font-underline-button":
if (document.getElementById(data['cell_id']).style.
      textDecoration !== "underline") {
document.getElementById(object.id).value = "
      underline";
730 document.getElementById(object.id).style.border = "1
      px solid black";
document.getElementById(data['cell_id']).style.
      textDecoration = "underline";
data['input_content'] = "underline";
} else {
document.getElementById(object.id).value = "none";
735 document.getElementById(object.id).style.border = ""
;
document.getElementById(data['cell_id']).style.
      textDecoration = "none";
data['input_content'] = "none";
}
break;

```

```

740 case "text-left-button":
document.getElementById(object.id).style.border = "1px
solid black";
document.getElementById("text-center-button").style.
border = "";
document.getElementById("text-right-button").style.
border = "";
document.getElementById(data['cell_id']).style.
textAlign = "left";
745 data['input_content'] = "left";
break;
case "text-center-button":
document.getElementById("text-left-button").style.
border = "";
document.getElementById(object.id).style.border = "1px
solid black";
750 document.getElementById("text-right-button").style.
border = "";
document.getElementById(data['cell_id']).style.
textAlign = "center";
data['input_content'] = "center";
break;
case "text-right-button":
755 document.getElementById("text-left-button").style.
border = "";
document.getElementById("text-center-button").style.
border = "";
document.getElementById(object.id).style.border = "1px
solid black";
document.getElementById(data['cell_id']).style.
textAlign = "right";
760 data['input_content'] = "right";
break;
}

// Send information to other users
if(localStorage.socket !== "false") {
765 socket.emit('send', {
room: document.getElementById('main_table').
getAttribute('data-moduleinstance'),
message: data
});
}
770

```



```

        // Send information to update_font.php for updating
        database
        $.ajax({
            method: "POST",
            url: "update_font.php",
            data: data
        });
    }
}

/**
 * Save cell change information.
 *
 * @param {object} object html object.
 */
function saveCellHistory(object) {
    let data = {
        update_type: "history",
        table_id: document.getElementById('main_table').
            getAttribute('data-moduleinstance'),
        sheet_id: document.getElementById('main_table').
            getAttribute('data-sheet'),
        cell_id: object.id,
        cell_content: object.value
    };

    // Send information to save_history.php for updating
    database
    $.ajax({
        method: "POST",
        url: "save_history.php",
        data: data
    });
}

/**
 * Create new sheet for the table.
 *
 * @param {object} object html object.
 */
function createSheet(object){
    let data = {
        update_type: "add_sheet",
        table_id: object.id.replace('add_sheet_for_module_', '')
    };

```

```

let new_sheet = document.createElement("BUTTON");
new_sheet.setAttribute("class", "m-tables-sheet-select");
815 new_sheet.innerHTML = "<img src=\"pix/refreshdouble.png\"
      alt=\"refresh\">";
document.querySelector("#sheet_bar").appendChild(new_sheet
    );

    //Send information to create_sheet.php for updating
      database
$.ajax({
820   method: "POST",
      url: "create_sheet.php",
      data: data
    });

825   setTimeout(function(){ location.reload(); }, 500);
}

/**
 * Delete sheet from the table.
 *
 */
830
function deleteSheet(){
    let data = {
        update_type: "delete_sheet",
835        table_id: document.getElementById('main_table').
            getAttribute('data-moduleinstance'),
        sheet_id: document.getElementById('main_table').
            getAttribute('data-sheet')
    };

    //Send information to create_sheet.php for updating
      database
840 $.ajax({
        method: "POST",
        url: "create_sheet.php",
        data: data
    });
845 }

/**
 * Grade attached cells
 *
 */
850

```

```

function gradeCell(){
    let main = document.getElementById('main_table');

    let data = {
855         sheet_id: main.getAttribute('data-sheet'),
            table_id: main.getAttribute('data-moduleinstance'),
            cell_name: document.getElementById('focused_cell').value
            ,
            user_id: document.getElementById('select_user_grade').
                value,
            grade: document.getElementById('input_grade').value
860     };

    if(document.getElementById("feedback_block").style.display
        !== "none"){
        data["feedback"] = document.getElementById("
            feedback_textarea").value;
    }
865

    $.ajax({
        method: "POST",
        url: "grade_cell.php",
        data: data
870    });

    document.getElementById('check_grade').classList.add('m-
        tables-show');
    }

875    /**
     * Hide element
     *
     */

    function onchangeInputGrade(){
880        document.getElementById('check_grade').classList.remove('m
            -tables-show');
    }

    /**
     * Set max and min range
885     *
     */

    function oninputGrade(object){
        if(object.value > 100){
            object.value = 100;

```

```

890     }
    else if(object.value < 0){
        object.value = 0;
    }
}

895
/**
 * Show feedback textarea
 *
 * @param {object} object html object.
900 */
function showFeedback(object){
    let feedback_block = document.getElementById("
        feedback_block");

    if(feedback_block.style.display === "none"){
905        feedback_block.style.display = "inline-block";
        object.classList.remove("fa-plus");
        object.classList.add("fa-minus");
    }
    else{
910        feedback_block.style.display = "none";
        object.classList.remove("fa-minus");
        object.classList.add("fa-plus");
    }
}

915
/**
 * Change cell visibility.
 *
 */
920 function onChangeSelectVisibility(){
    let data = {
        update_type: "visibility",
        table_id: document.getElementById('main_table').
            getAttribute('data-moduleinstance'),
        sheet_id: document.getElementById('main_table').
            getAttribute('data-sheet'),
925        cell_id: document.getElementById('focused_cell').value,
        cell_visibility: document.getElementById('
            select_cell_visibility').value
    };

    document.getElementById(data['cell_id']).setAttribute('
        data-visibility', data['cell_visibility'])

```

```

930      // Send information to other users
      if(localStorage.socket !== "false") {
        socket.emit('send', {
          room: document.getElementById('main_table').
            getAttribute('data-moduleinstance'),
935          message: data
        });
      }

      //Send information to create_sheet.php for updating
      database

940      $.ajax({
        method: "POST",
        url: "update_visibility.php",
        data: data
      });
945    }

    /**
    * Update attached cells database.
    *
950    * @param {object} object html object.
    * @param {string[]} messages array of messages.
    */
    function attachCells(object, messages){

955      let data = {
        update_type: object.getAttribute('data-update-type'),
        user_id: object.getAttribute('data-user'),
        table_id: object.getAttribute('data-table'),
        sheet_id: object.getAttribute('data-sheet')
960      };

      let first_cell = document.getElementById(object.
        getAttribute("data-first-cell"));
      let last_cell = document.getElementById(object.
        getAttribute("data-last-cell"));
      let regex = new RegExp("^(?:[A-Z]|[A-Z][A-Z]|[A-X][A-F][A-
        D])
        (?:[1-9]|[1-9][0-9]|[1-9][0-9][0-9]|[1-9][0-9][0-9][0-9]|[1-
        9][0-9][0-9][0-9][0-9])");
965      if(regex.test(first_cell.value) && regex.test(last_cell.
        value)){

```

```

data["first_cell"] = first_cell.value;
data["last_cell"] = last_cell.value;

970 // Send information to update_cell_focus.php for
    updating database
$.ajax({
    method: "POST",
    url: "attach_cells.php",
    data: data
975 });

data['update_type'] = "attach_cells";
if(localStorage.socket !== "false") {
    socket.emit('send', {room: data['table_id'], message:
        data});
980 }
}
else{
    alert(messages[1]);
}
985 }

```

П1.3. update_cell_focus.php

```

$user_data = array(
    'tableid' => optional_param('table_id', 0, PARAM_INT),
    'userid' => $USER->id);
5

$cell_focusin = optional_param('cell_id', null, PARAM_TEXT);

// Updating cell

10 if($DB->record_exists('tables_users_focus', $user_data)){
    $user_cell = $DB->get_record('tables_users_focus',
        $user_data, '*', MUST_EXIST);
    $user_cell->focused_cell = $cell_focusin;
    $user_cell->timemodified = time();
    $DB->update_record('tables_users_focus', $user_cell);
15 }
else{
    $user_data['focused_cell'] = $cell_focusin;
    $user_data['timecreated'] = time();
    $DB->insert_record('tables_users_focus', $user_data);
}

```

20 | }

П1.4. attach_cells.php

```

$update_type = optional_param('update_type', null,
    PARAM_TEXT);

switch($update_type){
5   case 'students':{
        $data = array(
            'sheetid' => optional_param('sheet_id', 0, PARAM_INT),
            'userid' => optional_param('user_id', 0, PARAM_INT));
        $table = 'tables_users_cells';
10        break;
    }
    case 'groups':{
        $data = array(
            'sheetid' => optional_param('sheet_id', 0, PARAM_INT),
            'groupid' => optional_param('user_id', 0, PARAM_INT));
15        $table = 'tables_groups_cells';
        break;
    }
}

20 // Updating data

attach_cells($data, $table, optional_param('first_cell', 0,
    PARAM_TEXT), optional_param('last_cell', 0, PARAM_TEXT));

25 function attach_cells($data, $table, $first_cell, $last_cell
    ){
    $first_column = preg_replace('/[~a-zA-Z]/', '',
        $first_cell);
    $first_row = preg_replace('/[~0-9]/', '', $first_cell);

    $last_column = preg_replace('/[~a-zA-Z]/', '', $last_cell)
        ;
30    $last_row = preg_replace('/[~0-9]/', '', $last_cell);

    $columns = get_cell_range($first_column, $last_column);
    $rows = get_cell_range($first_row, $last_row);

35    $cells = array();

```

```

foreach ($columns as $column){
    foreach ($rows as $row){
        array_push($cells, $column.$row);
    }
}

foreach($cells as $cell){
    $data['cellname'] = $cell;
    if(!$DB->record_exists($table, $data)){
        $data['timecreated'] = time();
        $DB->insert_record($table, $data);
    }
    if(!$DB->record_exists('tables_sheets_cells', array(
        'sheetid' => $data['sheetid'], 'name' => $cell))){
        $DB->insert_record('tables_sheets_cells', array(
            'sheetid' => $data['sheetid'], 'name' => $cell));
    }
}
}

```

П1.5. update_cell_size.php

```

$data = array (
    'sheetid' => optional_param('sheet_id', 0, PARAM_INT),
    'name' => optional_param('name', null, PARAM_TEXT));

switch(optional_param('update_type', null, PARAM_TEXT)){
    case 'resize_h':
        if ($DB->record_exists('tables_sheets_rows', $data)){
            $row = $DB->get_record('tables_sheets_rows', $data, '*',
                MUST_EXIST);
            $row->height = optional_param('height', 0, PARAM_INT);
            $row->timemodified = time();
            $DB->update_record('tables_sheets_rows', $row);
        }
        else {
            $data['height'] = optional_param('height', 0, PARAM_INT);
            ;
            $data['timecreated'] = time();
            $DB->insert_record('tables_sheets_rows', $data);
        }
    }
}

```



```

20 break;
    case 'resize_w': // Updating column
    if ($DB->record_exists('tables_sheets_columns', $data)){
        $column = $DB->get_record('tables_sheets_columns', $data
            , '*', MUST_EXIST);
        $column->width = optional_param('width', 0, PARAM_INT);
25 $column->timemodified = time();
        $DB->update_record('tables_sheets_columns', $column);
    }
    else {
        $data['width'] = optional_param('width', 0, PARAM_INT);
30 $data['timecreated'] = time();
        $DB->insert_record('tables_sheets_columns', $data);
    }

    break;
35 }

```

III.6. update_cell.php

```

$cell_data = array (
    'sheetid' => optional_param('sheet_id', 0, PARAM_INT),
    'name' => optional_param('cell_id', null, PARAM_TEXT));
5

$content = optional_param('cell_content', null, PARAM_TEXT);
$visibility = optional_param('cell_visibility', 'all',
    PARAM_TEXT);

// Updating cell
10

update_cell($cell_data, $content, $visibility);

function update_cell($cell_data, $content, $visibility){
    global $DB;
15

    if(($content == null || $content == "") && ($visibility ==
        'all')){
        // Delete cell

        $DB->delete_records('tables_sheets_cells', $cell_data);
20 }
    else{
        //Update cell

```

```

    if ($DB->record_exists('tables_sheets_cells', $cell_data
    )) {
25      $cell = $DB->get_record('tables_sheets_cells',
        $cell_data, '*', MUST_EXIST);
        $cell->content = $content;
        $cell->timemodified = time();
        $DB->update_record('tables_sheets_cells', $cell);
    }
30  else {
        $cell_data['content'] = $content;
        $cell_data['timecreated'] = time();
        $DB->insert_record('tables_sheets_cells', $cell_data);
    }
35  }
}

```

П1.7. update_font.php

```

$cell_data = array (
'sheetid' => optional_param('sheet_id', null, PARAM_INT),
'name' => optional_param('cell_id', null, PARAM_TEXT));
5
switch (optional_param('button_type', 0, PARAM_TEXT)){
    case "font-family-selector":
        if($DB->record_exists('tables_sheets_cells', $cell_data)){
            $cell = $DB->get_record('tables_sheets_cells',
            $cell_data, '*', MUST_EXIST);
10          $cell->font_family = optional_param('input_content', '
            Calibri', PARAM_TEXT);
            $cell->timemodified = time();
            $DB->update_record('tables_sheets_cells', $cell);
        }
        else{
15          $cell_data['font_family'] = optional_param('
            input_content', 'Calibri', PARAM_TEXT);
            $cell_data['timecreated'] = time();
            $DB->insert_record('tables_sheets_cells', $cell_data);
        }
20
        break;

        case "font-size-selector":

```

```

if($DB->record_exists('tables_sheets_cells', $cell_data)){
    $cell = $DB->get_record('tables_sheets_cells',
        $cell_data, '*', MUST_EXIST);
25    $cell->font_size = optional_param('input_content', 11,
        PARAM_INT);
    $cell->timemodified = time();
    $DB->update_record('tables_sheets_cells', $cell);
}
else{
30    $cell_data['font_size'] = optional_param('input_content',
        11, PARAM_INT);
    $cell_data['timecreated'] = time();
    $DB->insert_record('tables_sheets_cells', $cell_data);
}

35 break;

case "font-bold-button":
if($DB->record_exists('tables_sheets_cells', $cell_data)){
    $cell = $DB->get_record('tables_sheets_cells',
        $cell_data, '*', MUST_EXIST);
40    $cell->bold = optional_param('input_content', 'normal',
        PARAM_TEXT);
    $cell->timemodified = time();
    $DB->update_record('tables_sheets_cells', $cell);
}
else{
45    $cell_data['bold'] = optional_param('input_content', '
        normal', PARAM_TEXT);
    $cell_data['timecreated'] = time();
    $DB->insert_record('tables_sheets_cells', $cell_data);
}

50 break;

case "font-italic-button":
if($DB->record_exists('tables_sheets_cells', $cell_data)){
    $cell = $DB->get_record('tables_sheets_cells',
        $cell_data, '*', MUST_EXIST);
55    $cell->italic = optional_param('input_content', 'normal',
        PARAM_TEXT);
    $cell->timemodified = time();
    $DB->update_record('tables_sheets_cells', $cell);
}
else{

```

```

60     $cell_data['italic'] = optional_param('input_content', '
        normal', PARAM_TEXT);
    $cell_data['timecreated'] = time();
    $DB->insert_record('tables_sheets_cells', $cell_data);
}

65 break;

case "font-underline-button":
if($DB->record_exists('tables_sheets_cells', $cell_data)){
    $cell = $DB->get_record('tables_sheets_cells',
        $cell_data, '*', MUST_EXIST);
70     $cell->underline = optional_param('input_content', 'none
        ', PARAM_TEXT);
    $cell->timemodified = time();
    $DB->update_record('tables_sheets_cells', $cell);
}
else{
75     $cell_data['underline'] = optional_param('input_content'
        , 'none', PARAM_TEXT);
    $cell_data['timecreated'] = time();
    $DB->insert_record('tables_sheets_cells', $cell_data);
}

80 break;
case "text-left-button":
case "text-center-button":
case "text-right-button":
if($DB->record_exists('tables_sheets_cells', $cell_data)){
85     $cell = $DB->get_record('tables_sheets_cells',
        $cell_data, '*', MUST_EXIST);
    $cell->text_align = optional_param('input_content', '
        center', PARAM_TEXT);
    $cell->timemodified = time();
    $DB->update_record('tables_sheets_cells', $cell);
}
90 else{
    $cell_data['text_align'] = optional_param('input_content
        ', 'center', PARAM_TEXT);
    $cell_data['timecreated'] = time();
    $DB->insert_record('tables_sheets_cells', $cell_data);
}
95 break;
}

```

П1.8. save_history.php

```

    $data = array(
        'sheetid' => optional_param('sheet_id', 0, PARAM_INT),
        'userid' => $USER->id,
5      'cellname' => optional_param('cell_id', 0, PARAM_TEXT),
        'content' => optional_param('cell_content', 0, PARAM_TEXT));

    // Updating data

10    $data['timecreated'] = time();
    $DB->insert_record('tables_cells_history', $data);

```

П1.9. create_sheet.php

```

create_sheet($data, $update_type);

function create_sheet($data, $update_type){
5    global $DB;

    switch ($update_type){
        case "add_sheet":{
            $sheets = $DB->get_records('tables_sheets', $data);
            $data['name'] = "" . (count($sheets) + 1);
10          $data['timecreated'] = time();
            $DB->insert_record('tables_sheets', $data);
            break;
        }
        case "delete_sheet":{
15          $data['id'] = optional_param('sheet_id', 0, PARAM_INT);
            $DB->delete_records('tables_sheets_cells', array('
                sheetid' => $data['id']));
            $DB->delete_records('tables_sheets', $data);
            break;
20        }
    }
}

```

П1.10. amd/src/interact_resize.js

```

interact(".resizable-column")

```

```

.resizable({
  edges: {top: false, left: false, bottom: false, right:
5    true},
  listeners: {
    move: function(event) {
      let {x, y} = event.target.dataset;

      x = (parseFloat(x) || 0) + event.deltaRect.left;
10    y = (parseFloat(y) || 0) + event.deltaRect.top;

      Object.assign(event.target.style, {
        width: `${event.rect.width}px`,
        height: `${event.rect.height}px`,
15    transform: `translate(${x}px, ${y}px)`
      });

      Object.assign(event.target.dataset, {x, y});
      updateWidth(event.target);
20    }
  }
});

interact(".resizable-row")
25 .resizable({
  edges: {top: false, left: false, bottom: true, right:
    false},
  listeners: {
    move: function(event) {
      let {x, y} = event.target.dataset;

      x = (parseFloat(x) || 0) + event.deltaRect.left;
30    y = (parseFloat(y) || 0) + event.deltaRect.top;

      Object.assign(event.target.style, {
        width: `${event.rect.width}px`,
        height: `${event.rect.height}px`,
35    transform: `translate(${x}px, ${y}px)`
      });

      Object.assign(event.target.dataset, {x, y});
      updateHeight(event.target);
40    }
  }
});

```

П1.11. tests/unit_tests.php

```

defined('MOODLE_INTERNAL') || die();

global $CFG;
5 require_once($CFG->dirroot.'/mod/tables/lib.php');

class mod_tables_generator extends testing_module_generator
{

    public function get_column_width_test()
10 {
        global $DB;

        $data = array(
            'name' => "A",
            'sheetid' => 0,
15 'width' => 300);

        $DB->insert_record('tables_sheets_columns', $data);

        $test = get_column_width($data['name'], $data['sheetid']
20 );

        $this->assertEquals($data['width'], $test);
    }

    public function get_row_height_test()
25 {
        global $DB;

        $data = array(
            'name' => "1",
            'sheetid' => 0,
30 'height' => 300);

        $DB->insert_record('tables_sheets_rows', $data);

        $test = get_column_width($data['name'], $data['sheetid']
35 );

        $this->assertEquals($data['width'], $test);
    }

    public function get_cell_font_family_test(){
40

```

```

global $DB;

$data = array(
    'name' => "A1",
    'sheetid' => 0,
    'font_family' => 300);

$DB->insert_record('tables_sheets_cells', $data);

$test = get_cell_font_family($data['name'], $data['
    sheetid']);

$this->assertEquals($data['font_family'], $test);
}

public function get_cell_font_size_test(){
    global $DB;

    $data = array(
        'name' => "A1",
        'sheetid' => 0,
        'font_size' => 300);

    $DB->insert_record('tables_sheets_cells', $data);

    $test = get_cell_font_size($data['name'], $data['sheetid
        ']);

    $this->assertEquals($data['font_size'], $test);
}

public function get_cell_bold_test(){
    global $DB;

    $data = array(
        'name' => "A1",
        'sheetid' => 0,
        'bold' => 300);

    $DB->insert_record('tables_sheets_cells', $data);

    $test = get_cell_bold($data['name'], $data['sheetid']);

    $this->assertEquals($data['bold'], $test);
}

```



```

85 public function get_cell_italic_test(){
    global $DB;

    $data = array(
        'name' => "A1",
90     'sheetid' => 0,
        'italic' => 300);

    $DB->insert_record('tables_sheets_cells', $data);

95     $test = get_cell_italic($data['name'], $data['sheetid'])
        ;

    $this->assertEquals($data['italic'], $test);
}

100 public function get_cell_underline_test(){
    global $DB;

    $data = array(
        'name' => "A1",
105     'sheetid' => 0,
        'underline' => 300);

    $DB->insert_record('tables_sheets_cells', $data);

110     $test = get_cell_underline($data['name'], $data['sheetid']
        ');

    $this->assertEquals($data['underline'], $test);
}

115 public function get_cell_align_test(){
    global $DB;

    $data = array(
        'name' => "A1",
120     'sheetid' => 0,
        'text_align' => 300);

    $DB->insert_record('tables_sheets_cells', $data);

125     $test = get_cell_align($data['name'], $data['sheetid']);

```

```

    $this->assertEquals($data['text_align'], $test);
}

130 public function get_cell_range_test() {
    $data_first_cell = 'A';
    $data_last_cell = 'B';
    $test = get_cell_range($data_first_cell, $data_last_cell
    );
    $this->assertEquals($data_first_cell, $test[0]);
135 $this->assertEquals($data_first_cell, $test[1]);

    $data_first_cell = 'B';
    $data_last_cell = 'A';
    $test = get_cell_range($data_first_cell, $data_last_cell
    );
140 $this->assertEquals($data_first_cell, $test[1]);
    $this->assertEquals($data_first_cell, $test[0]);

    $data_first_cell = '1';
    $data_last_cell = '2';
145 $test = get_cell_range($data_first_cell, $data_last_cell
    );
    $this->assertEquals($data_first_cell, $test[0]);
    $this->assertEquals($data_first_cell, $test[1]);

    $data_first_cell = '2';
150 $data_last_cell = '1';
    $test = get_cell_range($data_first_cell, $data_last_cell
    );
    $this->assertEquals($data_first_cell, $test[1]);
    $this->assertEquals($data_first_cell, $test[0]);
}

155 public function generate_column_name_test(){
    $data = 1;
    $test = generate_column_name($data);
    $this->assertEquals("A", $test);

160 $data = 2;
    $test = generate_column_name($data);
    $this->assertEquals("B", $test);

165 $data = 27;
    $test = generate_column_name($data);
    $this->assertEquals("AA", $test);

```

```

170     $data = 28;
    $test = generate_column_name($data);
    $this->assertEquals("AB", $test);
}

175 public function attach_cells_test(){
    global $DB;

    $data = array(
        'sheetid' => 0,
        'groupid' => 0);
180 $table = 'tables_groups_cells';
    $first_cell = "A1";
    $last_cell = "B2";

    attach_cells($data, $table, $first_cell, $last_cell);

185 $test = $DB->record_exists($table, array('sheetid' =>
        $data['sheetid'],
        'groupid' => $data['groupid'],
        'cellname' => "A1"));

190 $this->assertEquals($test, true);

    $test = $DB->record_exists($table, array('sheetid' =>
        $data['sheetid'],
        'groupid' => $data['groupid'],
        'cellname' => "A2"));

195 $this->assertEquals($test, true);

    $test = $DB->record_exists($table, array('sheetid' =>
        $data['sheetid'],
        'groupid' => $data['groupid'],
200 'cellname' => "B1"));

    $this->assertEquals($test, true);

    $test = $DB->record_exists($table, array('sheetid' =>
        $data['sheetid'],
205 'groupid' => $data['groupid'],
        'cellname' => "B2"));

    $this->assertEquals($test, true);

```

```

210     $data = array(
        'sheetid' => 0,
        'userid' => 0);
    $table = 'tables_users_cells';
    $first_cell = "C1";
215    $last_cell = "D2";

    attach_cells($data, $table, $first_cell, $last_cell);

    $test = $DB->record_exists($table, array('sheetid' =>
        $data['sheetid'],
220    'userid' => $data['userid'],
        'cellname' => "C1"));

    $this->assertEquals($test, true);

225    $test = $DB->record_exists($table, array('sheetid' =>
        $data['sheetid'],
        'userid' => $data['userid'],
        'cellname' => "C2"));

    $this->assertEquals($test, true);
230

    $test = $DB->record_exists($table, array('sheetid' =>
        $data['sheetid'],
        'userid' => $data['userid'],
        'cellname' => "D1"));

235    $this->assertEquals($test, true);

    $test = $DB->record_exists($table, array('sheetid' =>
        $data['sheetid'],
        'userid' => $data['userid'],
        'cellname' => "D2"));
240

    $this->assertEquals($test, true);
}

public function grade_cell_test(){
245    global $DB;

    $data = array('userid' => 0,
        'cellid' => 0);

```

```

250     $grade = 100;
        $feedback = "Test feedback";

        grade_cell($data, $grade, $feedback);

255     $test = $DB->get_record("tables_cells_grade", $data, '*',
        , MUST_EXIST);

        $this->assertEquals($test->grade, $grade);
        $this->assertEquals($test->feedback, $feedback);
    }

260 public function update_cell_test(){
    global $DB;

    $cell_data = array (
265     'sheetid' => 0,
    'name' => "A1");

    $content = "Test content";
    $visibility = "user";

270     update_cell($cell_data, $content, $visibility);

    $test = $DB->get_record('tables_sheets_cells',
        $cell_data, '*', MUST_EXIST);

275     $this->assertEquals($test->content, $content);
    $this->assertEquals($test->visibility, $visibility);
}
}

```

III.12. tests/tables_jest.tests.js

```

const messages = ["Message 1", "Message 2"]
localStorage.socket = "false";

5 document.body.innerHTML =
    '<div class="m-tables-toolbar"> ' +
    '<div class="m-tables-toolbar-block">' +
    '<form class="m-tables-toolbar-load-up" method="post" action
        ="upload_from_xlsx.php?id=1">'+
    '<button class="m-tables-toolbar-button" type="submit">'+

```

```

10  ' ' +
    '</button>' +
    '</form>' +
    '<form class="m-tables-toolbar-load-down" method="post"
        action="upload_from_activity.php?id=1">' +
    '<button class="m-tables-toolbar-button" type="submit">' +
15  ' ' +
    '</button>' +
    '</form>' +
    '</div>' +
    '<div id="toolbar_font" class="m-tables-toolbar-block">' +
20  ' <div class="m-tables-toolbar-font-up">' +
    ' <input class="m-tables-font-family-selector" ' +
    ' id="font-family-selector" ' +
    ' title="" ' +
    ' name="font-family-selector" ' +
25  ' type="text" ' +
    ' value="Calibri" ' +
    ' autocomplete="off" ' +
    ' onchange="updateFont(this)" ' +
    ' list="fonts"/>' +
30  ' <datalist id="fonts">' +
    ' <option value="Calibri">Calibri</option>' +
    ' </datalist>\n' +
    ' <input class="m-tables-font-size-selector" \n' +
    ' id="font-size-selector" \n' +
35  ' title="\'' . get_string('\''font_size_title\'', \''mod_tables\'')
    . \'' \n' +
    ' onchange="updateFont(this)" \n' +
    ' type="number" min="1" max="409" value="11" xmlns="http://
        www.w3.org/1999/html"/>\n' +
    ' </div>\n' +
    ' <div class="m-tables-toolbar-font-down">\n' +
40  ' <button class="m-tables-toolbar-button" id="font-bold-
        button" onclick="updateFont(this)" \n' +
    ' title="\'' . get_string('\''font_bold_title\'', \''mod_tables
        \'') . \''>\n' +
    ' \n' +
    ' </button>\n' +
    ' <button class="m-tables-toolbar-button" id="font-italic-
        button" onclick="updateFont(this)" \n' +

```

```

45 'title="\'.get_string(\'font_italic_title\', \'mod_tables\')
    .\'">\n' +
    '\n' +
    '</button>\n' +
    '<button class="m-tables-toolbar-button" id="font-underline-
      button" onclick="updateFont(this)" \n' +
    'title="\'.get_string(\'font_underline_title\', \'mod_tables
      \').\'">\n' +
50 '\n' +
    '</button>\n' +
    '</div>\n' +
    '</div>\n' +
    '<div id="toolbar_align" class="m-tables-toolbar-block">\n'
    +
55 '<div class="m-tables-toolbar-align">\n' +
    '<button class="m-tables-toolbar-button" id="text-left-
      button" onclick="updateFont(this)" \n' +
    'title="\'.get_string(\'text_align_left_title\', \'
      mod_tables\').\'" >\n' +
    '\n' +
    '</button>\n' +
60 '<button class="m-tables-toolbar-button" id="text-center-
      button" onclick="updateFont(this)" \n' +
    'title="\'.get_string(\'text_align_center_title\', \'
      mod_tables\').\'" >\n' +
    '\n' +
    '</button>\n' +
    '<button class="m-tables-toolbar-button" id="text-right-
      button" onclick="updateFont(this)" \n' +
65 'title="\'.get_string(\'text_align_right_title\', \'
      mod_tables\').\'" >\n' +
    '\n' +
    '</button>\n' +
    '</div>\n' +
    '<div class="m-tables-toolbar-align">\n' +
70 '\n' +
    '</div>\n' +
    '</div>\n' +
    '<div class="m-tables-toolbar-block-attach">\n' +
    '<div class="m-tables-toolbar-attach">\n' +

```

```

75  <button class="m-tables-toolbar-button" id="
      attach_cell_to_users" onclick="onclickAttach(this)"\n' +
      <div class="m-dropdown" id="dropdown_attach_students" style
        ="display:none;" >\n' +
80  <div class="m-dropdown-display">\n' +
      <input class="m-dropdown-checked" type="text" id="
        display_selected_students" value="user 2">\n' +
      <input class="m-dropdown-search" autocomplete="off" type="
        text" oninput="onInputSearch(this)" id="search_students"
        data-attach-to="user">\n' +
      </div>\n' +
      <div class="m-dropdown-content" id="dropdown-content-users
        ">\n' +
85  <p>\n' +
      <input id="checkbox_attach_users" class="m-user-check" data
        -attach-to="user" data-attach-name="user 1" value="user
        1" type="checkbox" onclick="onclickCheckboxAttach(this)
        ">\n' +
      <label class="m-tables-user-label">user 1</label>\n' +
      </p>\n' +
      </div>\n' +
90  </div>\n' +
      <input class="m-dropdown-students-cell" id="first_cell-
        students" type="text" value="A1" readonly>\n' +
      <input class="m-dropdown-students-cell" id="last_cell-
        students" type="text" value="B2" readonly>\n' +
      <div id="submit_user_btns" style="display: none">\n' +
      <span class="m-tables-green-btn">\n' +
95  <i id="submit_attach_users" class="fa fa-check" data-attach
        -to="user" onclick="onclickSubmitAttach(this, messages)"
        ></i>\n' +
      </span>\n' +
      <span class="m-tables-red-btn">\n' +
      <i id="cansel_attach_users" class="fa fa-times" data-attach
        -to="user" onclick="onclickCanselAttach(this)" ></i>\n' +
      </span>\n' +
100 </div>\n' +
      </div>\n' +
      <div class="m-tables-toolbar-attach">\n' +
      <button class="m-tables-toolbar-button" id="
        attach_cell_to_groups" onclick="onclickAttach(this)"\n' +

```



```

105      ' title="" value="off" data-attach-to="group">\n' +
      '\n' +
      '</button>\n' +
      '<div class="m-dropdown" id="dropdown_attach_groups" style="
        display:none;" >\n' +
      '<div class="m-dropdown-display">\n' +
      '<input class="m-dropdown-checked" type="text" id="
        display_selected_groups" value="group 2">\n' +
110 '<input class="m-dropdown-search" autocomplete="off" type="
        text" oninput="onInputSearch(this)" id="search_groups"
        data-attach-to="group">\n' +
      '</div>\n' +
      '<div class="m-dropdown-content" id="dropdown-content-groups
        ">\n' +
      '<p>\n' +
      '<input id="checkbox_attach_groups" class="m-group-check"
        data-attach-to="group" data-attach-name="group 1" value="
        groupid" type="checkbox" onclick="onclickCheckboxAttach(
        this)">\n' +
115 '<label class="m-tables-user-label">Group 1</label>\n' +
      '</p>\n' +
      '</div>\n' +
      '</div>\n' +
      '<input class="m-dropdown-groups-cell" id="first_cell-groups
        " type="text" value="A1" readonly>\n' +
120 '<input class="m-dropdown-groups-cell" id="last_cell-groups"
        type="text" value="B2" readonly>\n' +
      '<div id="submit_group_btns" style="display: none">\n' +
      '<span class="m-tables-green-btn">\n' +
      '<i id="submit_attach_groups" class="fa fa-check" data-
        attach-to="group" onclick="onclickSubmitAttach(this,
        messages)" ></i>\n' +
      '</span>\n' +
125 '<span class="m-tables-red-btn">\n' +
      '<i id="cansel_attach_groups" class="fa fa-times" data-
        attach-to="group" onclick="onclickCanselAttach(this)" ></
        i>\n' +
      '</span>\n' +
      '</div>\n' +
      '</div>\n' +
130 '</div>\n' +
      '<div id="grade_block" class="m-tables-toolbar-block
        disabled">\n' +
      '<div class="m-tables-toolbar-grade-up">\n' +

```

```

135 ' <select id="select_user_grade">\n' +
' <option value="userid">\n' +
' user 1\n' +
' </option>' +
' </select>\n' +
' </div>\n' +
' <div class="m-tables-toolbar-grade-down">\n' +
140 ' <input id="input_grade" type="number" min="0" max="100"
oninput="oninputGrade(this)" onchange="onChangeInputGrade
()' ">\n' +
' <i class="fa fa-check m-tables-check-grade" id="check_grade
"></i>\n' +
' <button class="btn btn-primary m-tables-toolbar-grade-
button" onclick="gradeCell()">Оценить</button>\n' +
' <span class="m-tables-blue-btn">\n' +
' <i class="fa fa-plus" id="show_feedback_btn" onclick="
showFeedback(this)"></i>\n' +
145 ' </span>\n' +
' </div>\n' +
' </div>\n' +
' <div id="feedback_block" class="m-tables-toolbar-block m-
tables-toolbar-grade-textarea" style="display: none">\n'
+
' <textarea id="feedback_textarea"></textarea>\n' +
150 ' </div>\n' +
' <div class="m-tables-toolbar-block">\n' +
' <div id="visibility_block" class="m-tables-toolbar-visible-
up disabled">\n' +
' <select id="select_cell_visibility" onchange="
onChangeSelectVisibility()">\n' +
' <option value="teacher">teacher</option>\n' +
155 ' <option value="all">all</option>\n' +
' <option value="user">user</option>\n' +
' <option value="group">group</option>\n' +
' </select>\n' +
' </div> \n' +
160 ' </div> \n' +
' </div> \n' +
' <div class="m-tables-input-bar" id="input_bar">\n' +
' <input style="display: none"\n' +
' type="text" id="prev_cell">\n' +
165 ' <input class="m-tables-focused-cell" \n' +
' type="text" id="focused_cell" \n' +
' onchange="onChangeInputCell(this)" />\n' +
' <input class="m-tables-focused-cell-content" \n' +

```

```

170      'type="text" \n' +
      'onchange = "onChangeInputContent(this)" \n' +
      'id="focused_cell_content" />\n' +
      '</div>'+
      '<div class="m-tables-settings">\n' +
      '<table id="main_table" data-id="1" data-moduleinstance="1"
        data-sheet="1" data-user-role="teacher" data-user="1">\n'
        +
175      '<thead>\n' +
      '<tr>\n' +
      '<td></td>\n' +
      '<td>\n' +
      '<input class="resizable-column" \n' +
180      'type="text" \n' +
      'id="col_A" \n' +
      'style="width: 300px;" \n' +
      'value="A" readonly />\n' +
      '</td>\n' +
185      '</tr>\n' +
      '</thead>\n' +
      '<tbody>\n' +
      '<tr>\n' +
      '<td>\n' +
190      '<input class="resizable-row" \n' +
      'type="text" \n' +
      'id="row_1" \n' +
      'style="height:50px;" \n' +
      'value="1" readonly />\n' +
195      '</td>\n' +
      '<td>\n' +
      '<textarea name="cell_textarea" \n' +
      'disabled \n' +
      'data-visibility = "all" \n' +
200      'data-attached="false" \n' +
      'style="\n' +
      'font-family: Arial; \n' +
      'font-size: 14pt; \n' +
      'font-weight: bold; \n' +
205      'font-style: italic; \n' +
      'text-decoration: underline; \n' +
      'text-align: center; "\n' +
      'onfocus="onFocusInCell(this)" \n' +
      'onchange="saveCellHistory(this)" \n' +
210      'oninput="updateTablesCell(this)" \n' +
      'id="A1">' +

```

```

'Hello' +
'</textarea>\n' +
'</td>\n' +
215 ' <td>\n' +
'<textarea name="cell_textarea" \n' +
'disabled \n' +
'data-visibility = "all" \n' +
'data-attached="false" \n' +
220 'style="\n' +
'font-family: Calibri; \n' +
'font-size: 10pt; \n' +
'font-weight: normal; \n' +
'font-style: normal; \n' +
225 'text-decoration: none; \n' +
'text-align: left; "\n' +
'onfocus="onFocusInCell(this)" \n' +
'onchange="saveCellHistory(this)" \n' +
'oninput="updateTablesCell(this)" \n' +
230 'id="B2">\n' +
'World\n' +
'</textarea>\n' +
'</td>\n' +
'</tr>\n' +
235 ' </tbody>\n' +
' </table>\n' +
'<ul class="m-sheet-custom-menu" id="custom_menu_1">\n' +
'<li id="delete_sheet" data-action = "delete_sheet" >Delete
  </li>\n' +
'</ul>\n' +
240 ' <form method="post">\n' +
'<div class="m-tables-sheet-bar" id="sheet_bar">\n' +
'<button class="m-tables-sheet-select" type="submit" name="
  sheet" value="1" id="sheet_1" >\n' +
'Sheet 1 \n' +
'</button>\n' +
245 ' </div>\n' +
'<span class="m-tables-sheet-add">\n' +
'<i class="fa fa-plus" id="add_sheet_for_module_1" onclick="
  createSheet(this)"></i>\n' +
'</span>\n' +
'</form>\n' +
250 ' <button class="btn btn-primary" style="border-radius: 5px"
  onclick="deleteSheet()">Delete sheet</button>\n' +
'</div>\n' +

```

```

    ' <script> \n' +
    ' localStorage.KEY = false; \n' +
255 '     let messages = ["Message 1", "Message 2"] \n' +
    '     let socket = false \n' +
    '</script>\n';

describe('tables_tests', function() {
260   const func = require('../../../../MoodleServer401/
      server/moodle/mod/tables/tests/update_data.js');

   test('Attach users test', () => {
     func.onclickAttach(document.getElementById('
       attach_cell_to_users'))
     expect(document.getElementById('dropdown_attach_students
       ').style.display).toBe("inline-block");
265   expect(document.getElementById('attach_cell_to_users').
       style.border).toBe("1px solid black");
     expect(document.getElementById('input_bar').classList).
       toContain("disabled");

     Array.prototype.forEach.call(document.
       getElementsByClassName('m-tables-toolbar-block'),
       function (element, idx){
270       expect(element.classList).toContain("disabled")
     })

     func.onclickAttach(document.getElementById('
       attach_cell_to_users'))
     expect(document.getElementById('attach_cell_to_users').
       style.border).toBe("");

275   Array.prototype.forEach.call(document.
       getElementsByClassName('m-tables-toolbar-block'),
       function (element, idx){
         expect(element.classList).not.toContain("disabled")
       })
   });

280   test('Attach groups test', () => {

     func.onclickAttach(document.getElementById('
       attach_cell_to_groups'))
     expect(document.getElementById('dropdown_attach_groups')
       .style.display).toBe("inline-block");

```

```

285 expect(document.getElementById('attach_cell_to_groups').
      style.border).toBe("1px solid black");
expect(document.getElementById('input_bar').classList).
      toContain("disabled");

Array.prototype.forEach.call(document.
      getElementsByClassName('m-tables-toolbar-block'),
      function (element, idx){
290 expect(element.classList).toContain("disabled")
    })

func.onclickAttach(document.getElementById('
      attach_cell_to_groups'))
expect(document.getElementById('attach_cell_to_groups').
      style.border).toBe("");
295

Array.prototype.forEach.call(document.
      getElementsByClassName('m-tables-toolbar-block'),
      function (element, idx){
      expect(element.classList).not.toContain("disabled")
    })
300 });

test('Submit attach users test', () => {

    func.onclickAttach(document.getElementById('
      attach_cell_to_users'))
    expect(document.getElementById('dropdown_attach_students
      ').style.display).toBe("inline-block");
305

    func.onclickSubmitAttach(document.getElementById('
      submit_attach_users'), messages)
    expect(document.getElementById('submit_group_btns').
      style.display).toBe('none');

    expect(document.getElementById('first_cell-students').
      value).toBe('');
310 expect(document.getElementById('last_cell-students').
      value).toBe('');
    expect(document.getElementById('A1').style.border).toBe
      ('');
    expect(document.getElementById('B2').style.border).toBe
      ('');

```

```

    Array.prototype.forEach.call(document.
        getElementsByClassName('m-tables-toolbar-block'),
        function (element, idx){
315     expect(element.classList).not.toContain("disabled")
        })
    });

    test('Submit attach groups test', () => {
320
        func.onclickAttach(document.getElementById('
            attach_cell_to_groups'))
        expect(document.getElementById('dropdown_attach_groups')
            .style.display).toBe("inline-block");

        func.onclickSubmitAttach(document.getElementById('
            submit_attach_groups'), messages)
325     expect(document.getElementById('submit_group_btns').
        style.display).toBe('none');

        expect(document.getElementById('first_cell-groups').
            value).toBe('');
        expect(document.getElementById('last_cell-groups').value
            ).toBe('');
        expect(document.getElementById('A1').style.border).toBe
            ('');
330     expect(document.getElementById('B2').style.border).toBe
            ('');

        Array.prototype.forEach.call(document.
            getElementsByClassName('m-tables-toolbar-block'),
            function (element, idx){
                expect(element.classList).not.toContain("disabled")
            })
335    });

    test('Cansel attach test', () => {
        func.onclickAttach(document.getElementById('
            attach_cell_to_users'))
        document.getElementById('first_cell-students').value = '
            A1'
340     document.getElementById('last_cell-students').value = '
            B2'

        func.onclickCanselAttach(document.getElementById('
            cansel_attach_users'))

```

```

expect(document.getElementById('submit_user_btns').style
    .display).toBe('none');
expect(document.getElementById('first_cell-students').
    value).toBe('');
345 expect(document.getElementById('last_cell-students').
    value).toBe('');
expect(document.getElementById('A1').style.border).toBe
    ('');
expect(document.getElementById('B2').style.border).toBe
    ('');

func.onclickAttach(document.getElementById('
    attach_cell_to_groups'))
350 document.getElementById('first_cell-groups').value = 'A1
    ,
document.getElementById('last_cell-groups').value = 'B2'

func.onclickCanselAttach(document.getElementById('
    cansel_attach_groups'))
expect(document.getElementById('submit_user_btns').style
    .display).toBe('none');
355 expect(document.getElementById('first_cell-groups').
    value).toBe('');
expect(document.getElementById('last_cell-groups').value
    ).toBe('');
expect(document.getElementById('A1').style.border).toBe
    ('');
expect(document.getElementById('B2').style.border).toBe
    ('');
});
360

test('Update Cell test', () => {

    func.updateTablesCell(document.getElementById('A1'))
    expect(document.getElementById("focused_cell_content").
        value).toBe(document.getElementById('A1').innerHTML);
365 });

test('Click ceckbox attach test', () => {
    document.getElementById('chechbox_attach_users').checked
        = true;
    func.onclickCheckboxAttach(document.getElementById('
        chechbox_attach_users'))
370 expect(document.getElementById('
        display_selected_students').value).toBe("user 1");

```



```

document.getElementById('checkbox_attach_groups').
  checked = true;
func.onclickCheckboxAttach(document.getElementById('
  checkbox_attach_groups'))
expect(document.getElementById('display_selected_groups
  ').value).toBe("group 1");
375 });

test('Attach focus cell test', () => {
  document.getElementById('attach_cell_to_groups').value =
    "off";
  document.getElementById('attach_cell_to_users').value =
    "on";
380 document.getElementById('first_cell-students').value =
    "";
  document.getElementById('A1').style.border = "1px solid
    black";

  func.onFocusInCellAttach(document.getElementById('A1'))
  expect(document.getElementById('first_cell-students').
    value).toBe("A1");
385 expect(document.getElementById('A1').style.border).toBe
    ("1px solid #27a7d8");

  func.onFocusInCellAttach(document.getElementById('A1'))
  expect(document.getElementById('first_cell-students').
    value).toBe("A1");
  expect(document.getElementById('A1').style.border).toBe
    ("1px solid #ff9a00");
390

  func.onFocusInCellAttach(document.getElementById('A1'))
  expect(document.getElementById('first_cell-students').
    value).toBe("");
  expect(document.getElementById('A1').style.border).toBe
    ("");

395 func.onFocusInCellAttach(document.getElementById('A1'))
  expect(document.getElementById('first_cell-students').
    value).toBe("A1");
  expect(document.getElementById('A1').style.border).toBe
    ("1px solid #27a7d8");

  func.onFocusInCellAttach(document.getElementById('B2'))

```

```

400     expect(document.getElementById('last_cell-students').
        value).toBe("B2");
    expect(document.getElementById('B2').style.border).toBe
        ("1px solid #ff9a00");

    func.onFocusInCellAttach(document.getElementById('A1'))
    expect(document.getElementById('first_cell-students').
        value).toBe("");
405     expect(document.getElementById('A1').style.border).toBe
        ("");
    });

    test('Focus cell test', () => {

410        func.onFocusInCellFocus(document.getElementById('A1'))
        expect(document.getElementById("focused_cell").value).
            toBe("A1");
        expect(document.getElementById("prev_cell").value).toBe
            ("A1");
        expect(document.getElementById("focused_cell_content").
            value).toBe("Hello");
        expect(document.getElementById("font-family-selector").
            value).toBe("Arial");
415        expect(document.getElementById("font-size-selector").
            value).toBe("14");
        expect(document.getElementById("font-bold-button").style
            .border).toBe("1px solid black");
        expect(document.getElementById("font-italic-button").
            style.border).toBe("1px solid black");
        expect(document.getElementById("font-underline-button").
            style.border).toBe("1px solid black");
        expect(document.getElementById("text-center-button").
            style.border).toBe("1px solid black");
420

        func.onFocusInCellFocus(document.getElementById('B2'))
        expect(document.getElementById("focused_cell").value).
            toBe("B2");
        expect(document.getElementById("prev_cell").value).toBe
            ("B2");
        expect(document.getElementById("focused_cell_content").
            value).toBe("World");
425        expect(document.getElementById("font-family-selector").
            value).toBe("Calibri");
        expect(document.getElementById("font-size-selector").
            value).toBe("10");

```

```

expect(document.getElementById("font-bold-button").style
    .border).toBe("");
expect(document.getElementById("font-italic-button").
    style.border).toBe("");
expect(document.getElementById("font-underline-button").
    style.border).toBe("");
430 expect(document.getElementById("text-center-button").
    style.border).toBe("");
});

test('Change input cell test', () => {

435     document.getElementById('focused_cell').value = ""

    func.onChangeInputCell(document.getElementById('
        focused_cell'))
    expect(document.getElementById("focused_cell").value).
        toBe("");
    expect(document.getElementById("prev_cell").value).toBe
        ("");
440 expect(document.getElementById("focused_cell_content").
        value).toBe("");
    expect(document.getElementById("font-family-selector").
        value).toBe("Calibri");
    expect(document.getElementById("font-size-selector").
        value).toBe("10");
    expect(document.getElementById("font-bold-button").style
        .border).toBe("");
    expect(document.getElementById("font-italic-button").
        style.border).toBe("");
445 expect(document.getElementById("font-underline-button").
        style.border).toBe("");
    expect(document.getElementById("text-center-button").
        style.border).toBe("");

    document.getElementById('focused_cell').value = "A1"

450     func.onChangeInputCell(document.getElementById('
        focused_cell'))
    expect(document.getElementById("focused_cell").value).
        toBe("A1");
    expect(document.getElementById("prev_cell").value).toBe
        ("A1");
    expect(document.getElementById("focused_cell_content").
        value).toBe("Hello");

```

```

expect(document.getElementById("font-family-selector").
    value).toBe("Arial");
455 expect(document.getElementById("font-size-selector").
    value).toBe("14");
expect(document.getElementById("font-bold-button").style
    .border).toBe("1px solid black");
expect(document.getElementById("font-italic-button").
    style.border).toBe("1px solid black");
expect(document.getElementById("font-underline-button").
    style.border).toBe("1px solid black");
expect(document.getElementById("text-center-button").
    style.border).toBe("1px solid black");
460

document.getElementById('focused_cell').value = "B2"

func.onChangeInputCell(document.getElementById('
    focused_cell'))
expect(document.getElementById("focused_cell").value).
    toBe("B2");
465 expect(document.getElementById("prev_cell").value).toBe
    ("B2");
expect(document.getElementById("focused_cell_content").
    value).toBe("World");
expect(document.getElementById("font-family-selector").
    value).toBe("Calibri");
expect(document.getElementById("font-size-selector").
    value).toBe("10");
expect(document.getElementById("font-bold-button").style
    .border).toBe("");
470 expect(document.getElementById("font-italic-button").
    style.border).toBe("");
expect(document.getElementById("font-underline-button").
    style.border).toBe("");
expect(document.getElementById("text-center-button").
    style.border).toBe("");
});

475 test('Change input content test', () => {

    document.getElementById("focused_cell_content").value =
        "Test text"
    document.getElementById('focused_cell').value = "A1"

480 func.onChangeInputContent(document.getElementById("
    focused_cell_content"))

```

```

    expect(document.getElementById("A1").value).toBe("Test
      text");
  });

  test('Update font test', () => {
485    document.getElementById('focused_cell').value = "A1"
    func.onChangeInputCell(document.getElementById('
      focused_cell'))
    document.getElementById("font-family-selector").value =
      "Times new roman"

    func.updateFont(document.getElementById("font-family-
      selector"))
490    expect(document.getElementById("A1").style.fontFamily).
      toBe("Times new roman");

    document.getElementById("font-size-selector").value =
      "22"

    func.updateFont(document.getElementById("font-size-
      selector"))
495    expect(document.getElementById("A1").style.fontSize).
      toBe("22pt");

    document.getElementById('focused_cell').value = "B2"
    func.onChangeInputCell(document.getElementById('
      focused_cell'))

500    func.updateFont(document.getElementById("font-bold-
      button"))
    expect(document.getElementById("B2").style.fontWeight).
      toBe("bold");
    func.updateFont(document.getElementById("font-bold-
      button"))
    expect(document.getElementById("B2").style.fontWeight).
      toBe("normal");

505    func.updateFont(document.getElementById("font-italic-
      button"))
    expect(document.getElementById("B2").style.fontStyle).
      toBe("italic");
    func.updateFont(document.getElementById("font-italic-
      button"))
    expect(document.getElementById("B2").style.fontStyle).
      toBe("normal");

```

```

510     func.updateFont(document.getElementById("font-underline-
        button"))
    expect(document.getElementById("B2").style.
        textDecoration).toBe("underline");
    func.updateFont(document.getElementById("font-underline-
        button"))
    expect(document.getElementById("B2").style.
        textDecoration).toBe("none");

515     func.updateFont(document.getElementById("text-right-
        button"))
    expect(document.getElementById("B2").style.textAlign).
        toBe("right");

    func.updateFont(document.getElementById("text-center-
        button"))
    expect(document.getElementById("B2").style.textAlign).
        toBe("center");

520     func.updateFont(document.getElementById("text-left-
        button"))
    expect(document.getElementById("B2").style.textAlign).
        toBe("left");
  });

525  test('Grade cell test', () => {
    document.getElementById("feedback_block").style.display
      = "inline-block";
    document.getElementById("feedback_textarea").value = "
      Test feedback";

    func.gradeCell()
530    expect(document.getElementById('check_grade').classList)
      .toContain("m-tables-show");

    document.getElementById("feedback_block").style.display
      = "none";
  });

535  test('Change grade test', () => {

    func.onChangeInputGrade()
    expect(document.getElementById('check_grade').classList)
      .not.toContain("m-tables-show");

```

```

    });

540 test('Wrong input grade test', () => {
    document.getElementById('input_grade').value = 101;

    func.oninputGrade(document.getElementById('input_grade')
    )
545 expect(document.getElementById('input_grade').value).
    toBe("100");

    document.getElementById('input_grade').value = -1;

    func.oninputGrade(document.getElementById('input_grade')
    )
550 expect(document.getElementById('input_grade').value).
    toBe("0");

    document.getElementById('input_grade').value = 75;

    func.oninputGrade(document.getElementById('input_grade')
    )
555 expect(document.getElementById('input_grade').value).
    toBe("75");
    });

test('Show feedback test', () => {
    document.getElementById('input_grade').value = 101;
560 func.showFeedback(document.getElementById('
    show_feedback_btn'))
    expect(document.getElementById("feedback_block").style.
        display).toBe("inline-block");
    expect(document.getElementById('show_feedback_btn').
        classList).toContain("fa-minus");

565 func.showFeedback(document.getElementById('
    show_feedback_btn'))
    expect(document.getElementById("feedback_block").style.
        display).toBe("none");
    expect(document.getElementById('show_feedback_btn').
        classList).toContain("fa-plus");
    });

570 test('Change visibility test', () => {
    document.getElementById('focused_cell').value = "A1"

```

```

        document.getElementById('select_cell_visibility').value
            = "user"

        func.onChangeSelectVisibility()
575    expect(document.getElementById('A1').getAttribute("data-
            visibility")).toBe("user");
    });
});

```

II.13. styles.css

```

5    .path-mod-tables .m-tables-settings div {
        overflow: scroll;
    }

    .path-mod-tables .m-tables-settings table {
        display: block;
        overflow: scroll;
        max-height: 35em;
10    }

    .path-mod-tables .m-tables-settings td {
        padding: 0;
        border: 1px solid darkgray;
15    height: 25px;
        width: 25px;
    }

    .path-mod-tables .m-tables-settings textarea {
20    resize: none;
        vertical-align: top;
        border: 0;
        min-height: 50px;
        min-width: -webkit-fill-available;
25    width: 100%;
        height: 100%;
        font-family: Calibri, serif;
        font-size: 11pt;
    }
30

    .path-mod-tables .m-tables-settings textarea:focus {
        outline: none;
    }

```



```

35 .path-mod-tables .m-tables-settings textarea:disabled {
    background-color: #c3d6e9;
}

40 .path-mod-tables .m-tables-settings th {
    padding: 0;
    border: 1px solid darkgray;
    text-align: center;
    line-height: 10px;
    height: 25px;
45 width: 25px;
    background-color: #f8f9fa;
}

.path-mod-tables .m-tables-settings table::-webkit-scrollbar
50 {
    width: 100%;
}

.path-mod-tables .m-tables-settings table::-webkit-scrollbar
55 -track {
    background: #fff;
}

.path-mod-tables .m-tables-settings table::-webkit-scrollbar
    -thumb {
    background-color: #e9ecef;
    border-radius: 5px;
60 border: 1px solid #9da0a3;
}

.path-mod-tables .m-tables-input-bar {
    display: inline-flex;
65 width: 100%;
    height: 30px;
    padding: 5px;
    margin-bottom: 5px;
}

70 .path-mod-tables .m-tables-input-bar:disabled {
    pointer-events: none;
    opacity: 0.4;
}

75

```

```

80 .path-mod-tables .m-tables-focused-cell {
    height: 25px;
    border: 1px solid black;
    border-radius: 3px;
    width: 10%;
}

85 .path-mod-tables .m-tables-focused-cell-content {
    height: 25px;
    margin-left: 5px;
    border: 1px solid black;
    border-radius: 3px;
    width: 89%;
    float: right;
90 }

.path-mod-tables .m-tables-toolbar{
    display: inline-block;
    width: 100%;
95 border: 1px solid darkgray;
}

.path-mod-tables .m-tables-toolbar-block{
    float: left;
100 width: fit-content;
    height: 75px;
    border-right: 1px solid darkgray;
    margin: 5px 0 5px 0;
105 }

.path-mod-tables .m-tables-toolbar-block-attach{
    float: left;
    width: fit-content;
    height: 75px;
110 border-right: 1px solid darkgray;
    margin: 5px 0 5px 0;
}

.path-mod-tables .m-tables-toolbar-block:disabled {
115 pointer-events: none;
    opacity: 0.4;
}

.path-mod-tables .m-tables-toolbar-font-up{
120 height: 39px;

```

```
padding: 5px;
}

125 .path-mod-tables .m-tables-toolbar-font-down{
    height: 34px;
    padding: 5px;
}

130 .path-mod-tables .m-tables-toolbar-button{
    padding: 0;
    border: 0;
    background-color: white;
    width: 28px;
    height: 28px;
135 }

.path-mod-tables .m-tables-toolbar input:hover{
    background-color: #ececec;
}

140 .path-mod-tables .m-tables-font-family-selector{
    height: 35px;
    border: 1px solid #a4a4a4;
}

145 .path-mod-tables .m-tables-font-size-selector{
    min-width: 50px;
    max-width: 50px;
    height: 35px;
150 vertical-align: top;
    border: 1px solid #a4a4a4;
}

.path-mod-tables .m-tables-toolbar-align input:hover{
155 background-color: #ececec;
}

.path-mod-tables .m-tables-toolbar-align{
    height: 34px;
160 padding: 5px;
}

.path-mod-tables .m-tables-toolbar-attach{
    padding: 5px;
165 }
```

```
170 .path-mod-tables .m-tables-toolbar-img{
    padding: 0;
    margin: 1px;
    width: 24px;
    height: 24px;
}

175 .path-mod-tables .m-dropdown {
    position: static;
    vertical-align: middle;
}

180 .path-mod-tables .m-dropdown-display{
    border: 1px solid black;
    position: relative;
    width: 182px;
    height: 30px;
}

185 .path-mod-tables .m-dropdown-checked{
    position: absolute;
    border: none;
    z-index: 10;
    width: 100%;
    height: 100%;
    margin: 0;
    padding: 1px 2px;
}

190 .path-mod-tables .m-dropdown-search{
    z-index: 11;
    top: 0;
    left: 0;
    opacity: 0;
    position: absolute;
    border: none;
    width: 100%;
    height: 100%;
}

195 .path-mod-tables .m-dropdown-students-cell{
    display: none;
    vertical-align: middle;
    width: 30px;
}
```

```

    height: 26px;
    border: 1px solid black;
    border-radius: 2px;
}
215
.path-mod-tables .m-dropdown-groups-cell{
    display: none;
    vertical-align: middle;
    width: 30px;
220    height: 26px;
    border: 1px solid black;
    border-radius: 2px;
}

225 .path-mod-tables .m-dropdown-content {
    display: none;
    position: absolute;
    width: 182px;
    max-height: 200px;
230    overflow: auto;
    background-color: white;
    box-shadow: 0 8px 16px 0 rgba(0,0,0,0.2);
    z-index: 1;
}

235 .path-mod-tables .m-dropdown-content p {
    margin: 0;
    color: black;
    padding: 5px;
240    text-decoration: none;
}

.m-dropdown-content a:hover {background-color: #ddd;}

245 .m-dropdown:hover .m-dropdown-content {display: block;}

.m-dropdown-search:focus {opacity: 1;}

250 .path-mod-tables .m-tables-toolbar-grade{
    float: left;
    width: fit-content;
    height: 75px;
    border-right: 1px solid darkgray;
    margin: 5px 0 5px 0;
255 }

```

```

260 .path-mod-tables .m-tables-toolbar-grade-up{
    padding: 5px;
}

265 .path-mod-tables .m-tables-toolbar-grade-up select{
    width: 182px;
    height: 30px;
    padding: 1px 2px;
    border: 1px solid gray;
    border-radius: 2px;
}

270 .path-mod-tables .m-tables-toolbar-grade-up input{
    width: 30px;
    height: 26px;
    padding: 1px 2px;
    border: 1px solid gray;
    border-radius: 2px;
275 }

280 .path-mod-tables .m-tables-toolbar-grade-down{
    padding: 5px;
    text-align: center;
}

285 .path-mod-tables .m-tables-toolbar-grade-down input{
    width: 60px;
    height: 26px;
    padding: 1px 2px;
    border: 1px solid gray;
    border-radius: 2px;
    vertical-align: middle;
290 }

295 .path-mod-tables .m-tables-toolbar-grade-button{
    padding: 2px;
    height: 30px;
    font-size: 0.8em;
    margin-left: 35px;
}

300 .path-mod-tables .m-tables-toolbar-grade-textarea{
    align-content: center;
    padding: 5px;
}

```

```

}

.path-mod-tables .m-tables-toolbar-grade-textarea textarea{
  resize: none;
305   border-radius: 5px;
      height: 65px;
}

.path-mod-tables .m-tables-toolbar-load-up{
310   padding: 5px;
      margin: 0;
}

.path-mod-tables .m-tables-toolbar-load-down{
315   padding: 5px;
      margin: 0;
}

.path-mod-tables .m-userbox{
320   display: inline-table;
      width: 100%;
}

.path-mod-tables .m-userbox thead{
325   text-align: center;
      background-color: #f8f9fa;
}

.path-mod-tables .m-userbox td{
330   padding: 5px;
      border: 1px solid darkgray;
}

.path-mod-tables .m-attach-cells-btn{
335   float: right;
}

.path-mod-tables .m-tables-blue-btn i{
340   cursor: pointer;
      color: #0f6cbf;
}

.path-mod-tables .m-tables-red-btn i{
345   cursor: pointer;
      color: #ca3120;

```

```

}

.path-mod-tables .m-tables-green-btn i{
  cursor: pointer;
350   color: #357a32;
}

.path-mod-tables .m-attach-cells-bar input{
  border: 1px solid black;
355   border-radius: 5px;
  width: 50px;
}

.path-mod-tables .m-attach-cells-delete-btn{
360   margin-left: 5px;
  margin-right: 5px;
}

.path-mod-tables .m-attach-cells-selector{
365   margin: 5px 0;
  border-radius: 5px;
}

.path-mod-tables .m-tables-toolbar-visible-up{
370   padding: 5px;
}

.path-mod-tables .m-tables-toolbar-visible-up select{
375   width: 110px;
  height: 30px;
  padding: 1px 2px;
  border: 1px solid gray;
  border-radius: 2px;
}

380
.path-mod-tables .m-tables-sheet-bar{
  display: inline-block;
  height: fit-content;
  overflow: hidden !important;
385 }

.path-mod-tables .m-tables-sheet-select{
  margin: 1px;
  max-width: 120px;
390   min-width: 50px;

```



```

    vertical-align: middle;
    width: fit-content;
    border: 1px solid black;
    background-color: #bfd3ef80;
395    border-bottom-left-radius: 5px;
    border-top-left-radius: 5px;
}

.path-mod-tables .m-tables-sheet-select:disabled{
400    opacity: 1;
    color: black;
    background-color: #a2dda9;
}

405 .path-mod-tables .m-tables-sheet-select img{
    vertical-align: middle;
    height: 23px;
    animation-name: rotation;
    animation-duration: 1s;
410    animation-iteration-count: infinite;
    animation-timing-function: linear;
}

@keyframes rotation {
415    0% {
        transform: rotate(0deg);
    }
    100% {
        transform: rotate(360deg);
420    }
}

.path-mod-tables .m-tables-sheet-add{
425    display: contents;
}

.path-mod-tables .m-tables-sheet-add i{
430    margin: 1px 0;
    width: 25px;
    height: 27px;
    vertical-align: top;
    text-align-last: center;
    align-content: center;
    border: 1px solid #000;

```

```

435     background-color: #bfd3ef80;border-bottom-right-radius: 5
        px;border-top-right-radius: 5px;
    }

    .path-mod-tables .m-tables-send{
440     float: right;
    }

    .path-mod-tables .m-tables-user-label{
        display: inline;
    }
445

    .path-mod-tables .m-tables-check-grade{
        color: #357a32;
        display: none;
    }

450
    .path-mod-tables .m-tables-show {
        display: inline-block;
        animation: onclickShow 0.5s linear forwards;
    }

455
    @keyframes onclickShow {
        0% {
            opacity: 0;
        }
460        100% {
            opacity: 1;
        }
    }

465
    .path-mod-tables .m-tables-grades-maintable{
        width: 100%;
    }

    .path-mod-tables .m-tables-grades-maintable td{
470     border-left: 1px solid darkgray;
        border-top: 1px solid darkgray;
        border-bottom: 1px solid darkgray;
    }

475
    .path-mod-tables .m-tables-grades-maintable th{
        background-color: #bfd3ef;
        padding: 0.35rem;
        border: 1px solid #bfd3ef;
    }

```

```

border-bottom: 1px solid darkgray;
480 }

.path-mod-tables .m-tables-group-name{
    text-align: center;
}
485

.path-mod-tables .m-tables-group-students{
    padding: 0;
}

490 .path-mod-tables .m-tables-group-grade{
    text-align: center;
    border-right: 1px solid darkgray;
}

495 .path-mod-tables .m-tables-students-table{
    width: 100%;
}

500 .path-mod-tables .m-tables-students-table td{
    border: none;
}

.path-mod-tables .m-tables-students-table tbody tr:nth-of-
    type(odd){
505     background-color: #f7f7f7;
}

.path-mod-tables .m-tables-students-name{
    width: 80%;
    padding: .75rem;
510 }

.path-mod-tables .m-tables-students-grades{
    padding: 0;
}
515

.path-mod-tables .m-tables-grades-table{
    width: max-content;
    padding: .75rem;
}

520 .path-mod-tables .m-tables-grades-table td{
    padding: .35rem;
}

```

```

    border: none;
}
525
.path-mod-tables .m-tables-grades-table th{
    border: none;
}

530 .path-mod-tables .m-tables-grades-readonly-input{
    background: transparent;
    width: 60px;
    padding: 5px;
    border: none
535 }

.path-mod-tables .m-tables-grades-active-input{
    background: white;
    width: 60px;
540 padding: 5px;
    border: 1px solid darkgray;
    border-radius: 5px;
}

545 .path-mod-tables .m-tables-grades-feedback{
    resize: none;
    width: 20rem;
    height: 8rem;
}

550 .path-mod-tables .m-tables-grades-table tbody tr:nth-of-type
    (odd){
    background-color: transparent;
}

555 .disabled{
    pointer-events: none;
    opacity: 0.4;
}

560 .resizable {
    overflow: hidden;
    touch-action: none;

    /* This makes things *much* easier */
565 box-sizing: border-box;
}

```

```
570 .resizable-column {  
    overflow: hidden;  
    touch-action: none;  
  
    resize: none;  
    padding: 0;  
    vertical-align: top;  
575 border: 0;  
    min-height: 20px;  
    min-width: -webkit-fill-available;  
    width: 120px;  
    height: 30px;  
580 background-color: #bfd3ef;  
    text-align: center;  
    font-weight: bold;  
  
    box-sizing: border-box;  
585 }  
  
.resizable-row {  
    overflow: hidden;  
    touch-action: none;  
590  
    resize: none;  
    padding: 0;  
    vertical-align: top;  
    border: 0;  
595 min-height: -webkit-fill-available;  
    min-width: 25px;  
    width: 25px;  
    height: 20px;  
    background-color: #bfd3ef;  
600 text-align: center;  
    font-weight: bold;  
  
    box-sizing: border-box;  
605 }  
  
.m-sheet-custom-menu {  
    display: none;  
    position: fixed;  
    overflow: hidden;  
610 padding: 0;  
    border: 1px solid #CCC;
```

```

        white-space: nowrap;
        font-family: sans-serif;
        background: #FFF;
615     color: #333;
        border-radius: 3px;
    }

    .m-sheet-custom-menu li {
620     list-style-type: none;
        padding: 8px 12px;
        cursor: pointer;
    }

625     .m-sheet-custom-menu li:hover {
        background-color: #DEF;
    }

```

П1.14. amd/src/jquery_tables_functions.js

```

/**
 * JavaScript for the view.php.
 *
5   * @module      mod_tables/updatecell
 * @copyright    2023 Mazur Egor <mazur.eh@edu.spbstu.ru>
 * @license      http://www.gnu.org/copyleft/gpl.html GNU GPL v3
                or later
 */
$(document).ready(function () {
10     let data = {
        update_type: "focusout",
        cell_id: document.getElementById("prev_element").value
    };

15     if(localStorage.socket !== "false") {
        socket.emit('send', {
            room: document.getElementById('main_table').
                getAttribute('data-moduleinstance'),
            message: data
        });
20     }
});

// Trigger action when the contexmenu is about to be shown

```

```

$('.m-tables-sheet-select').bind("contextmenu", function (
    event) {
25   let active_sheet = document.getElementById('main_table').
       getAttribute('data-sheet');

   if(active_sheet !== event.target.id.replace('sheet_', ''))
       {
       $("#delete_sheet").attr('id', 'delete_'.concat(event.
           target.id))
       // Avoid the real one
30       event.preventDefault();
       // Show contextmenu
       $(".m-sheet-custom-menu").finish().toggle(100).
       // In the right position (the mouse)
       css({
35         top: event.pageY + "px",
         left: event.pageX + "px"
       });
       }
   });

   // If the document is clicked somewhere
   $(document).bind("mousedown", function (e) {

45       // If the clicked element is not the menu
       if (!$(e.target).parents(".m-sheet-custom-menu").length >
           0) {

           // Hide it
           $(".m-sheet-custom-menu").hide(100);
50       }
   });

   // If the menu element is clicked
55   $(".m-sheet-custom-menu li").click(function(){
       let custom_menu = $(".m-sheet-custom-menu");
       let sheet_id = $(this).attr("id").replace("delete_sheet_",
           '')

       // This is the triggered action name
60       switch($(this).attr("data-action")) {
           // A case for each action. Your actions here
           case "delete_sheet":{

```

```

        let data = {
            update_type: "delete_sheet",
            table_id: custom_menu.attr("id").replace("
65             custom_menu_", ""),
            sheet_id: sheet_id
        };

        //Send information to create_sheet.php for updating
        database

70     $.ajax({
        method: "POST",
        url: "create_sheet.php",
        data: data
    });
75     break;
    }
}

$(this).attr("id", "delete_sheet")
80 $("#sheet_".concat(sheet_id)).remove()
// Hide it AFTER the action was triggered
custom_menu.hide(100);
});

```

II1.15. upload_from_excel.php

```

<?php

require_once("../../config.php");
5 require_once("lib.php");
require_once(__DIR__."/classes/form/filepicker.php");

// Check that all the parameters have been provided.
global $DB, $USER, $CFG, $OUTPUT, $PAGE;

10 require_once("$CFG->libdir/formslib.php");

// Course module id.
$id = optional_param('id', 0, PARAM_INT);
15 // Activity instance id.
$t = optional_param('t', 0, PARAM_INT);

if ($id) {

```



```

    $cm = get_coursemodule_from_id('tables', $id, 0, false,
        MUST_EXIST);
20    $course = $DB->get_record('course', array('id' => $cm->
        course), '*', MUST_EXIST);
    $moduleinstance = $DB->get_record('tables', array('id' =>
        $cm->instance), '*', MUST_EXIST);
} else {
    $moduleinstance = $DB->get_record('tables', array('id' =>
        $t), '*', MUST_EXIST);
    $course = $DB->get_record('course', array('id' =>
        $moduleinstance->course), '*', MUST_EXIST);
25    $cm = get_coursemodule_from_instance('tables',
        $moduleinstance->id, $course->id, false, MUST_EXIST);
}

$url = new moodle_url('/mod/tables/upload_from_excel.php?id='
    . $id);

30    $PAGE->set_url($url);
    $PAGE->set_title(format_string($moduleinstance->name));
    $PAGE->set_heading(format_string($course->fullname));

    $PAGE->requires->jquery();
35    require_login($course, false, $cm);

    $context = context_module::instance($cm->id);

40    require_capability('moodle/course:manageactivities',
        $context);

    $mform = new pick_excel_file('upload_from_excel.php?id=' . $id)
        ;

    echo $OUTPUT->header();
45

    if ($fromform = $mform->get_data()) {
        $mform->save_file('xlsxfile', 'upload/upload.xlsx', true);

        redirect('load_excel.php?id=' . $id);
50    }
    else {
        $mform->display();
    }
    echo $OUTPUT->footer();

```

Π1.16. load_excel.php

```

<?php

require(__DIR__.'../../config.php');
require_once(__DIR__.'lib.php');

global $CFG, $PAGE, $DB, $USER;

// Course module id.
$cid = optional_param('id', 0, PARAM_INT);
// Activity instance id.
$id = optional_param('t', 0, PARAM_INT);

if ($cid) {
    $cm = get_coursemodule_from_id('tables', $cid, 0, false,
        MUST_EXIST);
    $course = $DB->get_record('course', array('id' => $cm->
        course), '*', MUST_EXIST);
    $moduleinstance = $DB->get_record('tables', array('id' =>
        $cm->instance), '*', MUST_EXIST);
} else {
    $moduleinstance = $DB->get_record('tables', array('id' =>
        $id), '*', MUST_EXIST);
    $course = $DB->get_record('course', array('id' =>
        $moduleinstance->course), '*', MUST_EXIST);
    $cm = get_coursemodule_from_instance('tables',
        $moduleinstance->id, $course->id, false, MUST_EXIST);
}

$PAGE->set_url('/mod/tables/load_excel.php?id='.$cid);
$PAGE->requires->jquery();

require_login($course, false, $cm);

$context = context_module::instance($cm->id);

require_capability('moodle/course:manageactivities',
    $context);

$file = 'upload/upload.xlsx';

if (file_exists($file) && is_file($file) && is_readable(
    $file)) {

```

```

$z=new ZipArchive();
if ($z->open($file)) {

    $str_values=array();

40
    if ($fp=$z->getStream('xl/workbook.xml')){
        $data='';
        while (!feof($fp)) {
            $data.=fread($fp, 1024);
45
        }
        fclose($fp);

        $xml=simplexml_load_string($data);

50
        if(isset($xml->sheets)){
            $sheets = $xml->sheets;
            $sheets_count = count($sheets->sheet);
        }
    }

55
    // Прочитать строковые значения
    if ($fp=$z->getStream('xl/sharedStrings.xml')) {
        $data='';
        while (!feof($fp)) {
60
            $data.=fread($fp, 1024);
        }
        fclose($fp);

        $xml=simplexml_load_string($data);

65
        if (isset($xml->si) && count($xml->si)) {
            foreach ($xml->si as $data) {
                $data=(array)$data;
                $str_values[]=$data['t'];
70
            }
        }
    }

    $sheets_to_load = array();

75
    for($i = 1; $i<=$sheets_count; $i++){
        $xls_values=array();

        // Прочитать значения каждого листа

```

```

80 if ($fp=$z->getStream('xl/worksheets/sheet'.$i.'.xml'))
    {
        $data='';
        while (!feof($fp)) {
            $data.=fread($fp, 1024);
        }
85 fclose($fp);

        $xml=simplexml_load_string($data);

        if (isset($xml->sheetData)) {
90             $sheetData=(array)($xml->sheetData);
            if (isset($sheetData['row']) && count($sheetData['row']
                row'])>0) {

                if (!is_array($sheetData['row'])) {
                    $sheetData['row'] = array(0 => $sheetData['row
                        ']);
95                 }

                foreach($sheetData['row'] as $row) {
                    $row=(array)$row;

100                 // Особый случай для одноколоночной страницы
                    if (!is_array($row['c'])) {
                        $row['c']=array($row['c']);
                    }

                    foreach ($row['c'] as $col) {
105                         $col=(array)$col;

                        // Столбец и колонка
                        preg_match('/([A-Z]+)(\d+)/',$col['
                            @attributes']['r'],$matches);

110                        // Строка из списка
                        if (isset($col['@attributes']['t'])
                            && $col['@attributes']['t']=='s'
                            && isset($str_values[$col['v']]))
                        {
115                             $xls_values[$matches[2]][$matches[1]]=
                                $str_values[$col['v']];

                        }

                        // Непосредственное значение

```

```

120         elseif (isset($col['v'])) {
            $xls_values[$matches[2]][$matches[1]]=$col
                ['v'];
        }

        }
125     }
    $sheets_to_load[$i] = $xls_values;
}
}
}
130 }
$z->close();

for($i = 1; $i <= count($sheets_to_load); $i++){
    $data = array (
135     'name' => $i,
    'tableid' => $moduleinstance->id);

    if($DB->record_exists('tables_sheets', $data)){
140     $sheetid = $DB->get_record('tables_sheets', $data)->
        id;
        $DB->delete_records('tables_sheets_cells', array('
            sheetid' => $sheetid));
        for($j = 1, $k = 0; $j <= count($sheets_to_load[$i])
            ; $j++, $k++){
            $sheet_key = array_keys($sheets_to_load[$i])[$k];
            $keys = array_keys($sheets_to_load[$i][$sheet_key
                ]);
145     foreach($keys as $key){
        $cell_data = array (
            'sheetid' => $sheetid,
            'name' => $key.$j);

        if($DB->record_exists('tables_sheets_cells',
150     $cell_data)){
            $cell_data = $DB->get_record('
                tables_sheets_cells', $cell_data, '*',
                MUST_EXIST);
            $cell_data->content = $sheets_to_load[$i][
                $sheet_key][$key];
            $cell_data->timmodified = time();
            $DB->update_record('tables_sheets_cells', (
                object)$cell_data);
        }
    }
}
}
}

```

```

155         }
        else{
            $cell_data['content'] = $sheets_to_load[$i][
                $sheet_key][$key];
            $cell_data['timecreated'] = time();
            $DB->insert_record('tables_sheets_cells',
160                 $cell_data);
        }
    }
}
else{
165     $DB->insert_record('tables_sheets', $data);
    $sheetid = $DB->get_record('tables_sheets', $data)->
        id;
    for($j = 1, $k = 0; $j <= count($sheets_to_load[$i])
        ; $j++, $k++){
        $sheet_key = array_keys($sheets_to_load[$i])[$k];
        $keys = array_keys($sheets_to_load[$i][$sheet_key
170             ]);
        foreach($keys as $key){
            $cell_data = array (
                'sheetid' => $sheetid,
                'name' => $key.$j);

175             $cell_data['content'] = $sheets_to_load[$i][
                $sheet_key][$key];
            $cell_data['timecreated'] = time();
            $DB->insert_record('tables_sheets_cells',
                $cell_data);
        }
    }
}
180 }
}
}
else {
185     // что-то не так с доступностью файла
    redirect('view.php?id='.$id);
}
}
else {
190     // что-то не так с наличием файла
    redirect('view.php?id='.$id);
}
}

```

Π1.17. upload_from_activity.php

```

<?php

require_once("../../config.php");
require_once("lib.php");
require_once(__DIR__.'../classes/form/filepicker.php');

// Check that all the parameters have been provided.
global $DB, $USER, $CFG, $OUTPUT, $PAGE;

require_once("$CFG->libdir/formslib.php");

// Course module id.
$id = optional_param('id', 0, PARAM_INT);
// Activity instance id.
$t = optional_param('t', 0, PARAM_INT);

if ($id) {
    $cm = get_coursemodule_from_id('tables', $id, 0, false,
        MUST_EXIST);
    $course = $DB->get_record('course', array('id' => $cm->
        course), '*', MUST_EXIST);
    $moduleinstance = $DB->get_record('tables', array('id' =>
        $cm->instance), '*', MUST_EXIST);
} else {
    $moduleinstance = $DB->get_record('tables', array('id' =>
        $t), '*', MUST_EXIST);
    $course = $DB->get_record('course', array('id' =>
        $moduleinstance->course), '*', MUST_EXIST);
    $cm = get_coursemodule_from_instance('tables',
        $moduleinstance->id, $course->id, false, MUST_EXIST);
}

$url = new moodle_url('/mod/tables/upload_from_activity.php?
    id='.$id);

$PAGE->requires->js(new moodle_url($CFG->wwwroot . '/mod/
    tables/amd/src/upload.js?v=1.1'));

$PAGE->set_url($url);
$PAGE->set_title(format_string($moduleinstance->name));
$PAGE->set_heading(format_string($course->fullname));

```

```

$PAGE->requires->jquery();

require_login($course, false, $cm);

40 $context = context_module::instance($cm->id);

require_capability('moodle/course:manageactivities',
    $context);

echo $OUTPUT->header();

45 $user_focus_data = $DB->get_record('tables_users_focus',
    array('tableid' => $moduleinstance->id, 'userid' => $USER
    ->id));
if($_POST["sheet"]){
    $active_sheet = $_POST["sheet"];
    $user_focus_data->active_sheet = $active_sheet;
50 }
else{
    $active_sheet = $user_focus_data->active_sheet;
}

55 $sheet = $DB->get_record("tables_sheets", array("id" =>
    $active_sheet), "*", MUST_EXIST);
if($sheet->activityloadtype != ""){
    $type = $sheet->activityloadtype;
}
else{
60     $type = "quiz";
}

if($sheet->updateonreloadpage){
    $checked = "checked";
65 }
else{
    $checked = "";
}

70 if(isset($_POST['activity_type_selector'])){
    $type = $_POST['activity_type_selector'];
}

echo '
75 <form name="select_loading_activity" action="" method="post
    ">

```



```

80 <select class="m-attach-cells-selector" name="
    activity_type_selector" onchange="this.form.submit()">
<option value="quiz" ';if($type == "quiz"){ echo ' selected';
    } echo '>
'.get_string("modulename", "mod_quiz").'
</option>
80 </select>
</form>
<form name="select_loading_activity" action="load_activity.
    php?id='.$id.'" method="post">
<select class="m-attach-cells-selector" name="
    activity_selector">';
85 $activities = get_all_instances_in_course($type, $course);
foreach($activities as $activity){
    if($sheet->activityloadid == $activity->id){
        $selected = "selected";
    }
    else{
90     $selected = "";
    }
    echo '
    <option value="' . $activity->id.'" ' . $selected.'>
    ' . $activity->name.'
95    </option>';
}
echo '
</select>
<input type="checkbox" id="' . $active_sheet.'" ' . $checked.'
    onchange="onchangeUpdate(this)"/>
100 <label for="update">Update</label>
<button class="btn btn-primary" style="border-radius: 5px"
    type="submit" name="upload" value="' . $type.'">'.
    get_string("upload").'</button>
</form>
';
105 echo $OUTPUT->footer();

```

П1.18. load_activity.php

```

<?php

require(__DIR__ . '/../.../config.php');

```

```

5   require_once(__DIR__.'./lib.php');

   global $CFG, $PAGE, $OUTPUT, $DB, $USER;

   // Course module id.
10  $id = optional_param('id', 0, PARAM_INT);
   // Activity instance id.
   $t = optional_param('t', 0, PARAM_INT);

   if ($id) {
15     $cm = get_coursemodule_from_id('tables', $id, 0, false,
        MUST_EXIST);
     $course = $DB->get_record('course', array('id' => $cm->
        course), '*', MUST_EXIST);
     $moduleinstance = $DB->get_record('tables', array('id' =>
        $cm->instance), '*', MUST_EXIST);
   } else {
     $moduleinstance = $DB->get_record('tables', array('id' =>
20     $t), '*', MUST_EXIST);
     $course = $DB->get_record('course', array('id' =>
        $moduleinstance->course), '*', MUST_EXIST);
     $cm = get_coursemodule_from_instance('tables',
        $moduleinstance->id, $course->id, false, MUST_EXIST);
   }

   $PAGE->set_url('/mod/tables/load_activity.php?id='.$id);
25  $PAGE->requires->jquery();

   require_login($course, false, $cm);

   $context = context_module::instance($cm->id);
30  require_capability('moodle/course:manageactivities',
        $context);

   $user_focus_data = $DB->get_record('tables_users_focus',
        array('tableid' => $moduleinstance->id, 'userid' => $USER
        ->id));
   if($_POST["sheet"]){
35     $active_sheet = $_POST["sheet"];
     $user_focus_data->active_sheet = $active_sheet;
   }
   else{
     $active_sheet = $user_focus_data->active_sheet;
40  }

```

```

echo $OUTPUT->header();

$type = $_POST['upload'];
45 $activity_id = $_POST['activity_selector'];

load_from_activity($context, $course, $active_sheet, $type,
    $activity_id);

redirect('view.php?id='.$id);
50

function load_from_activity($context, $course, $active_sheet
    , $type, $activity_id)
{
    global $DB;
    $students = get_enrolled_users($context);
55

    $DB->delete_records('tables_sheets_cells', array('sheetid'
        => $active_sheet));

    $sheet = $DB->get_record('tables_sheets', array ('id' =>
        $active_sheet), '*', MUST_EXIST);
    $sheet->activityloadtype = $type;
60 $sheet->activityloadid = $activity_id;
    $sheet->updateonreloadpage = "true";
    $sheet->timemodified = time();

    $DB->update_record("tables_sheets", $sheet);
65

    switch ($type){
        case "quiz":{
            loadCell($active_sheet, 'A1', get_string('group'), "
                bold");
            loadCell($active_sheet, 'B1', get_string('user'), "
                bold");
70 loadCell($active_sheet, 'C1', get_string('attempt', '
                mod_scom'), "bold");
            loadCell($active_sheet, 'D1', get_string('variant', '
                quiz_statistics'), "bold");
            loadCell($active_sheet, 'E1', get_string('question'),
                "bold");
            loadCell($active_sheet, 'F1', get_string('summary'), "
                bold");
            loadCell($active_sheet, 'G1', get_string('rightanswer',
                , 'mod_tables'), "bold");

```

```

75 loadCell($active_sheet, 'H1', get_string('answer', '
    mod_tables'), "bold");
loadCell($active_sheet, 'I1', get_string('maxfraction',
    , 'mod_tables'), "bold");
loadCell($active_sheet, 'J1', get_string('fraction', '
    mod_tables'), "bold");

80 $s_rows = 2;

foreach ($students as $student){
    loadCell($active_sheet, 'A'.$s_rows, $group_names);
    loadCell($active_sheet, 'B'.$s_rows, $student->
        firstname." ".$student->lastname);

85 $groups = $DB->get_records('groups_members', array('
    userid' => $student->id));
$group_names = "";
foreach ($groups as $group){
    $group_names .= $DB->get_record('groups', array('
        id' => $group->groupid), '*', MUST_EXIST)->name
        ;
    $group_names .= " ";
90 }

$quiz_attempts = quiz_get_user_attempts($activity_id
    , $student->id);
$attempt_number = 1;

95 $have_attempts = false;

foreach ($quiz_attempts as $quiz_attempt){
    if($quiz_attempt->userid == $student->id){
        $have_attempts = true;
100 loadCell($active_sheet, 'C'.$s_rows, "Попытка ".
        $attempt_number);

    $module_id = $DB->get_record('modules', array('
        name' => 'quiz'), '*', MUST_EXIST)->id;
    $quiz_module_instance = $DB->get_record('
        course_modules',
    array('course' => $course->id, 'module' =>
        $module_id, 'instance' => $quiz_attempt->quiz
        ), '*', MUST_EXIST)->id;
105 $quiz_context = $DB->get_record('context',

```

```

array('contextlevel' => $context->contextlevel,
      'instanceid' => $quiz_module_instance), '*',
      MUST_EXIST)->id;

$sql = "SELECT qu.contextid, qa.*, qas.fraction,
             qas.userid, qas.questionattemptid
FROM {question_attempts} qa
JOIN {question_usages} qu
ON qu.id = qa.questionusageid
JOIN {question_attempt_steps} qas
ON qa.id = qas.questionattemptid
WHERE qu.contextid = $quiz_context AND qas.
      userid = $student->id";
115 $q_usages = $DB->get_records_sql($sql);

foreach($q_usages as $q_usage){
    $question_attempts = $DB->get_records('
        question_attempts', array('questionusageid'
            => $q_usage->questionusageid));
    loadCell($active_sheet, 'D'.$s_rows, "Вариант
        ".$q_usage->variant);
120 foreach ($question_attempts as $qa){
    $question_name = $DB->get_record('question',
        array('id' => $qa->questionid), '*',
        MUST_EXIST)->name;
    loadCell($active_sheet, 'E'.$s_rows,
        $question_name);
    loadCell($active_sheet, 'F'.$s_rows, $qa->
        questionsummary);
    loadCell($active_sheet, 'G'.$s_rows, $qa->
        rightanswer);
125 loadCell($active_sheet, 'H'.$s_rows, $qa->
        responsesummary);
    loadCell($active_sheet, 'I'.$s_rows, round(
        $qa->maxfraction, 2));
    $fraction = $DB->get_record('
        question_attempt_steps',
        array('questionattemptid' => $qa->id, '
            sequencenumber' => 2), '*', MUST_EXIST)->
        fraction;

    loadCell($active_sheet, 'J'.$s_rows, round(
        $fraction, 2));
    $s_rows++;
}
130

```

```

    }
    $attempt_number++;
135 }
    else{
        $have_attempts = false;
    }
}
140 if($have_attempts == false){
    loadCell($active_sheet, 'A'.$s_rows, $student->
        firstname." ".$student->lastname);
    loadCell($active_sheet, 'B'.$s_rows, get_string('
        noattempts', 'mod_tables'));
    $s_rows+=2;
}
145 else if($have_attempts == true){
    loadCell($active_sheet, 'A'.$s_rows, get_string('
        sumresult', 'mod_tables'));
    loadCell($active_sheet, 'B'.$s_rows, round(
        $quiz_attempt->sumgrades, 2));
    $s_rows+=2;
}
150 }

    break;
}
}
155 }

```

П1.19. history.php

```

<?php

require_once("../..//config.php");
5 require_once("lib.php");

// Check that all the parameters have been provided.
global $DB, $USER, $CFG, $OUTPUT, $PAGE;

10 require_once($CFG->libdir.'/tablelib.php');

// Course module id.
$id = optional_param('id', 0, PARAM_INT);
// Activity instance id.

```

```

15 $t = optional_param('t', 0, PARAM_INT);

    if ($id) {
        $cm = get_coursemodule_from_id('tables', $id, 0, false,
            MUST_EXIST);
        $course = $DB->get_record('course', array('id' => $cm->
            course), '*', MUST_EXIST);
20 $moduleinstance = $DB->get_record('tables', array('id' =>
            $cm->instance), '*', MUST_EXIST);
    } else {
        $moduleinstance = $DB->get_record('tables', array('id' =>
            $t), '*', MUST_EXIST);
        $course = $DB->get_record('course', array('id' =>
            $moduleinstance->course), '*', MUST_EXIST);
        $cm = get_coursemodule_from_instance('tables',
            $moduleinstance->id, $course->id, false, MUST_EXIST);
25 }
    $active_sheet = $DB->get_record('tables_users_focus', array(
        'tableid' => $moduleinstance->id, 'userid' => $USER->id))
        ->active_sheet;
    if(isset($_POST['switch_sheet'])) {
        $active_sheet = $_POST['switch_sheet'];
    }
30
    $url = new moodle_url('/mod/tables/history.php', array('id'
        => $id));

    $PAGE->set_url($url);
    $PAGE->set_title(format_string($moduleinstance->name));
35 $PAGE->set_heading(format_string($course->fullname));

    $PAGE->requires->jquery();

    require_login($course, false, $cm);
40
    $context = context_module::instance($cm->id);

    require_capability('moodle/course:manageactivities',
        $context);

45
    echo $OUTPUT->header();

    echo '
    <form style="display: inline-block" name="
        select_attach_table" action="" method="post">

```

```

50 <select class="m-attach-cells-selector" name="switch_sheet"
    onchange="this.form.submit()">';
$all_sheets = $DB->get_records('tables_sheets', array('
    tableid' => $moduleinstance->id));
foreach($all_sheets as $sheet){
    echo '
    <option value="'. $sheet->id. '"'; if($active_sheet == $sheet
        ->id){ echo ' selected'; } echo '>
        '.get_string("sheet", "mod_tables")." ". $sheet->name.'
55 </option>';
    }
    echo '
</select>
</form>
60 <form style="display: inline-block" name="clear_history_form"
    " action="clear_history.php?id=' . $id. '" method="post">
    <button class="btn btn-primary m-attach-cells-selector" id="
        clear_history" name="clear_history" type="submit" value="
        '. $active_sheet. '">Clear history</button>
    </form>
    ';

65 if($DB->record_exists('tables_cells_history', array('sheetid
    ' => $active_sheet))){
    $cells_history = $DB->get_records('tables_cells_history',
        array('sheetid' => $active_sheet));

    $table = new table_sql("cell-history-table-{$course->id}")
        ;

70 $table->set_sql('{tables_cells_history}.*, {
    tables_cells_history}.cellname, {user}.firstname AS
    firstname, {user}.lastname AS lastname, {
    tables_cells_history}.content, FROM_UNIXTIME({
    tables_cells_history}.timecreated) as time',
    "{tables_cells_history} JOIN {user} ON {
        tables_cells_history}.userid = {user}.id",
    '{tables_cells_history}.sheetid=' . $active_sheet);

    $table->no_sorting('content');
75 $table->define_columns(array('cellname', 'fullname', '
    content', 'time'));
    $table->define_headers(array(get_string('cellname', '
        mod_tables'), get_string('fullname'),

```



```

    get_string('cellcontent', 'mod_tables'), get_string('
        timemodified', 'mod_tables'))));

    // The name column is a header.
80 $table->define_header_column('fullname');

    // Make this table sorted by last name by default.
    $table->sorttable(true, 'lastname');

85 $table->out(40, true);
}
else{
    echo '<span>'. get_string('cellhistorynone', 'mod_tables')
        . '</span>';
}

90 echo '<script src="//cdn.socket.io/socket.io-1.2.0.js"></
    script>';

echo $OUTPUT->footer();

```

Π1.20. users.php

```

<?php

require_once("../../config.php");
5 require_once("lib.php");

    // Check that all the parameters have been provided.
    global $DB, $USER, $CFG, $OUTPUT, $PAGE;
    // Course module id.
10 $id = optional_param('id', 0, PARAM_INT);
    // Activity instance id.
    $t = optional_param('t', 0, PARAM_INT);

    if ($id) {
15 $cm = get_coursemodule_from_id('tables', $id, 0, false,
        MUST_EXIST);
        $course = $DB->get_record('course', array('id' => $cm->
            course), '*', MUST_EXIST);
        $moduleinstance = $DB->get_record('tables', array('id' =>
            $cm->instance), '*', MUST_EXIST);
    } else {

```

```

    $moduleinstance = $DB->get_record('tables', array('id' =>
        $t), '*', MUST_EXIST);
20    $course = $DB->get_record('course', array('id' =>
        $moduleinstance->course), '*', MUST_EXIST);
    $cm = get_coursemodule_from_instance('tables',
        $moduleinstance->id, $course->id, false, MUST_EXIST);
}

$active_sheet = $DB->get_record('tables_users_focus', array(
    'tableid' => $moduleinstance->id, 'userid' => $USER->id))
->active_sheet;
25 if(isset($_POST['switch_sheet'])){
    $active_sheet = $_POST['switch_sheet'];
}

$url = new moodle_url('/mod/tables/users.php', array('id' =>
    $id));
30

$PAGE->set_url($url);
$PAGE->set_title(format_string($moduleinstance->name));
$PAGE->set_heading(format_string($course->fullname));

35 $PAGE->requires->jquery();
$PAGE->requires->js(new moodle_url($CFG->wwwroot . '/mod/
    tables/amd/src/attach_cells.js?v=3.6'));
$PAGE->requires->js(new moodle_url($CFG->wwwroot . '/mod/
    tables/amd/src/connect_to_websocket.js?v=4.3'));

40 require_login($course, false, $cm);

$context = context_module::instance($cm->id);

require_capability('moodle/course:manageactivities',
    $context);
45

echo $OUTPUT->header();

$selector = "groups";
if(isset($_POST['attach_to'])){
50     $selector = $_POST['attach_to'];
}

echo '
<form name="select_attach_table" action="" method="post">

```

```

55 <select class="m-attach-cells-selector" name="attach_to"
    onchange="this.form.submit()">
    <option value="students";if($selector == "students"){ echo
        ' selected'; } echo'>
    '.get_string("students").'
    </option>
    <option value="groups";if($selector == "groups"){ echo '
        selected'; } echo'>
60 '.get_string("groups").'
    </option>
    </select>
    <select class="m-attach-cells-selector" name="switch_sheet"
        onchange="this.form.submit()">';
    $all_sheets = $DB->get_records('tables_sheets', array('
        tableid' => $moduleinstance->id));
65 foreach($all_sheets as $sheet){
    echo '
        <option value="' . $sheet->id . '"';if($active_sheet == $sheet
            ->id){ echo ' selected'; } echo'>
        '.get_string("sheet", "mod_tables")." " . $sheet->name . '
        </option>';
70 }
    echo '
    </select>
    </form>
    <div id="main_table" data-moduleinstance="' . $moduleinstance
        ->id . '" data-user="' . $USER->id . '">';
75
    switch($selector){
        case 'students':
            {
                $students = get_enrolled_users($context);
80
                echo '
                <table class="m-userbox">
                <thead>
                <tr>
85 <td>'
                . get_string('students') .
                '</td>
                <td>'
                . get_string('email') .
90 <td>'
                . get_string('role') .

```

```

'</td>
<td>'
95 . get_string('group') .
'</td>
<td>'
. get_string('attachedcells', 'mod_tables') .
'</td>
100 </tr>
</thead>
<tbody>';
foreach ($students as $student) {
    $viewableroles = get_viewable_roles($context, $student
        ->id);
105 $roles = get_user_roles_in_course($student->id,
    $course->id);

    if((strpos($roles, $viewableroles[4]) !== false) || (
        strpos($roles, $viewableroles[5]) !== false) || (
        strpos($roles, $viewableroles[6]) !== false) || (
        strpos($roles, $viewableroles[7]) !== false) || (
        strpos($roles, $viewableroles[8]) !== false)){
        if($DB->record_exists('tables_users_cells', array('
            userid' => $student->id, 'sheetid' =>
            $active_sheet))){
            $user_attached_cells = $DB->get_records('
                tables_users_cells', array('userid' => $student
                    ->id, 'sheetid' => $active_sheet));
110 }
        else {
            $user_attached_cells = get_string('
                attachedcellsnoone', 'mod_tables');
        }

        if($DB->record_exists('groups_members', array('
            userid' => $student->id))) {
115 $groups = $DB->get_records('groups_members', array
            ('userid' => $student->id));
            foreach ($groups as $group){
                $group_names = $DB->get_record('groups', array('
                    id' => $group->groupid), '*', MUST_EXIST)->
                    name;
                $group_names .= " </br>";
120 }
            }
        }
    }
    else{

```

```

    $group_names = get_string('groupsnone');
}

125
echo '<tr>
<td>' . $student->firstname . " " . $student->
    lastname . '</td>
<td>' . $student->email . '</td>
<td>' . $roles . '</td>
130
<td>' . $group_names . '</td>
<td id="' . $student->id . '">';
$attached_cells = $DB->get_records('
    tables_users_cells', array('sheetid' =>
        $active_sheet, 'userid' => $student->id));
foreach ($attached_cells as $attached_cell){
    echo '
135
    <span id="' . $attached_cell->id . '">' . $attached_cell
        ->cellname . ' <span class="m-tables-blue-btn"><i
            class="fa fa-trash" data-sheet="' .
                $active_sheet . '" data-attached-to="student"
                data-attached-cell="' . $attached_cell->id . '"
                data-attached-id="' . $student->id . '" data-update
                ="delete_cell" onclick="deleteAttachedCell(this
                )"></i></span></span>
    ' ;
}
if(count($attached_cells) != 0){
    echo '<span>Удалить все
140
    <span class="m-tables-red-btn">
    <i class="fa fa-trash" data-sheet="' . $active_sheet
        . '" data-attached-to="student" data-attached-
        cell="" data-attached-id="' . $student->id . '"
        data-update="delete_all_cells" onclick="
            deleteAttachedCell(this)"></i>
    </span>
    </span>';
}
145
echo '
</td>
</tr>';
}

150
}
echo '</tbody>
</table>';
break;

```

```

}
155 case 'groups':
{
    $groups = groups_get_activity_allowed_groups($cm);

    echo '
160 <table class="m-userbox">
    <thead>
    <tr>
    <td>'
    . get_string('groups') .
165 ' </td>
    <td>'
    . get_string('name') .
    ' </td>
    <td>'
170 . get_string('attachedcells', 'mod_tables') .
    ' </td>
    </tr>
    </thead>
    <tbody>';
175 foreach ($groups as $group) {
    if($DB->record_exists('tables_groups_cells', array('
        groupid' => $group->id, 'sheetid' => $active_sheet)
    )){
        $group_attached_cells = $DB->get_records('
            tables_groups_cells', array('groupid' => $group->
            id, 'sheetid' => $active_sheet));
    }
    else {
180 $group_attached_cells = get_string('
        attachedcellsnoone', 'mod_tables');
    }

    echo '<tr>
    <td>' . $group->name . ' </td>
185 <td>';
    $attached_cells = $DB->get_records('
        tables_groups_cells', array('sheetid' =>
        $active_sheet, 'groupid' => $group->id));
    foreach ($attached_cells as $attached_cell){
        echo '
        <span id="' . $attached_cell->id . '">' . $attached_cell->
        cellname . ' <span class="m-tables-blue-btn"><i
        class="fa fa-trash" data-sheet="' . $active_sheet . '

```

```

        " data-attached-to="group" data-attached-cell="' .
        $attached_cell->id.'" data-attached-id="' . $group
        ->id.'" data-update="delete_cell" onclick="
        deleteAttachedCell(this)"></i></span></span>
190     ' ;
    }
    if(count($attached_cell) != 0){
        echo '<span id="delete_all_' . $group->id.'">Удалить вс
        e
        <span class="m-tables-red-btn">
195     <i class="fa fa-trash" data-sheet="' . $active_sheet .'
        " data-attached-to="group" data-attached-cell="'
        data-attached-id="' . $group->id.'" data-update="
        delete_all_cells" onclick="deleteAttachedCell(
        this)"></i>
        </span>
        </span>' ;
    }
    echo '</td>
200 </tr>' ;

    }
    echo ' </tbody>
    </table>' ;
205 break;
    }
}

echo '<script> let messages = ["' . get_string('
    alertselectcellss', 'mod_tables')."'] </script>
210 <script src="//cdn.socket.io/socket.io-1.2.0.js"></script>
    <script>let socket = io("' . $moduleinstance->wsserver.'" )</
    script>
    ' ;

echo $OUTPUT->footer();

```


Код разработанного веб-сокета

П2.1. socketio.js

```
var app = require('express')();

const opts = {
  5   key: fs.readFileSync('wsserver_na4u_ru.key'),
      cert: fs.readFileSync('wsserver_na4u_ru.crt')
}

var http = require('https').createServer(opts, app);
10 var socket = require('socket.io')(http);

var port = process.env.APP_PORT;

var customer = {
15   queue: [],
      chat: [],
      quit: [],
      total: 0
};

20
socket.on('connection', function(client){
  var username = false;
  var isStaff = false;
  client.on('login', function(data) {
25     username = data.username;
        isStaff = data.isStaff;
  });
  client.on('subscribe', function(data) {
30     if (data.room == "staff_lobby")
        {
            // staff subscribe
        }
        else
        {
35           // customer subscribe
            if (! isStaff)
            {
                var exists = false;
```

```

40     for (var i=customer.queue.length-1; i>=0; i--) {
        if (customer.queue[i].username === username)
        {
            exists = true;
            customer.queue[i].join_at = new Date().
                toISOString();
            break;          //<-- Uncomment if only the first
                           term has to be removed
45     }
    }

    if (exists === false)
    {
50        customer.total++;
        customer.queue.push({
            id: customer.total,
            username: username,
            room: data.room,
55            status: 'new',
            join_at: new Date().toISOString()
        });
    }
}
60 else
{
    // if staff will update queue is talked
    for (var i=customer.queue.length-1; i>=0; i--) {
        if (customer.queue[i].username === data.room)
65        {
            customer.queue[i].status = "staff talked";
            break;          //<-- Uncomment if only the first
                           term has to be removed
        }
    }
70 }

    client.join(data.room);
    if (! isStaff)
75    {
        console.log(username + " start private chat. You are
            "+ customer.total + ".")
    }
    else
    {

```

```

80         console.log(username + " start private chat. You are
           "+ customer.total + ".")
        }
        socket.in("staff_lobby").emit('new_user', {});

        }

85     console.log('joining room', username, data.room);
  });

  client.on('unsubscribe', function(data) {

90      // if staff will update queue is talked
      for (var i=customer.queue.length-1; i>=0; i--) {
          if (customer.queue[i].username === data.room)
          {
95              customer.queue[i].status = "out";
              break;          //<-- Uncomment if only the first term
                             has to be removed
          }
      }

100     // remove from queue
      for (var i=customer.queue.length-1; i>=0; i--) {
          if (customer.queue[i].username === username)
          {
105              customer.queue.splice(i, 1);
              break;          //<-- Uncomment if only the first term
                             has to be removed
          }
      }

      console.log(username + ' leaving room', data.room);
110     client.leave(data.room);
  });

  client.on('send', function(data) {
      console.log('sending message');
115     socket.in(data.room).emit('message', {
        sender: username,
        message: data.message,
        send_at: new Date().toISOString(),
        isStaff: isStaff
120     });
  });
  });

```

```

    });

    app.get('/customer/queue', function(req, res) {
125       res.json(customer.queue);
    });

    http.listen(process.env.APP_PORT, process.env.APP_IP,
        function(){
130       console.log('listening on *:' + process.env.APP_PORT);
    });

```

П2.2. amd/src/connect_to_websocket.js

```

$(document).ready(function() {
    if(typeof socket !== 'undefined'){
        let username = document.getElementById('main_table').
            getAttribute('data-user');
5       let room = document.getElementById('main_table').
            getAttribute('data-moduleinstance');
        let isStaff = "false";

        socket.emit('login', {username: username, isStaff:
            isStaff});
        socket.emit('subscribe', {room: room});

10       $( window ).on( "unload", function() {
            socket.emit('unsubscribe', {room: room});

            let data = {
15               update_type: "focusout",
               cell_id: document.getElementById("prev_element").
                   value
            };
            socket.emit('send', { room: document.getElementById('
                main_table').getAttribute('data-moduleinstance'),
                message: data});
        } );

20       socket.on('message', function(message){
            if(message.sender !== document.getElementById('
                main_table').getAttribute('data-user')){
                let style = "";
                let cell;

```

```

25     let data = message.message;
    //console.log(data)

    switch(data.update_type){
    case "input":
30     if(localStorage[data.cell_id] === undefined){
        if(localStorage.activity_role === 'teacher'){
            localStorage[data.cell_id] = 'teacher';
        }
        else{
35         localStorage[data.cell_id] = data.
            cell_visibility;
        }
    }

    if(localStorage.activity_role === 'teacher'){
40     cell = document.getElementById(data.cell_id);
        cell.value = data.cell_content;
    }
    else if(localStorage[data.cell_id] === data.
        cell_visibility){
45     cell = document.getElementById(data.cell_id);
        cell.value = data.cell_content;
    }
    break;
    case "visibility":
        localStorage[data.cell_id] = data.cell_visibility;
50     location.reload();
    break;
    case "focusin":
        cell = document.getElementById(data.cell_id);

55     cell.setAttribute("disabled", "");
        cell.removeAttribute("class", "resizable");
        break;
    case "focusout":
        if(data.cell_id !== ""){
60     cell = document.getElementById(data.cell_id);
        let attached_cell = cell.getAttribute('data-
            attached');

        if(attached_cell === '1' || document.
            getElementById('main_table').getAttribute('
            data-user-role') === 'teacher'){
            try{

```

```

65         cell.removeAttribute("disabled");
           cell.setAttribute("class", "resizable");
       }
       catch (e){
           console.log("No focused cell")
70           console.log(e)
       }
       break;
   }
}
75 break;
case "attach_cells":
if(document.getElementById('main_table').
    getAttribute('data-user') === data.user_id){
    location.reload();
}
80 break;
case "delete_cell":
if(document.getElementById('main_table').
    getAttribute('data-user') === data.user_id){
    location.reload();
}
85 break;
case "delete_all_cells":
console.log('deleted')
if(document.getElementById('main_table').
    getAttribute('data-user') === data.user_id){
    location.reload();
90 }
break;
case "resize_h":
let row = document.getElementById(data.name);
style = style.concat("height:", data.height, "px;"
);
95 row.setAttribute("style", style);
break;
case "resize_w":
let column = document.getElementById(data.name);
style = style.concat("width:", data.width, "px;");
100
column.setAttribute("style", style);
break;
case "font":
switch(data.button_type){
105     case "font-family-selector":

```

```

document.getElementById(data['cell_id']).style.
    fontFamily = data.input_content;
break;
case "font-size-selector":
document.getElementById(data['cell_id']).style.
    fontSize = data.input_content.concat("pt");
110 break;
case "font-bold-button":
document.getElementById(data['cell_id']).style.
    fontWeight = data.input_content;
break;
case "font-italic-button":
115 document.getElementById(data['cell_id']).style.
    fontStyle = data.input_content;
break;
case "font-underline-button":
document.getElementById(data['cell_id']).style.
    textDecoration = data.input_content;
break;
120 case "text-left-button":
document.getElementById(data['cell_id']).style.
    textAlign = data.input_content;
break;
case "text-center-button":
document.getElementById(data['cell_id']).style.
    textAlign = data.input_content;
125 break;
case "text-right-button":
document.getElementById(data['cell_id']).style.
    textAlign = data.input_content;
break;
    }
130 break;
    }
    }
    });
    }
135 else{
    console.log("Error something with wssocket")
    alert("Error something with wssocket")
    }
});

```